

Docker, Kubernetes, GKE & GCP Artifact Registry: Complete Beginner to Advanced Guide

Prodapt-AID Practice

August 2026

Contents

About This Guide	5
Chapter 1: Introduction	6
What Is a Container, Really?	6
Why Docker Became the Standard	6
Why You Need Orchestration	7
Why GKE Specifically	7
Why a Dedicated Image Registry Matters	7
Roadmap for This Book	8
Key Takeaways	8
Chapter 2: Docker Fundamentals	9
Core Vocabulary	9
Anatomy of a Dockerfile	9
Layers and the Build Cache	10
Core Day-to-Day Commands	10
Key Takeaways	11
Chapter 3: Writing Production-Grade Dockerfiles	13
Multi-Stage Builds	13
Choosing Minimal Base Images	14
.dockerignore	14
Running as a Non-Root User	15
The HEALTHCHECK Instruction	15
Image Tagging Strategy	16
Key Takeaways	16
Chapter 4: Docker Networking & Volumes	17
The Default Bridge Network	17
Container-to-Container Communication	17
Volumes, Bind Mounts, and tmpfs	18
A Brief Look at Docker Compose	19
Key Takeaways	19

Chapter 5: Docker Registries & Image Distribution	20
What Is a Registry, and Why Docker Hub?	20
Image Naming Anatomy	20
Tags vs. Digests	20
Logging In, Pushing, and Pulling	21
Public vs. Private Registries	22
Key Takeaways	22
Chapter 6: GCP Artifact Registry Fundamentals	24
What Is Artifact Registry?	24
Creating a Repository	24
Image Path Structure	25
Core IAM Roles	26
Key Takeaways	27
Chapter 7: Authenticating Docker to Artifact Registry	28
Configuring the Docker CLI	28
The Three Authentication Paths	28
Granting Push Access with IAM	30
Key Takeaways	30
Chapter 8: Building & Pushing Images to Artifact Registry	31
Manual Build, Tag, and Push	31
Automating with Cloud Build	32
Tagging Strategy for CI	33
Key Takeaways	34
Chapter 9: Vulnerability Scanning & Artifact Registry Security	35
Automatic Scanning on Push	35
Viewing Scan Results	35
Gating Deployments on Scan Results	36
Cleanup Policies	37
Customer-Managed Encryption Keys (CMEK)	38
Key Takeaways	38
Chapter 10: Kubernetes Fundamentals	39
The Problem: More Containers Than One Engineer Can Manage	39
The Two Halves of a Cluster	39
Declarative, Not Imperative	41
Talking to Kubernetes: kubectl	42
Key Takeaways	43
Chapter 11: Pods, Deployments & ReplicaSets	44
The Pod: Kubernetes' Smallest Deployable Unit	44
Why You Need More Than Just a Pod	44
The Deployment: Your Primary Tool for Running Applications	44
The ReplicaSet: The Layer You Rarely Touch Directly	46
Rolling Updates and Rollbacks	47
Self-Healing in Action	47
Key Takeaways	48
Chapter 12: Services & Ingress Networking	49
The Problem: Pods Are Disposable, But Traffic Needs a Stable Target	49
The Three Core Service Types	49
A Complete Service Manifest	50
DNS-Based Service Discovery	50
Routing External Traffic: The Ingress	51

Checking Your Work	52
Key Takeaways	53
Chapter 13: ConfigMaps, Secrets, Volumes & Storage	54
ConfigMaps: Non-Sensitive Configuration	54
Secrets: Sensitive Data	54
Consuming ConfigMaps and Secrets: Environment Variables vs. Mounted Files	55
Storage: PersistentVolumes, PersistentVolumeClaims, and StorageClasses	56
Bringing It Together	57
Key Takeaways	57
Chapter 14: Introduction to GKE	59
What GKE Adds on Top of Open-Source Kubernetes	59
Autopilot vs. Standard Mode	59
Creating a Cluster	60
Node Pools in Standard Mode	60
Key Takeaways	61
Chapter 15: Deploying to GKE from Artifact Registry	63
Step 1: Build and Push the Image	63
Step 2: Write the Deployment and Service Manifest	63
Step 3: Apply the Manifest	64
Step 4: Verify the Deployment	64
How GKE Nodes Authenticate to Artifact Registry	65
Key Takeaways	66
Chapter 16: GKE Networking	67
VPC-Native Clusters and Alias IP Ranges	67
Exposing Services: GKE Ingress and the Gateway API	67
Restricting Pod-to-Pod Traffic with Network Policies	68
Key Takeaways	69
Chapter 17: GKE Identity & Security	70
Workload Identity: Pods Authenticating to GCP APIs	70
IAM and RBAC Working Together	71
Binary Authorization: Gating Deployment on Trust	72
Pod Security Basics	72
Key Takeaways	73
Chapter 18: Autoscaling in GKE	74
Horizontal Pod Autoscaler (HPA)	74
Vertical Pod Autoscaler (VPA)	75
Cluster Autoscaler: Standard Mode	75
Autopilot's Fully Automatic Node Management	76
Node Auto-Provisioning (Standard Mode)	76
Key Takeaways	76
Chapter 19: CI/CD: Docker to Artifact Registry to GKE	77
The Big Picture: One Push, One Pipeline	77
Why the Vulnerability Scan Is a Gate, Not a Report	77
Authenticating CI/CD to Google Cloud: Workload Identity Federation	79
A Complete GitHub Actions Workflow	79
Key Takeaways	81
Chapter 20: Monitoring, Logging & Observability	82
Cloud Monitoring and Cloud Logging Are On by Default	82
Probes: Teaching Kubernetes What "Healthy" Means	82
Checking Resource Usage Day to Day	85

The Built-In GKE Monitoring Dashboard	85
Key Takeaways	85
Chapter 21: Troubleshooting Guide	87
ImagePullBackOff / ErrImagePull	87
CrashLoopBackOff	87
Pending Pods That Never Get Scheduled	88
A Service That Is Unreachable	88
An Ingress That Never Provisions an External IP	89
Debugging Toolkit	89
Key Takeaways	90
Chapter 22: Production Best Practices & Enterprise Architecture	91
Architecture Overview	91
Namespaces for Isolation	91
Resource Requests, Limits & Quality of Service	93
Multi-Environment Strategy	94
Production Considerations	95
Key Takeaways	96
Chapter 23: Interview Questions	97
Docker Interview Questions (1-16)	97
Kubernetes Interview Questions (1-16)	100
GKE & Artifact Registry Interview Questions (1-16)	103
Chapter 24: Commands Cheat Sheet	108
Docker Commands	108
kubectl Commands	109
gcloud Commands (Artifact Registry & GKE)	110
Chapter 25: Summary	112
Key Takeaways	112
A Suggested Learning Roadmap	112
Recommended Certifications	113
A Closing Note on Cost	113

From Your First Container to Production-Grade GKE Architecture

A practical, hands-on guide for Cloud Engineers, DevOps Engineers, Platform Engineers, and Solution Architects

Prepared by: **Prodapt-AID Practice**
Edition: **2026**

About This Guide

This guide takes you from zero container experience to designing and operating production-grade Kubernetes workloads on Google Kubernetes Engine (GKE), backed by Docker and GCP Artifact Registry. It is written for five audiences at once — complete beginners, Cloud Engineers, DevOps Engineers, Platform Engineers, and Solution Architects — so every concept is first explained simply, then reinforced with a real, runnable example.

How to use this guide:

TIP

If you are brand new to containers, read Chapters 1-9 in order — they cover Docker and Artifact Registry before Kubernetes is introduced. If you already know Docker, jump to Chapter 10 for Kubernetes and GKE. If you are preparing for an interview, jump straight to Chapter 23.

Every chapter follows the same pattern: a plain-language explanation, an architecture diagram where useful, a complete and runnable example (Dockerfile, Kubernetes manifest, or `gcloud/kubectl` command), a clear explanation of what the code does, and — where applicable — a troubleshooting and best-practices section.

Examples use illustrative values such as project ID `my-gcp-project-123456`, region `us-central1`, Artifact Registry repository `my-repo`, and GKE cluster name `my-gke-cluster`. Replace these with your own values before running any command.

Chapter 1: Introduction

If you have ever heard a developer say “it works on my machine” right before an application broke in production, you have already met the problem this book exists to solve. Software behaves differently depending on the operating system, installed libraries, environment variables, and a dozen other details that are easy to overlook. Containers were invented to make that problem disappear, and this book will take you from a complete beginner’s understanding of containers all the way to running production workloads on Google Kubernetes Engine (GKE) with images stored in Google Cloud’s Artifact Registry.

What Is a Container, Really?

A container is a lightweight, isolated environment that packages an application together with everything it needs to run: the code, the runtime, system libraries, and configuration. Unlike a traditional installation, a container does not depend on whatever happens to already be installed on the host machine. If it runs on your laptop, it will run the same way on a colleague’s laptop, in a test environment, and in production, because the container carries its own self-contained filesystem and dependencies with it.

It helps to compare containers with virtual machines (VMs), since both are ways of isolating workloads.

- A virtual machine virtualizes an entire computer, including its own kernel and operating system. Starting a VM means booting an operating system from scratch, which typically takes tens of seconds to minutes and consumes gigabytes of disk space and a dedicated slice of memory.
- A container shares the host machine’s operating system kernel and only isolates the process, filesystem, and network namespace for the application. Starting a container is closer to starting a regular process, which typically takes a fraction of a second and consumes only the memory the application itself needs.

This difference has enormous practical consequences. Because containers are so much lighter than VMs, you can run far more of them on the same hardware, which is why containers enabled the shift toward microservices architectures where an application is split into many small, independently deployable services. And because a container image bundles the exact runtime and dependencies an application was built and tested with, moving that image from a developer’s laptop to a staging environment to production carries none of the “it works on my machine” risk.

NOTE

A container is not a “lightweight VM.” It has no kernel of its own — it is an isolated process running on the host’s existing kernel, which is precisely why it starts so fast and uses so little overhead.

Why Docker Became the Standard

Container technology (Linux namespaces and cgroups) existed before Docker, but it was difficult to use directly. Docker, released in 2013, wrapped these low-level kernel features in a simple, consistent developer experience: a Dockerfile to describe how to build an image, a single `docker build` command to produce it, and a single `docker run` command to start it anywhere Docker was installed. Docker also popularized the idea of a container registry — a place to push a built image and pull it down again on any other machine, which is what made containers practical to share across teams and deploy to remote servers.

Docker’s simplicity and its openness (the underlying image format was standardized as the Open Container Initiative, or OCI, specification) meant that essentially every cloud provider, CI/CD tool, and orchestration platform adopted compatible tooling. Today, “containerizing” an application almost always means writing a Dockerfile and building an OCI-compliant image, even if the image later runs under a different container runtime.

Why You Need Orchestration

Running one container on one machine with `docker run` is simple. But real applications rarely stay that simple for long. Consider what happens as an application grows:

- You need multiple replicas of the same service for reliability and to handle load.
- Those replicas need to be spread across multiple machines so a single hardware failure does not take down the whole application.
- When a container crashes, something needs to notice and restart it automatically.
- When you deploy a new version, you want it rolled out gradually, with the ability to roll back if something goes wrong.
- Services need to find and talk to each other reliably even as individual containers come and go.
- You need to scale the number of running containers up and down based on demand.

Doing all of this by hand, with scripts and manual `docker run` commands across a fleet of servers, does not scale past a handful of containers. This is the problem that **container orchestration** solves, and Kubernetes has become the dominant orchestration platform for exactly this reason. Kubernetes takes a declarative description of what you want (“run three replicas of this image, expose them on this port, restart failed containers”) and continuously works to make reality match that description, across as many machines as you give it.

TIP

A useful mental model: Docker answers “how do I package and run one container?” while Kubernetes answers “how do I run and manage many containers, reliably, across many machines?” You need both, and they solve different layers of the same problem.

Why GKE Specifically

Kubernetes itself is powerful but operationally demanding — someone has to install it, patch it, upgrade it, and keep its control plane (the brain of the cluster) highly available. Google Kubernetes Engine removes that burden by offering Kubernetes as a managed service on Google Cloud Platform (GCP). A few reasons GKE stands out among managed Kubernetes offerings:

- **Google-managed control plane.** Google runs, patches, upgrades, and monitors the Kubernetes control plane for you, including automatic failover across zones. You focus on your workloads, not on keeping `etcd` and the API server healthy.
- **Autopilot vs. Standard mode.** GKE Autopilot manages the underlying nodes for you as well — you simply describe your workloads, and Google provisions and right-sizes the compute automatically, billing per pod rather than per node. GKE Standard mode gives you full control over node pools, machine types, and node-level configuration, which is preferable when you need fine-grained tuning or specialized hardware. Chapter 8 covers this choice in depth.
- **Deep GCP integration.** GKE workloads can authenticate using Google Cloud IAM, GKE clusters plug directly into Google Cloud’s VPC networking and Cloud Load Balancing, and logs and metrics flow automatically into Cloud Logging and Cloud Monitoring. This tight integration means less custom glue code compared to running Kubernetes yourself on raw virtual machines.
- **Security defaults and node auto-upgrade.** GKE was Google’s original Kubernetes offering (Kubernetes itself grew out of Google’s internal Borg system), and it tends to receive new upstream Kubernetes features early, along with sensible hardened defaults.

Why a Dedicated Image Registry Matters

Once you have built a container image, it needs to live somewhere that Kubernetes (or any other machine) can pull it from. You could technically copy image files around manually, but that is fragile and does not scale. A **container registry** is a dedicated service for storing, versioning, and serving container images by name and tag, such as `us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1`. A good registry also handles access control, so only authorized users and services can push or pull images, and vulnerability scanning, so you know if an image contains a known security flaw.

On Google Cloud, the current recommended registry is **Artifact Registry**. It succeeds the older Container Registry (`gcr.io`) and unifies storage for container images alongside other package formats such as Maven, npm, Python, and Go modules, all under one service with consistent IAM-based permissions. Google has deprecated Container Registry in favor of Artifact Registry, so every example in this book uses Artifact Registry as the destination for images.

BEST PRACTICE

If you are starting a new project today, use Artifact Registry rather than the legacy `gcr.io` Container Registry. It is the actively developed, fully supported option on GCP going forward.

Roadmap for This Book

This book is organized to build your knowledge in layers, each one depending on the last:

1. **Docker fundamentals** (Chapters 2–3) — images, containers, Dockerfiles, and how to write lean, secure, production-grade Dockerfiles.
2. **Docker registries and GCP Artifact Registry** (Chapters 4) — pushing and pulling images, authentication, and repository management on GCP.
3. **Kubernetes fundamentals** (Chapters 5–6) — pods, deployments, services, and the core objects you will use constantly.
4. **GKE specifics** (Chapters 7–8) — cluster creation, Autopilot vs. Standard, node pools, and GKE networking and IAM integration.
5. **Production architecture** (Chapter 9) — how these pieces fit together in a real, scalable, secure system on GCP.
6. **Troubleshooting** (Chapter 10) — diagnosing the most common Docker and GKE failures.
7. **Interview questions** (Chapter 11) — a study aid for job interviews covering everything in the book.
8. **Commands cheat sheet** (Chapter 12) — a fast reference for the Docker, `kubectl`, and `gcloud` commands used throughout.

By the end, you should be comfortable writing a Dockerfile for a real application, pushing its image to Artifact Registry, and deploying it to a GKE cluster with confidence about why each step exists — not just how to type it.

Key Takeaways

- Containers package an application with its dependencies for consistent behavior everywhere; unlike VMs, they share the host kernel, so they start faster and pack more densely onto the same hardware.
- Docker succeeded because it made building, running, and sharing containers simple and standardized, which is why “containerize it” almost always means “write a Dockerfile” today.
- Orchestration (Kubernetes) becomes necessary once you have more containers than you can reliably manage by hand — it automates scaling, healing, rollout, and service discovery.
- GKE removes the operational burden of running Kubernetes yourself by managing the control plane and integrating tightly with GCP IAM, networking, and observability, with a choice between hands-off Autopilot and fully configurable Standard mode.
- Artifact Registry is GCP’s current, unified image and package registry, replacing the older Container Registry (`gcr.io`), and is the destination used throughout this book.

Chapter 2: Docker Fundamentals

Before writing a single production Dockerfile, you need a solid grip on Docker’s core vocabulary and everyday commands. This chapter covers the building blocks: images, containers, Dockerfiles, and registries, how a Dockerfile turns into a running application, and the commands you will type dozens of times a day once you start working with Docker seriously.

Core Vocabulary

It is easy to use these four terms loosely, but they mean specific, distinct things:

- **Image.** A read-only template that contains an application’s filesystem: its code, runtime, libraries, and default configuration. An image is not running anything by itself — think of it as a snapshot or a blueprint.
- **Container.** A running (or stopped) instance of an image. When you start a container, Docker adds a thin writable layer on top of the image’s read-only layers and runs the process defined by the image. You can start many containers from the same image, and each one is isolated from the others.
- **Dockerfile.** A plain-text file containing step-by-step instructions for building an image — “start from this base, copy in these files, run this install command, and start the app with this command.”
- **Registry.** A server that stores and distributes images by name and tag, so that any machine (a teammate’s laptop, a CI pipeline, a GKE node) can pull down the exact image that was built, rather than rebuilding it or copying files by hand.

A simple way to connect these: you write a **Dockerfile**, use it to build an **image**, push that image to a **registry**, and then run one or more **containers** from it anywhere Docker (or Kubernetes) is running.

Anatomy of a Dockerfile

A Dockerfile is read from top to bottom, and each instruction adds a new layer to the resulting image. Here is a complete, simple Dockerfile for a small Node.js web application:

```
FROM node:20-slim

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm install --production

COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

Let’s walk through each instruction:

- **FROM node:20-slim** — every Dockerfile starts with a base image to build on top of. Here we start from an official Node.js 20 image based on a slimmed-down Debian, rather than installing Node.js from scratch ourselves.
- **WORKDIR /app** — sets the working directory inside the image for all instructions that follow. If /app does not exist yet, Docker creates it. This avoids repeating full paths in every subsequent instruction.
- **COPY package.json package-lock.json ./** — copies just the dependency manifest files from your build context (the folder you are building from) into the image. Notice this happens before copying the rest of the source code — more on why in the next section.

- `RUN npm install --production` — executes a command at build time, inside the image, installing the dependencies listed in `package.json`. `RUN` instructions are how you install packages, compile code, or perform any other one-time setup step.
- `COPY . .` — copies the rest of the application source code into the image, now that dependencies are already installed.
- `EXPOSE 3000` — documents that the application inside the container listens on port 3000. This is purely informational metadata for anyone reading the image — it does not actually publish the port; that happens with `docker run -p`.
- `CMD ["node", "server.js"]` — specifies the default command to run when a container starts from this image.

NOTE

`CMD` and `ENTRYPOINT` are often confused. `ENTRYPOINT` defines the fixed executable that always runs when the container starts, while `CMD` supplies default arguments that can be overridden at `docker run` time. If you define only `CMD`, the whole command is replaceable; if you define `ENTRYPOINT`, anything passed after the image name on `docker run` is appended as arguments to it instead of replacing it. Many production images combine both: an `ENTRYPOINT` for the fixed binary and a `CMD` for its default arguments.



Figure 1: The Docker workflow: a Dockerfile is built into an image, which is then run as a container.

The diagram above shows the full lifecycle: you author a Dockerfile, `docker build` turns it into an image, and `docker run` starts a container from that image. Everything else in this chapter is about doing each of those steps well.

Layers and the Build Cache

Every instruction in a Dockerfile (`FROM`, `COPY`, `RUN`, and so on) produces a **layer**, and Docker caches each layer. When you rebuild an image, Docker checks each instruction in order: if the instruction and everything that affects it (like the files being copied) are unchanged since the last build, Docker reuses the cached layer instead of re-executing it. The moment one instruction's inputs change, that layer and every layer after it must be rebuilt, even if they themselves did not change.

This is precisely why the Dockerfile above copies `package.json` and `package-lock.json` and runs `npm install` *before* copying the rest of the source code with `COPY . .`. Application source code changes constantly, but dependency lists change rarely. If you copied everything in one step and then ran `npm install`, then every single code change — even editing one line in one file — would invalidate the cache for the dependency-install layer, forcing a full reinstall of every package on every build. By separating the two `COPY` steps, a typical rebuild only re-executes the fast final `COPY . .` step and skips the slow `npm install` step entirely, because its layer is still valid.

BEST PRACTICE

Order Dockerfile instructions from least-frequently-changing to most-frequently-changing. Dependency manifests and installation steps belong near the top; your actual application source belongs near the bottom.

Core Day-to-Day Commands

Once an image is built, these are the commands you will reach for constantly.

```
# Build an image from the Dockerfile in the current directory, tagging it "myapp:v1"
docker build -t myapp:v1 .

# Run a container from that image, mapping host port 8080 to container port 3000
docker run -d -p 8080:3000 --name myapp-container myapp:v1

# List currently running containers
docker ps

# List all containers, including stopped ones
docker ps -a

# Open an interactive shell inside a running container, useful for debugging
docker exec -it myapp-container /bin/bash

# View the logs a container has produced
docker logs myapp-container

# Follow logs continuously, like "tail -f"
docker logs -f myapp-container

# Stop a running container gracefully
docker stop myapp-container

# Remove a stopped container
docker rm myapp-container
```

A few points worth highlighting:

- `docker build -t myapp:v1 .` — the `-t` flag assigns a name and tag to the resulting image; the trailing `.` tells Docker where to find the build context (the Dockerfile and the files it references).
- `docker run -d -p 8080:3000` — `-d` runs the container in detached (background) mode, and `-p HOST:CONTAINER` publishes a container port to a port on the host machine, so external traffic on host port 8080 reaches the application listening on container port 3000.
- `docker exec` is invaluable for troubleshooting: it lets you get a shell (or run any single command) inside an already-running container without restarting it.
- `docker stop` sends a graceful shutdown signal before force-killing the container, while `docker rm` deletes a stopped container's writable layer entirely — the underlying image is untouched.

TIP

Running `docker ps -a` and `docker rm` on old, stopped containers regularly (or using `docker container prune`) keeps your local machine from accumulating clutter, since stopped containers still consume disk space until removed.

Key Takeaways

- Images are static templates, containers are running instances of images, Dockerfiles describe how to build images, and registries store and distribute images.
- `CMD` sets an overridable default command; `ENTRYPOINT` sets a fixed executable — production images often combine both.
- Docker caches each Dockerfile instruction as a layer, so ordering instructions from least- to most-frequently-changing (dependencies before source code) dramatically speeds up rebuilds.
- `docker build`, `docker run`, `docker ps`, `docker exec`, `docker logs`, `docker stop`, and `docker rm` form the core toolkit you will use every day when working with containers.
- Understanding the build-then-run flow (Dockerfile to image to container) is the foundation everything later in this book — multi-stage builds, Kubernetes pods, GKE deployments — builds

on top of.

Chapter 3: Writing Production-Grade Dockerfiles

The Dockerfile in Chapter 2 is enough to get an application running, but it is not the kind of Dockerfile you want deployed to production. A production-grade image needs to be small, secure, predictable, and observable. This chapter covers five practices that turn a working Dockerfile into a production-ready one: multi-stage builds, minimal base images, `.dockerignore`, running as a non-root user, and health checks, finishing with a note on how to tag images sensibly.

Multi-Stage Builds

A common problem with simple Dockerfiles is that the tools needed to *build* an application (compilers, build-time dependencies, source code, package caches) end up baked permanently into the final image, even though the running application never needs them. This bloats the image and increases its attack surface, since every extra tool or library is one more thing that could contain a vulnerability.

Multi-stage builds solve this by letting a Dockerfile use multiple `FROM` instructions, each starting a new stage, where later stages can selectively copy only specific files from earlier stages. Everything else from the build stage — the compiler, source files, intermediate artifacts, package manager caches — is left behind.

Here is a complete example for a Go application, where compiling the binary requires the full Go toolchain, but running it does not:

```
# ---- Build stage ----
FROM golang:1.22 AS builder

WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 go build -o /out/app .

# ---- Runtime stage ----
FROM gcr.io/distroless/static-debian12

WORKDIR /app
COPY --from=builder /out/app .

USER nonroot:nonroot
EXPOSE 8080

ENTRYPOINT ["/app/app"]
```

- The first stage, named `builder`, uses the full `golang:1.22` image (over 800 MB) to compile a statically linked binary.
- `COPY --from=builder /out/app .` in the second stage reaches back into the first stage and copies out only the compiled binary — none of the Go toolchain, source code, or module cache comes along.
- The final image is based on `gcr.io/distroless/static-debian12`, an extremely minimal runtime image (discussed below), so the shipped image contains essentially nothing but the binary itself and its runtime user.

The size difference is dramatic in practice: a build-stage image measured in hundreds of megabytes commonly produces a final runtime image of just a few megabytes to a few tens of megabytes. A smaller image means faster pulls onto GKE nodes, faster pod startup, and — just as importantly — far fewer packages and libraries that could carry a known vulnerability.

BEST PRACTICE

Use multi-stage builds by default for any compiled language (Go, Java, Rust, C++) and for any application with a heavy build toolchain (webpack bundling for a frontend app, for example). Only the final runnable artifact belongs in the last stage.

Choosing Minimal Base Images

The base image you start FROM has an outsized effect on final image size and security. Common choices, roughly from largest/most full-featured to smallest/most minimal:

- **Full OS images** (e.g., ubuntu, debian) — include a full package manager, shell, and many common utilities. Easiest to debug, but largest and with the most packages that could contain vulnerabilities.
- **-slim variants** (e.g., node:20-slim, python:3.12-slim) — a trimmed-down Debian base that still has a shell and basic tools, but strips out documentation, extra packages, and build tools not needed at runtime. A good middle ground for many applications.
- **Alpine-based images** (e.g., node:20-alpine, python:3.12-alpine) — built on Alpine Linux, using the lightweight musl C library instead of glibc, often just 5-10 MB for the base itself. Very small, but the musl/glibc difference can occasionally cause subtle compatibility issues with certain compiled dependencies.
- **Distroless images** (e.g., gcr.io/distroless/static-debian12) — contain only an application and its runtime dependencies, with no shell, no package manager, and no OS utilities at all.

The trade-off is straightforward: the smaller and more minimal the image, the smaller its attack surface and the faster it is to pull and start, but the harder it is to debug interactively, since there is often no shell to `docker exec` into. A distroless image cannot run `docker exec -it mycontainer /bin/sh` at all, because there is no shell binary present.

WARNING

Do not choose distroless images for a service your team is still actively debugging in a live environment unless you also have solid centralized logging and metrics in place — without a shell, your only visibility into a running container is what it logs and exposes, not what you can inspect interactively.

.dockerignore

Just as `.gitignore` tells Git which files to leave out of version control, a `.dockerignore` file tells Docker which files to exclude from the build context sent to the Docker daemon when you run `docker build`. This matters for two distinct reasons:

```
.git
node_modules
*.log
.env
Dockerfile
.dockerignore
dist/
coverage/
```

- **Build speed and size.** Docker sends the entire build context to the daemon before building anything. Excluding large, irrelevant directories like `node_modules` or `.git` keeps that transfer fast and keeps unrelated files from ever being considered for a `COPY . .` step.
- **Security.** Without a `.dockerignore`, a broad `COPY . .` instruction can accidentally copy local secrets — a `.env` file with API keys, cloud credentials, or a `.git` folder with history — directly into an image layer. Once a secret is baked into an image layer, it typically remains recoverable

from that layer even if a later instruction deletes the file, and it travels with the image to every registry and machine it is pushed to.

WARNING

Never rely on deleting a secret file in a later RUN instruction to keep it out of the image — earlier layers are still part of the image history and can be extracted. Keep secrets out of the build context entirely using `.dockerignore`, and inject them at runtime instead (for example, via Kubernetes Secrets, covered in a later chapter).

Running as a Non-Root User

By default, unless a Dockerfile says otherwise, processes inside a container run as the root user. This is convenient during development but risky in production: if an attacker manages to exploit a vulnerability in your application and break out of the container’s isolation (a container escape), running as root inside the container significantly increases what they can do to the underlying host, since container isolation is not as strong a security boundary as full virtualization.

The USER instruction switches the user that subsequent instructions, and the final running container, execute as:

```
FROM node:20-slim

WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install --production
COPY . .

RUN groupadd -r appgroup && useradd -r -g appgroup appuser
RUN chown -R appuser:appgroup /app

USER appuser

EXPOSE 3000
CMD ["node", "server.js"]
```

Here, a dedicated system user and group are created, ownership of the application directory is transferred to that user, and USER appuser ensures the container process itself never runs as root. Many official images (including distroless variants) already ship a built-in unprivileged user such as nonroot, which you can use directly instead of creating your own, as shown in the multi-stage Go example earlier.

BEST PRACTICE

Treat USER root (the default if you omit USER entirely) as something to actively opt out of, not a neutral default. Kubernetes can also enforce this at the cluster level with Pod Security Standards, but building non-root images is the foundation that policy depends on.

The HEALTHCHECK Instruction

A container can be “running” from Docker’s perspective while the application inside it is completely unresponsive — stuck in a deadlock, unable to reach its database, or still starting up. The HEALTHCHECK instruction lets an image define its own notion of health, which Docker (and later, Kubernetes-style probes) can use to detect this:

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1
```

This tells Docker to run the given command every 30 seconds, allow up to 3 seconds for it to respond, wait 5 seconds after container start before the first check counts, and mark the container unhealthy after 3 consecutive failures. `docker ps` will then show a healthy or unhealthy status alongside the container, which orchestration tooling can act on automatically.

NOTE

In Kubernetes and GKE, you will typically define liveness and readiness probes directly on the pod spec rather than relying solely on Dockerfile HEALTHCHECK, since Kubernetes has richer, cluster-aware handling of unhealthy pods (such as restarting or removing them from load balancing). Still, including a `/health` endpoint and a HEALTHCHECK in the image is good practice, since it makes the image self-documenting and testable with plain Docker too.

Image Tagging Strategy

How you tag an image determines whether “deploy version 1.4.2” is a precise, reproducible statement or a guess. A few points matter:

- **Semantic version tags** (e.g., `app:1.4.2`) communicate meaning to humans and are useful for release notes and rollback discussions.
- **Immutable, content-addressed tags**, most commonly the image’s SHA-256 digest (e.g., `app@sha256:3f29a...`), guarantee you are referencing the exact bytes of a specific build. Unlike a human-readable tag, a digest cannot be silently repointed to a different image later.
- **:latest is mutable and risky**. Anyone who pushes an image without specifying a tag overwrites what `:latest` points to, so two people (or two deployments) referencing `myapp:latest` at different times can silently end up running different code. It also makes rollback ambiguous, since there is no record of what `:latest` used to mean.

BEST PRACTICE

In any production deployment manifest, reference images by an immutable version tag or, ideally, by digest (for example, `us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1` paired with its digest), never by `:latest`. This makes deployments reproducible and rollbacks trivial — you simply redeploy the previous known-good tag or digest.

Key Takeaways

- Multi-stage builds keep heavy build toolchains out of the final image, sharply reducing size and attack surface — copy only the final artifact into the runtime stage.
- Minimal base images (slim, Alpine, distroless) shrink images further but trade away debuggability, since many have no shell at all.
- A `.dockerignore` file speeds up builds and, critically, prevents secrets and unnecessary files from being baked permanently into image layers.
- Running as a non-root user via `USER` limits the damage possible if an attacker escapes a compromised container.
- HEALTHCHECK lets Docker detect an unresponsive application inside an otherwise-running container, and tagging images with semantic versions or immutable digests instead of `:latest` keeps deployments reproducible.

Chapter 4: Docker Networking & Volumes

Every container starts life isolated — its own filesystem, its own process list, and its own network stack. That isolation is exactly what makes Docker useful, but real applications need to talk to the outside world, talk to each other, and keep data around after a container stops. This chapter covers the two features that make that possible: networking and storage.

The Default Bridge Network

When Docker is installed, it creates a default network called bridge. Every container you run without specifying a network joins this bridge, gets a private IP address, and can reach the internet through NAT — but it cannot be reached from outside the host unless you explicitly publish a port.

Publishing a port maps a port on the host machine to a port inside the container:

```
docker run -d --name web -p 8080:80 nginx:latest
```

- `-p 8080:80` means “host port 8080 forwards to container port 80.”
- The format is always `-p host_port:container_port`.
- You can publish multiple ports by repeating `-p`, and you can bind to a specific host interface with `-p 127.0.0.1:8080:80`.
- Omitting the host port (`-p 80`) lets Docker assign a random free host port, useful for automated test environments.

TIP

Run `docker port web` at any time to see the actual mappings Docker assigned to a running container.

Once published, `http://localhost:8080` on the host reaches port 80 inside the container. Without `-p`, the container is still running and reachable from other containers on the same Docker network, but it is invisible to the outside world.

Container-to-Container Communication

The default bridge network is deliberately minimal: containers on it can reach each other only by IP address, and it does not provide automatic DNS resolution between containers. For any multi-container setup — an app talking to a database, for example — you should create a **user-defined bridge network** instead.

```
docker network create app-network
```

```
docker run -d --name db --network app-network postgres:16
```

```
docker run -d --name api --network app-network -p 8080:8080 my-api:v1
```

- `docker network create app-network` creates an isolated virtual network.
- Both containers join it via `--network app-network`.
- Inside the `api` container, the database is reachable simply as `db` — Docker’s embedded DNS server resolves container names to their current IP addresses automatically.

This name-based resolution is the key benefit over the default bridge. Your application code never needs to know a container’s IP address; it just connects to a hostname like `db` or `redis`, and Docker keeps that name pointing at the right container even if it restarts and gets a new IP. This is also conceptually the same pattern Kubernetes uses at a larger scale with Services and cluster DNS, which you will meet in later chapters.

BEST PRACTICE

Avoid the default bridge network for anything beyond quick single-container tests. Always create a named, user-defined network for multi-container applications so you get DNS-based service discovery.

Volumes, Bind Mounts, and tmpfs

Containers are meant to be disposable — you should be able to delete one and recreate it without a second thought. But the data an application produces (database files, uploaded files, logs you want to keep) usually should *not* disappear with the container. Docker gives you three mechanisms for handling storage, and picking the right one for the job matters.

Volumes — persistent application data

A **volume** is storage managed entirely by Docker, stored under Docker's own directory on the host (typically `/var/lib/docker/volumes/`) and referenced by name rather than by host path.

```
docker volume create db-data
```

```
docker run -d --name db \
-v db-data:/var/lib/postgresql/data \
postgres:16
```

- `db-data` is created once and can be reused across container restarts, upgrades, or replacements.
- Docker manages the volume's lifecycle independently of any single container, so `docker rm db` does not delete the data.
- Volumes are the right choice for anything that needs to survive container recreation: database files, message queue state, persistent caches.

Bind mounts — local development source code

A **bind mount** maps a specific path on the host filesystem directly into the container. Unlike a volume, you control exactly where the data lives on the host.

```
docker run -d --name dev-app \
-v /home/user/my-app/src:/app/src \
node:20 npm run dev
```

- Any change you make to files in `/home/user/my-app/src` on your laptop is immediately visible inside the container.
- This is the standard pattern for local development: edit code in your normal editor, and a file-watcher inside the container (e.g., nodemon, Flask's reloader) picks up the change instantly, without rebuilding the image.
- Bind mounts are generally discouraged in production because they tie a container to a specific host's filesystem layout, which breaks portability across a cluster of machines.

tmpfs — ephemeral in-memory scratch space

A **tmpfs mount** stores data only in the host's memory. It never touches disk, and it disappears the moment the container stops.

```
docker run -d --name cache-app \
--tmpfs /app/cache:size=100m \
my-api:v1
```

- Useful for temporary scratch space, sensitive data you don't want persisted to disk (session tokens, temporary decrypted files), or performance-sensitive short-lived caches.
- Not available on Windows containers, and by definition not durable across a container restart.

NOTE

Choosing the wrong mechanism is a common beginner mistake — using a bind mount for production database storage ties your data to one specific machine, while using a volume for local source code means you lose live-reload during development.

Mechanism	Managed by	Survives container removal	Typical use
Volume	Docker	Yes	Database files, persistent app state
Bind mount	You (host path)	Yes (it's just a host folder)	Local dev source code, config injection
tmpfs	Kernel (RAM)	No	Secrets, scratch space, ephemeral cache

A Brief Look at Docker Compose

For local multi-container setups, writing out several `docker run` commands by hand gets tedious. Docker Compose lets you describe a small stack in one YAML file and bring it up with a single command. This book does not focus on Compose — in production, Kubernetes takes over this role — but a minimal example is worth seeing once:

```
services:
  web:
    image: my-api:v1
    ports:
      - "8080:8080"
    volumes:
      - app-data:/app/data

volumes:
  app-data:
```

Running `docker compose up -d` starts the web service, publishes port 8080, and attaches the named volume `app-data` for anything the app needs to persist. Compose automatically creates a user-defined network for all services in the file, so if you added a `db` service alongside `web`, they would resolve each other by name exactly as described above — no manual `docker network create` required.

Key Takeaways

- The default bridge network isolates containers by default; use `-p host:container` to publish a port and expose a service to the outside world.
- User-defined bridge networks give you automatic DNS resolution by container name, which is essential for reliable multi-container communication.
- Use volumes for persistent application data, bind mounts for local development source code, and tmpfs for ephemeral in-memory scratch space — choosing correctly avoids both data loss and lost productivity.
- Docker Compose can describe a small multi-container stack in one YAML file, but Kubernetes is this book's focus for anything beyond local development.

Chapter 5: Docker Registries & Image Distribution

Building an image on your laptop is only half the story. To actually run that image somewhere else — a colleague’s machine, a CI pipeline, a production server, or a Kubernetes cluster — it needs to travel from where it was built to where it will run. That’s the job of an **image registry**: a storage and distribution system for container images, conceptually similar to how a package manager repository distributes libraries, but for entire container images instead of source files.

What Is a Registry, and Why Docker Hub?

A registry stores images under repositories, tracks every tag pushed to each repository, and serves image layers over HTTP(S) to anyone with permission to pull them. When you run `docker pull nginx` or `docker run redis`, and you haven’t specified anything else, Docker reaches out to **Docker Hub** — `docker.io` — because it is configured as the default registry in every standard Docker installation.

Docker Hub is enormously useful as the de facto public commons for container images: official images for `nginx`, `postgres`, `redis`, `node`, and thousands of other open-source projects live there, along with millions of publicly published community images. For learning, prototyping, and pulling well-known base images, it’s the natural starting point.

NOTE

“Registry” and “repository” are often used loosely, but they mean different things. A registry (like Docker Hub or Artifact Registry) is the service. A repository (like `library/nginx` or `my-repo/app`) is a named collection of related images inside that registry, distinguished by tags.

Image Naming Anatomy

A fully qualified image reference has three parts:

```
registry/repository:tag
```

For example:

```
us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
```

- `us-central1-docker.pkg.dev` — the registry hostname (here, GCP Artifact Registry in `us-central1`, covered fully in Chapter 6).
- `my-gcp-project-123456/my-repo/app` — the repository path, which for Artifact Registry includes the project ID and repository name.
- `v1` — the tag, a human-readable label for a specific version of the image.

When you omit the registry hostname entirely, as in `nginx` or `postgres:16`, Docker assumes Docker Hub and expands it internally to `docker.io/library/nginx` or `docker.io/library/postgres:16`. When you omit the tag, Docker assumes `:latest`.

WARNING

`:latest` is not a special, always-up-to-date tag — it is just the default tag name applied when you don’t specify one. Relying on it in production is risky because it gives you no guarantee about which actual build you’re running; always pin explicit version tags in production deployments.

Tags vs. Digests

A **tag** (like `v1`, `2.3.1`, or `latest`) is a mutable pointer. Someone can rebuild the image, push it again under the same tag, and the tag now points at completely different content. This is convenient for iterative development but dangerous for reproducibility — a deployment that says “run `app:v1`” might pull different bytes today than it did last month.

A **digest** is an immutable, content-addressed identifier — a SHA-256 hash of the image manifest, written as `sha256:<64-character-hex-string>`. Because the digest is derived from the image's content, it is mathematically guaranteed to always point at the exact same bytes.

```
docker pull us-central1-docker.pkg.dev/my-gcp-project-123456/my-  
repo/app@sha256:1a2b3c4d...
```

- Pulling by digest (using `@sha256:...` instead of `:tag`) guarantees you get exactly the image that was scanned, tested, and approved — nothing can silently change underneath it.
- Production and compliance-sensitive deployments often pin digests rather than tags for this reason, especially in regulated environments where reproducibility must be provable.
- You can find an image's digest with `docker inspect --format='{{index .RepoDigests 0}}' app:v1` or by looking at the registry's web UI.

Logging In, Pushing, and Pulling

To push an image, you first authenticate to the target registry, then tag your local image with the full registry path, then push it.

```
docker login docker.io
```

```
docker build -t app:v1 .
```

```
docker tag app:v1 mydockerhubuser/app:v1
```

```
docker push mydockerhubuser/app:v1
```

```
docker pull mydockerhubuser/app:v1
```

```
docker run -d mydockerhubuser/app:v1
```

- `docker login` prompts for credentials (or a personal access token) and stores an auth token locally so subsequent push/pull commands are authenticated automatically.
- `docker tag` does not create a new image — it adds an additional name pointing at the same image ID, which is why the registry hostname and repository must be part of the tag you push.
- `docker push` uploads only the layers the registry doesn't already have, since image layers are content-addressed and deduplicated.
- `docker pull` downloads the image so it can be run locally; `docker run` will implicitly pull first if the image isn't already present.

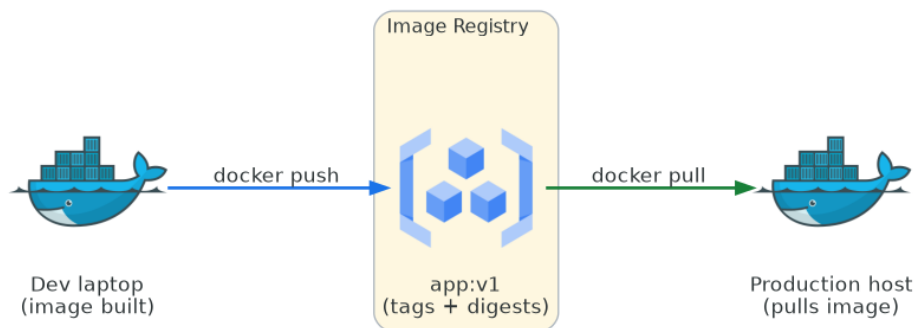
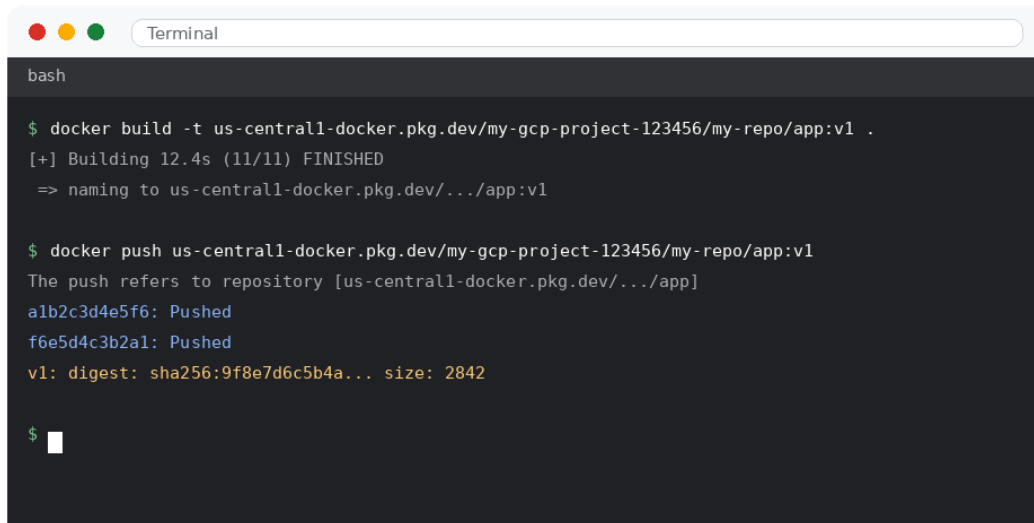


Figure 2: A container registry lets one host push an image and another host pull that exact same image, keeping environments consistent.

The diagram above illustrates the standard distribution flow: a developer builds and pushes an image from a dev machine to a registry, and one or more production hosts (or, later, GKE nodes) independently pull that same image from the registry to run it. The registry is the single source of truth that decouples “where an image is built” from “where it runs” — neither side needs direct access to the other.

A terminal window titled "Terminal" with a dark background and light text. It shows the execution of Docker commands. First, a build command is run, which completes successfully. Then, a push command is run, which uploads two image layers to a registry. The output shows the layer hashes and the final image digest and size.

```
bash

$ docker build -t us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .
[+] Building 12.4s (11/11) FINISHED
=> naming to us-central1-docker.pkg.dev/.../app:v1

$ docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
The push refers to repository [us-central1-docker.pkg.dev/.../app]
alb2c3d4e5f6: Pushed
f6e5d4c3b2a1: Pushed
v1: digest: sha256:9f8e7d6c5b4a... size: 2842

$
```

Figure 3: Illustrative docker push uploading image layers to a registry.

Public vs. Private Registries

Docker Hub’s free tier is a **public** registry by default: anything you push without marking it private is visible to anyone. This is fine for open-source projects but unacceptable for proprietary application code, so Docker Hub also offers private repositories on paid plans, and every major cloud provider offers its own **private registry** service — Amazon ECR, Azure Container Registry, and GCP’s Artifact Registry among them.

There are two separate reasons most companies do not rely on public Docker Hub for production workloads:

- **Rate limits.** Docker Hub enforces pull rate limits for anonymous and free authenticated accounts (historically on the order of 100–200 pulls per 6-hour window per IP or account). A Kubernetes cluster with dozens of nodes, all pulling the same base images repeatedly during deploys and autoscaling events, can exhaust that limit quickly, causing pull failures at exactly the moment you’re trying to scale up.
- **Control, security, and locality.** A private registry keeps proprietary images out of a public namespace, integrates with your cloud IAM system for fine-grained access control, sits inside the same cloud region as your workloads (dramatically reducing pull latency and egress cost), and can be configured to automatically scan every pushed image for known vulnerabilities.

BEST PRACTICE

Use Docker Hub freely for pulling well-known public base images (nginx, postgres, alpine, and similar), but push your own application images to a private registry close to where they’ll run. In this book, that private registry is GCP Artifact Registry, which Chapter 6 covers in full.

Key Takeaways

- A registry stores and distributes images; Docker Hub is the default public registry that Docker uses when no other registry is specified.
- Image references follow `registry/repository:tag`; omitting the registry assumes Docker Hub,

- and omitting the tag assumes `:latest`.
- Tags are mutable pointers, while digests (`sha256:...`) are immutable and content-addressed — pin digests when reproducibility matters.
 - `docker login`, `docker tag`, `docker push`, and `docker pull` form the standard workflow for moving images between a dev machine and a registry.
 - Docker Hub's pull rate limits and lack of cloud-native access control are the main reasons production workloads run against a private registry, such as GCP Artifact Registry, instead.

Chapter 6: GCP Artifact Registry Fundamentals

Chapter 5 explained why production workloads need a private registry rather than relying on public Docker Hub. On Google Cloud, that private registry is **Artifact Registry** — Google’s unified service for storing, managing, and securing build artifacts and container images. This chapter covers everything you need to create a repository, understand its structure, push and pull images against it, and control who can do what through IAM.

What Is Artifact Registry?

Artifact Registry is Google Cloud’s fully managed registry for software packages. “Unified” is the key word: rather than having one product for container images and a separate one for language packages, Artifact Registry hosts multiple package formats side by side within the same service, each in its own repository.

NOTE

Artifact Registry is the current recommended service and the direction Google Cloud is actively investing in. It replaces the older **Container Registry** (`gcr.io`), which is being phased out — Google has been progressively deprecating Container Registry and encouraging all users, new and existing, to migrate to Artifact Registry. Any new project should start directly with Artifact Registry rather than `gcr.io`.

Artifact Registry supports several repository formats, each suited to a different ecosystem:

- **Docker** — container images (the focus of this book).
- **Maven** — Java packages (`.jar` artifacts and POM files).
- **npm** — Node.js/JavaScript packages.
- **Python** — Python packages, servable as a pip-compatible index.
- **Apt** — Debian/Ubuntu-style `.deb` system packages.
- **Yum** — RPM-based Linux system packages.

A single GCP project can hold repositories of several different formats simultaneously, but each individual repository is locked to one format when it’s created. This book uses the Docker format exclusively, since our focus is containers destined for GKE, but it’s worth knowing the other formats exist in case your organization also wants to consolidate its Maven or npm artifacts in the same place.

Creating a Repository

Before you can push anything, you create a repository inside a GCP project using the `gcloud artifacts repositories create` command:

```
gcloud artifacts repositories create my-repo \
  --repository-format=docker \
  --location=us-central1 \
  --description="Docker images for my application"
```

- `my-repo` is the repository name — it becomes part of every image path pushed to it.
- `--repository-format=docker` tells Artifact Registry to treat this repository as a container image store.
- `--location=us-central1` pins the repository to a single region.
- `--description` is optional metadata, useful for humans browsing the console later.

Region vs. Multi-Region

The `--location` flag accepts either a single region (`us-central1`, `eu-west1`, `asia-southeast1`) or a multi-region (`us`, `eu`, `asia`):

- A **regional** repository stores data in one specific region, giving the lowest possible pull latency for workloads (like GKE nodes) running in that same region, and the lowest cost.
- A **multi-region** repository replicates data across multiple regions within a broader geography, trading some cost and write latency for higher availability and lower latency when pulling from workloads spread across several regions.

BEST PRACTICE

Choose a regional repository in the same region as your GKE cluster whenever your workloads live in a single region — this minimizes pull latency and avoids unnecessary cross-region network egress charges. Reach for multi-region only when you genuinely run workloads across multiple regions or continents.

Verify the repository exists and inspect its settings at any time:

```
gcloud artifacts repositories describe my-repo --location=us-central1
```

Image Path Structure

Every image in Artifact Registry is addressed by a single, fully qualified path with a fixed structure:

LOCATION-docker.pkg.dev/PROJECT_ID/REPOSITORY/IMAGE:TAG

Applied to our placeholders throughout this book, a real path looks like:

us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1

- us-central1-docker.pkg.dev — the registry hostname, which encodes both the location and the fact that this is the Docker-format endpoint (pkg.dev is shared across all Artifact Registry formats; -docker. is the Docker-specific subdomain within it).
- my-gcp-project-123456 — the GCP project that owns the repository.
- my-repo — the repository created above.
- app — the image name within that repository (a repository can hold many differently named images).
- v1 — the tag.

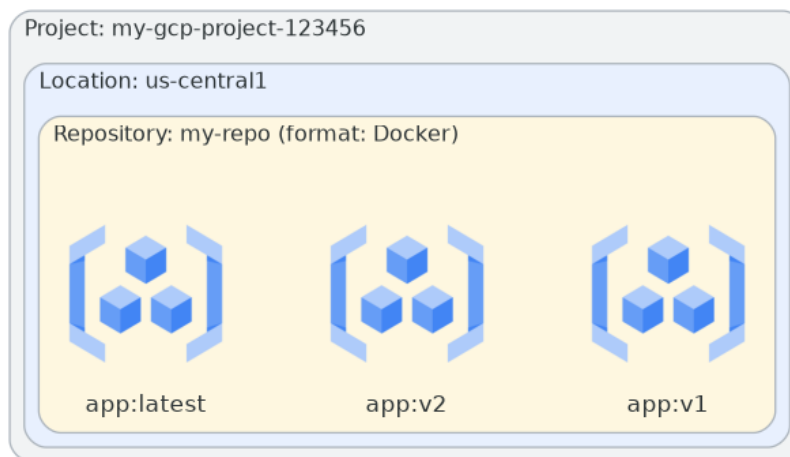


Figure 4: Artifact Registry's resource hierarchy: a project contains locations, which contain repositories, which contain versioned image tags.

The hierarchy is strictly nested: a **project** contains one or more **locations** (regions or multi-regions) at which repositories live, each **location** contains one or more **repositories**, each **repository** contains one or more **images** (distinguished by name), and each **image** carries one or more **tags** pointing at specific pushed versions. Understanding this nesting makes the long path string easy to read at a glance — read it left to right as location, then project, then repository, then image, then tag.

Tagging and pushing an image follows the same pattern introduced in Chapter 5, just with this longer registry hostname:

```
docker tag app:v1 us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
```

Before the push succeeds, Docker needs credentials for the `us-central1-docker.pkg.dev` hostname. The standard approach is the `gcloud` credential helper, which routes Docker's authentication through your existing `gcloud` login rather than a separate username and password:

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

This command adds an entry to Docker's config file so that every `docker push` or `docker pull` against that hostname is transparently authenticated using your active `gcloud` credentials.

Core IAM Roles

Access to Artifact Registry repositories is controlled through standard GCP IAM, and three predefined roles cover the vast majority of real-world needs:

Role	Grants
<code>roles/artifactregistry.reader</code>	Pull images and view repository metadata; no push or delete permission
<code>roles/artifactregistry.writer</code>	Everything reader grants, plus push new images and tags
<code>roles/artifactregistry.admin</code>	Everything writer grants, plus create, delete, and modify repositories and their IAM policies

- Grant **reader** to service accounts and users who only need to consume images — this includes the GKE node service account, which only ever pulls images to run them.
- Grant **writer** to CI/CD pipeline service accounts (Cloud Build, GitHub Actions, GitLab CI runners) that build and push new image versions.
- Reserve **admin** for a small number of platform-team humans or automation responsible for provisioning and managing repositories themselves, not for day-to-day build or deploy jobs.

Assigning a role to a service account looks like this:

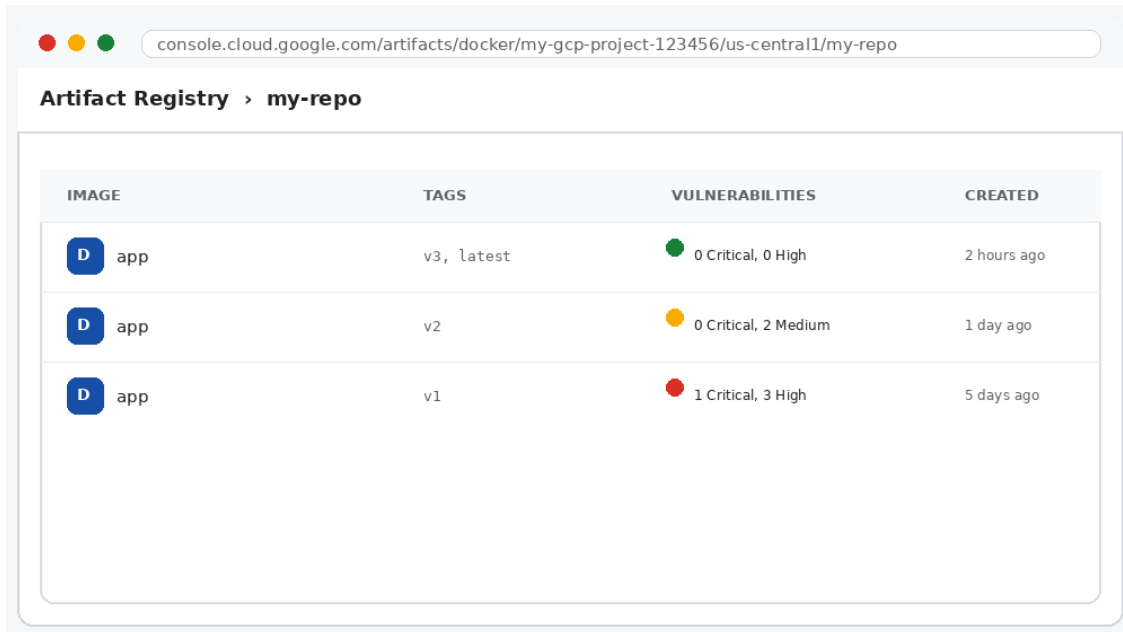
```
gcloud artifacts repositories add-iam-policy-binding my-repo \
  --location=us-central1 \
  --member="serviceAccount:ci-pipeline@my-gcp-project-123456.iam.gserviceaccount.com" \
  --role="roles/artifactregistry.writer"
```

TIP

Following least-privilege, grant IAM roles at the individual repository level (as shown above) rather than at the whole project level whenever you have more than one repository with different audiences — this ensures a CI pipeline that only needs to push to `my-repo` cannot accidentally

also push to an unrelated repository in the same project.

Once images are flowing into a repository, the Cloud Console gives you a visual view of every image, its tags, push timestamps, and — if vulnerability scanning is enabled — a security posture summary for each pushed digest.



The screenshot shows the Google Cloud Console interface for an Artifact Registry repository. The browser address bar displays the URL: `console.cloud.google.com/artifacts/docker/my-gcp-project-123456/us-central1/my-repo`. The page title is "Artifact Registry > my-repo". Below the title is a table with four columns: "IMAGE", "TAGS", "VULNERABILITIES", and "CREATED". The table contains three rows of data, each representing a Docker image named "app".

IMAGE	TAGS	VULNERABILITIES	CREATED
 app	v3, latest	 0 Critical, 0 High	2 hours ago
 app	v2	 0 Critical, 2 Medium	1 day ago
 app	v1	 1 Critical, 3 High	5 days ago

Figure 5: Illustrative an Artifact Registry repository view, showing image tags alongside vulnerability severity.

Key Takeaways

- Artifact Registry is Google Cloud's unified, multi-format registry and the recommended successor to the older Container Registry (`gcr.io`), which is being deprecated.
- It supports Docker, Maven, npm, Python, Apt, and Yum repository formats, each isolated within its own repository, but this book focuses on Docker images.
- Repositories are created with `gcloud artifacts repositories create`, choosing between a single region (lower latency and cost) and a multi-region (higher availability across a broader geography).
- Every image is addressed by the fixed path `LOCATION-docker.pkg.dev/PROJECT_ID/REPOSITORY/IMAGE:TAG` reflecting a strict project-location-repository-image-tag hierarchy.
- `roles/artifactregistry.reader`, `.writer`, and `.admin` provide a clean least-privilege model: pull-only for consumers like GKE nodes, push access for CI/CD pipelines, and full management reserved for platform administrators.

Chapter 7: Authenticating Docker to Artifact Registry

Before Docker can push or pull images to and from GCP Artifact Registry, it needs a way to prove who it is. Artifact Registry is a private, IAM-protected service — unlike Docker Hub’s public registry, nothing is anonymous. This chapter explains how to configure Docker to authenticate correctly, and — more importantly — which authentication method to use depending on whether you are working on your laptop or running inside a CI/CD pipeline.

Getting this right matters for security. The wrong choice (a long-lived downloaded key sitting on disk, for example) is one of the most common causes of leaked credentials in real incidents. The right choice depends on where the `docker push` or `docker pull` command is actually running.

Configuring the Docker CLI

The simplest and most common way to authenticate Docker to Artifact Registry is the `gcloud` credential helper. It configures Docker’s `config.json` to call `gcloud` automatically whenever it talks to a matching registry host, so you never type a password.

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

This command edits `~/.docker/config.json` and registers `us-central1-docker.pkg.dev` as a host that should use `gcloud` as its credential helper. From that point on, any `docker push` or `docker pull` targeting an image path under that host — for example `us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1` — is transparently authenticated using whatever identity `gcloud` is currently using.

- Run this once per machine (or CI runner image) before the first push.
- The hostname must match your Artifact Registry region exactly (`us-central1-docker.pkg.dev`, `europa-west1-docker.pkg.dev`, and so on).
- You can register multiple regional hosts in one call by separating them with commas.

NOTE

`gcloud auth configure-docker` does not create credentials itself — it only tells Docker *how* to fetch them. The actual identity comes from one of the three authentication paths described next.

The Three Authentication Paths

There are three distinct ways `gcloud` can supply an identity to Docker. Choosing the right one for the right context is the core skill of this chapter.

1. Application Default Credentials (local development)

For day-to-day work on a laptop or developer workstation, the recommended approach is Application Default Credentials (ADC), set up with:

```
gcloud auth login
gcloud auth application-default login
```

This opens a browser-based OAuth flow, and the resulting user credentials are cached locally (typically under `~/.config/gcloud/`). The Docker credential helper picks these up automatically. This is convenient, tied to your own human identity, and easy to revoke — but it is meant for individual developers, not for automated systems or production pipelines.

2. Downloaded Service Account JSON Keys (avoid where possible)

An older pattern is to create a service account, download a long-lived JSON key file, and point `gcloud` or Docker at it:

```
gcloud iam service-accounts keys create key.json \
  --iam-account=ar-writer@my-gcp-project-123456.iam.gserviceaccount.com

gcloud auth activate-service-account --key-file=key.json
```

This works, but it creates a static, exportable secret that never expires unless someone manually rotates or deletes it.

WARNING

Downloaded JSON key files are a major source of credential leaks — they get committed to git repositories, baked into container images, or copied into CI logs. Avoid them unless there is genuinely no alternative, and if you must use one, store it in a secret manager, rotate it regularly, and never bake it into an image layer.

3. Workload Identity Federation (CI/CD pipelines)

For automated pipelines — GitHub Actions, GitLab CI, Jenkins, or Cloud Build running outside GCP — the modern, recommended approach is Workload Identity Federation (WIF). Instead of a static key, WIF lets an external identity provider (such as GitHub's OIDC token issuer) exchange a short-lived token for temporary GCP credentials, scoped to a specific service account, with no downloadable secret ever existing.



Figure 6: Workload Identity Federation lets a CI/CD runner authenticate to Artifact Registry without a long-lived service account key.

The flow works like this: the CI job presents its platform-issued OIDC token to GCP's Security Token Service; GCP validates the token against a configured Workload Identity Pool and Provider; if the token matches the trust policy (for example, "only tokens from this exact GitHub repository and branch"), GCP issues a short-lived access token impersonating the mapped service account; that token is then used by `gcloud auth configure-docker` to authenticate the docker push.

- No static key ever leaves GCP — tokens are minted on demand and expire quickly.
- Access can be scoped tightly (a specific repo, branch, or environment).
- Revoking access means deleting or updating the pool/provider mapping — no key rotation required.

```
gcloud iam workload-identity-pools create "github-pool" \
  --project="my-gcp-project-123456" \
  --location="global" \
  --display-name="GitHub Actions Pool"

gcloud iam workload-identity-pools providers create-oidc "github-provider" \
  --project="my-gcp-project-123456" \
  --location="global" \
  --workload-identity-pool="github-pool" \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-mapping="google.subject=assertion.sub"
```

BEST PRACTICE

Use ADC for local development, Workload Identity Federation for every CI/CD pipeline, and treat downloaded JSON keys as a last resort reserved for legacy systems that cannot support federated

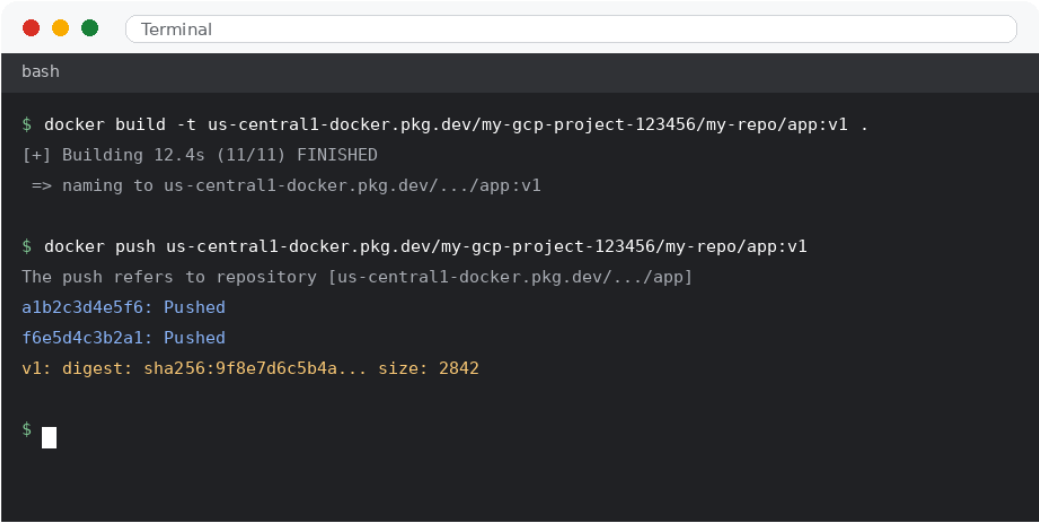
identity.

Granting Push Access with IAM

Regardless of which authentication path supplies the identity, that identity still needs an IAM role granting it permission to push images. The minimal role for pushing (and pulling) is `roles/artifactregistry.writer`, granted at the repository level:

```
gcloud artifacts repositories add-iam-policy-binding my-repo \  
  --project=my-gcp-project-123456 \  
  --location=us-central1 \  
  --member="serviceAccount:ar-writer@my-gcp-project-123456.iam.gserviceaccount.com" \  
  --role="roles/artifactregistry.writer"
```

- `roles/artifactregistry.writer` includes both push and pull permissions.
- `roles/artifactregistry.reader` is sufficient for pipelines that only need to pull (for example, a GKE node pulling images at deploy time).
- Prefer granting roles at the repository level rather than the project level, so a compromised credential only exposes one repository.

A terminal window titled "Terminal" showing a bash shell. The user runs `docker build -t us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .`, which outputs `[+] Building 12.4s (11/11) FINISHED` and `=> naming to us-central1-docker.pkg.dev/.../app:v1`. Then the user runs `docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1`, which outputs `The push refers to repository [us-central1-docker.pkg.dev/.../app]`, `alib2c3d4e5f6: Pushed`, `f6e5d4c3b2a1: Pushed`, and `v1: digest: sha256:9f8e7d6c5b4a... size: 2842`. The prompt `$` is visible at the bottom.

```
bash  
  
$ docker build -t us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .  
[+] Building 12.4s (11/11) FINISHED  
=> naming to us-central1-docker.pkg.dev/.../app:v1  
  
$ docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1  
The push refers to repository [us-central1-docker.pkg.dev/.../app]  
alib2c3d4e5f6: Pushed  
f6e5d4c3b2a1: Pushed  
v1: digest: sha256:9f8e7d6c5b4a... size: 2842  
  
$
```

Figure 7: Illustrative confirmation that Docker is configured to authenticate to Artifact Registry.

Key Takeaways

- `gcloud auth configure-docker LOCATION-docker.pkg.dev` wires Docker's credential helper to Artifact Registry but does not itself create credentials.
- Use Application Default Credentials for local developer machines — simple and tied to a human identity.
- Avoid downloaded service account JSON keys where possible; they are static, long-lived secrets that are easy to leak.
- Use Workload Identity Federation for CI/CD pipelines so tokens are short-lived and no static key ever exists.
- Grant the narrowest IAM role needed — typically `roles/artifactregistry.writer` for push, `roles/artifactregistry.reader` for pull-only — at the repository level.

Chapter 8: Building & Pushing Images to Artifact Registry

With Docker authenticated (Chapter 7), you are ready to actually get images into Artifact Registry. This chapter walks through the manual process — building locally, tagging, and pushing — and then the automated alternative most teams graduate to: Cloud Build triggered directly from a source repository. It closes with a tagging strategy that keeps your registry organized and your deployments reproducible.

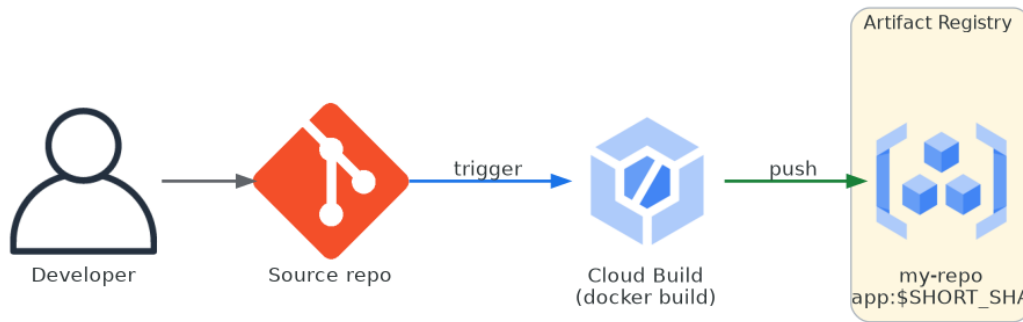


Figure 8: Cloud Build pulls source from a Git repository, builds the image, and pushes the result to Artifact Registry.

The diagram above shows the two paths this chapter covers: a developer building and pushing directly from their workstation, and the automated path where a commit to a source repository triggers Cloud Build, which builds the image and pushes it to Artifact Registry without any human running `docker push` by hand.

Manual Build, Tag, and Push

Assume you have a working Dockerfile in your project directory. The manual workflow has three steps: build, tag, and push.

1. Build the image locally with a local tag

```
docker build -t app:v1 .
```

2. Tag it for Artifact Registry using the full registry path

```
docker tag app:v1 us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
```

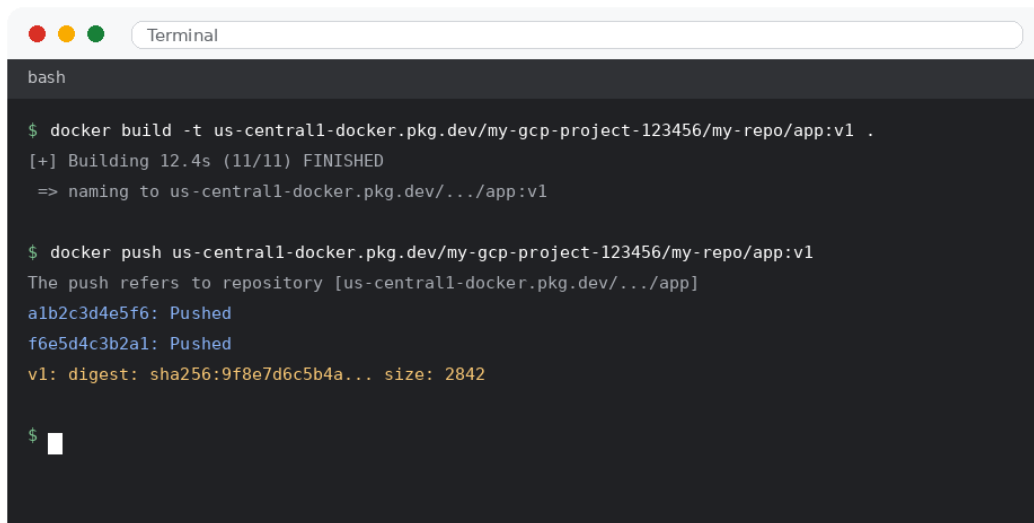
3. Push the tagged image to Artifact Registry

```
docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
```

- `docker build` creates the image and stores it in your local Docker daemon under the tag `app:v1`.
- `docker tag` does not copy any data — it just creates a second name (a pointer to the same image ID) that matches Artifact Registry’s required format: `LOCATION-docker.pkg.dev/PROJECT_ID/REPOSITORY`.
- `docker push` uploads the image layers to Artifact Registry, reusing whatever layers already exist remotely and only transferring the ones that changed.

You can combine tagging into the build step directly, which many teams prefer since it avoids a separate `docker tag` command:

```
docker build -t us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .
docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
```

A terminal window titled "Terminal" with a dark background. It shows the execution of two Docker commands. The first command is `docker build -t us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .`, which outputs `[+] Building 12.4s (11/11) FINISHED` and `=> naming to us-central1-docker.pkg.dev/.../app:v1`. The second command is `docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1`, which outputs `The push refers to repository [us-central1-docker.pkg.dev/.../app]`, `alb2c3d4e5f6: Pushed`, `f6e5d4c3b2a1: Pushed`, and `v1: digest: sha256:9f8e7d6c5b4a... size: 2842`. The prompt `$` is visible at the bottom.

```
bash

$ docker build -t us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .
[+] Building 12.4s (11/11) FINISHED
=> naming to us-central1-docker.pkg.dev/.../app:v1

$ docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
The push refers to repository [us-central1-docker.pkg.dev/.../app]
alb2c3d4e5f6: Pushed
f6e5d4c3b2a1: Pushed
v1: digest: sha256:9f8e7d6c5b4a... size: 2842

$
```

Figure 9: Illustrative docker build followed by docker push to Artifact Registry.

TIP

Run `docker images` after the build to confirm the tag exists locally, and `gcloud artifacts docker images list us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo` after the push to confirm it landed remotely.

Automating with Cloud Build

Manually pushing from a laptop does not scale, and it means the only proof an image matches source code is trust. Cloud Build removes both problems: it builds inside GCP from a defined source (a Git repository), runs the exact same steps every time, and pushes the result automatically, typically triggered by a git push or merge.

A build is defined declaratively in a `cloudbuild.yaml` file at the root of your repository:

```
steps:
  # Step 1: Build the image, tagging with the short commit SHA
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'build'
      - '-t'
      - 'us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:$SHORT_SHA'
      - '.'

  # Step 2: Push the image to Artifact Registry
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'push'
      - 'us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:$SHORT_SHA'

images:
  - 'us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:$SHORT_SHA'

options:
  logging: CLOUD_LOGGING_ONLY
```

- Each entry under `steps` runs as its own container; here both steps reuse the official docker

builder image.

- `$SHORT_SHA` is a Cloud Build substitution variable populated automatically from the triggering commit — no manual tagging needed.
- The top-level `images` field tells Cloud Build which images to register with it after a successful build, so they show up in the build's provenance metadata.
- `options.logging` routes build logs to Cloud Logging, which is required when the build's service account does not have the older default logging bucket permissions.

To wire this to a repository so it runs on every push, create a trigger:

```
gcloud builds triggers create github \
  --project=my-gcp-project-123456 \
  --repo-name=my-app-repo \
  --repo-owner=my-github-org \
  --branch-pattern="^main$" \
  --build-config=cloudbuild.yaml
```

From this point on, every merge to main automatically builds a fresh image and pushes it to my-repo, with no developer ever running `docker push` for that branch.

BEST PRACTICE

Keep `cloudbuild.yaml` in the same repository as the application code, versioned alongside it, so the build definition changes in lockstep with the Dockerfile it builds.

Tagging Strategy for CI

How you tag images has a direct effect on whether a production incident can be traced back to an exact commit. A good tagging strategy uses several tags for the same underlying image, each serving a different purpose.

- **Short git commit SHA (primary, immutable tag).** Every image built by CI should be tagged with the short SHA of the commit it was built from (`app:a1b2c3d`). This tag is unique, immutable, and directly traceable to source code — it should be the tag your deployment manifests actually reference.
- **Semantic version tags (for releases).** When a commit is cut as an official release, additionally tag the same image with a semantic version, such as `app:v1.4.0`. This gives humans a readable, meaningful label for release notes and rollback discussions, while still pointing at the exact same immutable image as its SHA tag.
- **latest (convenience only, never for deployments).** The latest tag is useful for local exploration (“give me whatever was built most recently”) but it is mutable — it gets silently re-pointed by the next push. Never reference latest in a Kubernetes manifest or any deployment automation, because a rollback or redeploy could pull a completely different image than the one that was tested.

```
docker tag app:a1b2c3d
  ↪ us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1.4.0
docker tag app:a1b2c3d
  ↪ us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:latest
docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1.4.0
docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:latest
```

WARNING

Deploying against `:latest` is one of the most common causes of “it worked yesterday, it broke today” incidents — the tag moved underneath you. Always deploy against an immutable tag such as the commit SHA.

Key Takeaways

- The manual workflow is build, then `docker tag` to the full Artifact Registry path, then `docker push`.
- Cloud Build automates this entirely from a `cloudbuild.yaml` file and a trigger, removing manual pushes from the release process.
- `$SHORT_SHA` and similar Cloud Build substitutions let every automated build tag itself traceably back to source.
- Tag every CI-built image with the immutable commit SHA; add semantic version tags for releases; treat `latest` as a convenience label only.
- Never deploy against a mutable tag like `latest` — deployments should always reference an immutable, traceable tag.

Chapter 9: Vulnerability Scanning & Artifact Registry Security

Pushing an image to Artifact Registry is not the end of the security story — it is the beginning. Every image you push carries whatever operating system packages and language libraries were baked into it, and any one of those can have a known vulnerability. This chapter covers how Artifact Registry automatically scans images for you, how to read the results, how to stop vulnerable or stale images from piling up, and a brief look at encryption options for regulated workloads.

Automatic Scanning on Push

Artifact Registry integrates with Container Analysis, a GCP service that automatically scans container images for known vulnerabilities the moment they are pushed. You do not need to configure or trigger anything — as soon as `docker push` (or a Cloud Build push step) completes, the On-Demand Scanning and Container Analysis pipeline picks up the new image, inspects its OS packages and, for supported ecosystems, its language-level dependencies, and compares them against public vulnerability databases (including the National Vulnerability Database).

NOTE

The Container Scanning API must be enabled on the project for automatic scanning to run. It is enabled by default in most new projects, but if scan results seem to be missing, check `gcloud services list --enabled-for-containerscanning.googleapis.com`.

Scans are not a one-time event either. Because new vulnerabilities are discovered constantly, Artifact Registry periodically re-scans previously pushed images so that an image considered “clean” last month is flagged automatically if a new CVE affecting one of its packages is published later — without anyone having to push it again.

Viewing Scan Results

The most direct way to see vulnerability findings for a specific image is the `gcloud artifacts docker images describe` command with the `--show-package-vulnerability` flag:

```
gcloud artifacts docker images describe \
  us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 \
  --show-package-vulnerability
```

```
image_summary:
  digest: sha256:8f3e...c21a
  vulnerability_counts:
    CRITICAL: 0
    HIGH: 2
    MEDIUM: 5
    LOW: 11
package_vulnerability_summary:
  vulnerabilities:
    - severity: HIGH
      cve: CVE-2024-XXXXX
      package: openssl
      fixed_version: 3.0.13-r1
    - severity: HIGH
      cve: CVE-2024-YYYYY
      package: libcurl
      fixed_version: 8.5.0-r2
```

- The `vulnerability_counts` block gives you a quick severity breakdown: Critical, High, Medium, and Low.

- Each entry under `package_vulnerability_summary` names the affected package, the CVE identifier, and — critically — the version that fixes it, so remediation is usually as simple as rebuilding on a newer base image.
- Critical and High findings are the ones that typically warrant blocking a deployment; Medium and Low are often accepted with a documented exception, depending on your organization's risk tolerance.

IMAGE	TAGS	VULNERABILITIES	CREATED
app	v3, latest	0 Critical, 0 High	2 hours ago
app	v2	0 Critical, 2 Medium	1 day ago
app	v1	1 Critical, 3 High	5 days ago

Figure 10: Illustrative a vulnerability scan summary, showing severity counts for a pushed image.

You can also browse scan results visually in the Cloud Console under Artifact Registry, where each image's detail page shows the same severity breakdown with clickable links to each CVE's public record.

TIP

Build vulnerability checks into your pipeline rather than checking manually after every push — a Cloud Build step can call this same `gcloud` command and fail the build if Critical vulnerabilities are found.

Gating Deployments on Scan Results

Knowing that an image has Critical vulnerabilities is only useful if something actually stops it from reaching production. Two common patterns handle this:

- **Pipeline-level gating.** A CI/CD step (in Cloud Build, GitHub Actions, or similar) calls `gcloud artifacts docker images describe --show-package-vulnerability`, parses the result, and fails the pipeline if Critical (or High, depending on policy) findings are present. This is simple to set up but relies on every pipeline remembering to include the check.
- **Cluster-level enforcement with Binary Authorization.** A stronger control is to enforce this at the GKE admission layer itself, so that even an image pushed outside a compliant pipeline cannot be deployed. Binary Authorization lets you require that images be signed by a trusted attester — for example, an attester that only signs images with zero Critical or High vulnerabilities — before GKE will admit them to a cluster. This is covered in full in a later chapter dedicated to Binary Authorization; for now, treat automatic scanning as the data source that a Binary Authorization policy consumes.

BEST PRACTICE

Do not rely on scanning alone as a control. Scanning tells you a problem exists; gating (in the pipeline or, more robustly, via Binary Authorization at the cluster) is what actually prevents a vulnerable image from running.

Cleanup Policies

Every image you push — including every intermediate build tagged only with a commit SHA, and every image left “dangling” after a tag is moved — continues to occupy storage and remains a potential target as long as it exists in the registry. Left unmanaged, a busy CI pipeline can accumulate thousands of old images within months.

Artifact Registry supports cleanup policies that automatically delete images matching rules you define, such as “delete untagged images older than 30 days” or “keep only the 20 most recent tagged versions per package.” Policies are defined in a JSON file and applied to a repository:

```
[
  {
    "name": "delete-untagged-older-than-30d",
    "action": {"type": "Delete"},
    "condition": {
      "tagState": "UNTAGGED",
      "olderThan": "2592000s"
    }
  },
  {
    "name": "keep-recent-tagged-versions",
    "action": {"type": "Keep"},
    "mostRecentVersions": {
      "keepCount": 20
    }
  }
]
```

```
gcloud artifacts repositories set-cleanup-policies my-repo \
--project=my-gcp-project-123456 \
--location=us-central1 \
--policy=cleanup-policy.json
```

- The first rule deletes untagged image versions once they are older than 30 days (2,592,000 seconds) — these are typically leftovers from tags being moved or replaced and rarely need to be kept.
- The second rule keeps only the most recent 20 tagged versions, letting older CI builds age out automatically instead of accumulating forever.
- Reducing the number of stored images also reduces the attack surface — an old image nobody remembers exists cannot be accidentally deployed, and it cannot be scanned as a false sense of security for something no longer in use.

WARNING

Test cleanup policies with `--dry-run` (or review conditions carefully) before applying them broadly — an overly aggressive policy can delete a tagged release image that is still referenced by a rollback plan.

Customer-Managed Encryption Keys (CMEK)

By default, Artifact Registry encrypts all stored image data at rest using Google-managed encryption keys, which requires no configuration and is sufficient for most workloads. For regulated industries — finance, healthcare, government — organizations sometimes need direct control over the encryption key's lifecycle: who can use it, when it rotates, and the ability to revoke access by disabling the key itself. Artifact Registry supports this through customer-managed encryption keys (CMEK), backed by Cloud KMS, where you create and control the key and grant Artifact Registry's service agent permission to use it for encryption and decryption.

```
gcloud artifacts repositories create my-repo \
  --project=my-gcp-project-123456 \
  --location=us-central1 \
  --repository-format=docker \
  --kms-key=projects/my-gcp-project-123456/locations/us-central1/keyRings/my-
  ↪ keyring/cryptoKeys/my-key
```

- CMEK must be configured at repository creation time — it cannot be added to an existing repository afterward.
- If your organization does not have a specific compliance requirement mandating customer-controlled keys, Google-managed encryption is simpler to operate and equally secure.

Key Takeaways

- Artifact Registry automatically scans every pushed image via Container Analysis, with no manual trigger required, and periodically re-scans for newly discovered vulnerabilities.
- Use `gcloud artifacts docker images describe --show-package-vulnerability` to see severity counts (Critical, High, Medium, Low) and specific CVEs with fixed versions.
- Scan results are most valuable when they gate deployments — either in the CI/CD pipeline or, more robustly, via Binary Authorization at the GKE cluster level (covered fully later).
- Cleanup policies automatically remove untagged or excess old images, controlling storage cost and shrinking the attack surface.
- CMEK, backed by Cloud KMS, gives regulated workloads direct control over encryption keys, but must be set at repository creation time.

Chapter 10: Kubernetes Fundamentals

In the previous chapters you learned how to package an application into a Docker container and push it to Google Artifact Registry. That solves one problem: getting a consistent, portable unit of software. But it creates a new question. Once you have five containers, or fifty, or five hundred, spread across many machines, who decides which machine runs which container? Who notices when a container crashes and restarts it? Who spreads traffic across multiple copies of your app, and who safely rolls out version 2 without taking down version 1 first?

That is the exact problem Kubernetes exists to solve.

The Problem: More Containers Than One Engineer Can Manage

Imagine you are running your application by hand. You SSH into a virtual machine, run `docker run` for your container, and it works. Now imagine you need three copies of that container for reliability, spread across three different machines so a single hardware failure does not take down your whole app. Now imagine one of those containers crashes at 3 AM. Now imagine you need to deploy a new version without any downtime. Now imagine you need to do all of this for twenty different services, not just one.

Doing this manually does not scale. You would need to constantly watch, restart, reschedule, and reconfigure things faster than any human team can react. This is the exact gap Kubernetes fills. Kubernetes is a container orchestration platform: a system that automates the deployment, scaling, networking, and healing of containerized applications across a cluster of machines.

Specifically, Kubernetes takes over four jobs that used to require constant manual attention:

- **Scheduling** — deciding which machine (called a node) in the cluster should run each container, based on available CPU, memory, and other constraints.
- **Self-healing** — automatically detecting when a container or an entire machine fails, and starting replacement containers elsewhere.
- **Scaling** — increasing or decreasing the number of running copies of your application in response to load, either manually or automatically.
- **Rolling updates** — replacing old versions of your application with new versions gradually, so that a bad rollout can be caught and reversed before it takes down the whole service.

NOTE

Kubernetes is often abbreviated “K8s” — the 8 stands for the eight letters between the “K” and the “s” in “Kubernetes.”

Google Kubernetes Engine (GKE), which you will use starting in the next few chapters, is Google Cloud’s managed Kubernetes service. It runs the Kubernetes control plane for you, so you do not have to install or patch it yourself. But the concepts in this chapter apply to Kubernetes everywhere, not just on GKE.

The Two Halves of a Cluster

A Kubernetes cluster is made up of two kinds of machines working together: the control plane, which makes decisions, and the worker nodes, which actually run your containers.

Control Plane Components

The control plane is the “brain” of the cluster. It does not run your application containers itself — it decides where they should run and continuously checks that reality matches what you asked for. It is made up of four main pieces:

- **API server (kube-apiserver)** — the single front door into the cluster. Every request, whether from `kubectl`, a dashboard, or an internal component, goes through the API server. It validates requests and is the only component that talks directly to the cluster’s data store.

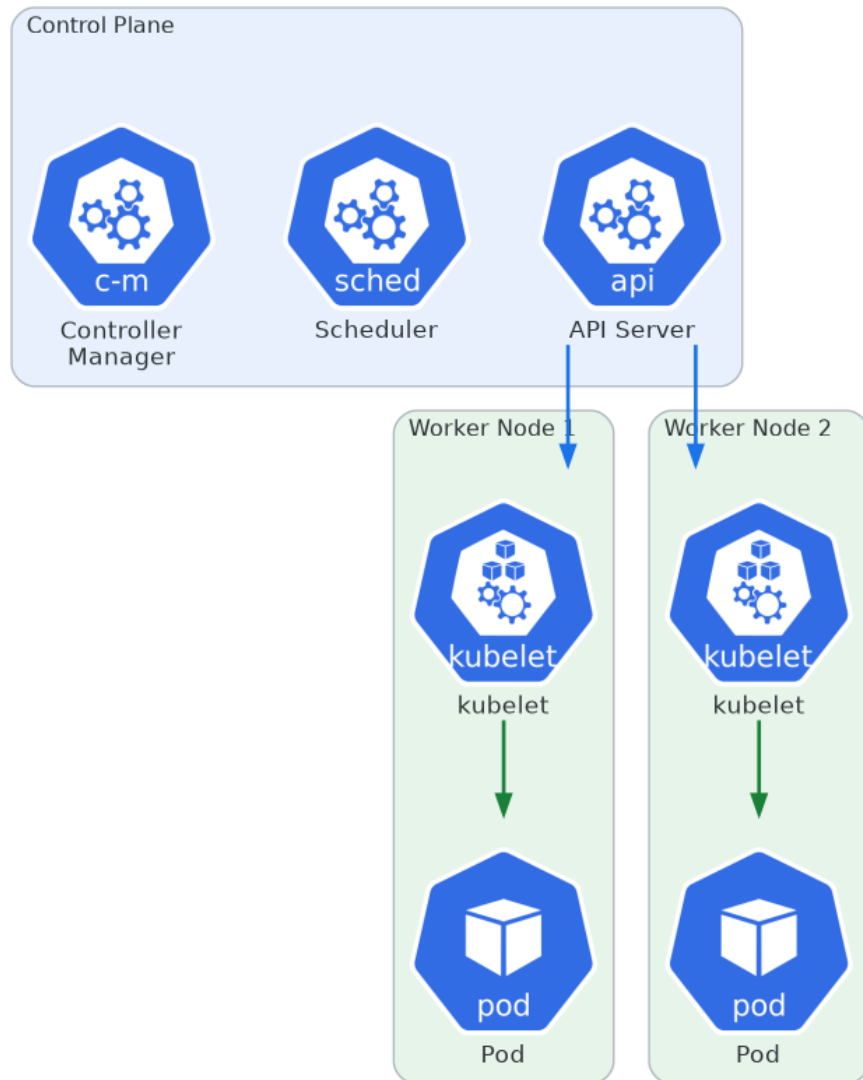


Figure 11: A Kubernetes cluster's control plane schedules and manages workloads running as Pods on worker nodes.

- **etcd** — a small, highly reliable key-value database that stores the entire state of the cluster: every Deployment, every Service, every Pod’s configuration. Think of it as the cluster’s single source of truth.
- **Scheduler (kube-scheduler)** — watches for newly created Pods that have not yet been assigned to a node, and picks the best available node for each one based on resource requirements, constraints, and current node capacity.
- **Controller manager (kube-controller-manager)** — runs a collection of background control loops that watch the cluster’s actual state and push it toward the desired state stored in etcd. For example, if a Deployment says “I want 3 replicas” and only 2 are running, a controller notices and creates one more.

On GKE, Google fully manages the control plane for you. You never SSH into it, patch it, or worry about etcd backups — Google handles all of that behind the scenes, which is one of the biggest advantages of a managed Kubernetes service over running Kubernetes yourself.

Worker Node Components

Worker nodes are the machines that actually run your application containers. Each node runs three key components:

- **kubelet** — an agent that runs on every node and talks to the API server. It receives instructions about which containers should be running on its node, and makes sure they actually are, restarting them if they die.
- **kube-proxy** — handles network rules on each node so that traffic sent to a Service (covered in Chapter 12) gets correctly routed to the right containers, wherever they happen to be running.
- **Container runtime** — the underlying software that actually pulls container images and runs them, such as containerd. This is the layer that does the equivalent of `docker run`, driven entirely by instructions from the kubelet.

Put simply: the control plane decides *what should happen*, and the worker nodes, guided by kubelet, make it happen.

Declarative, Not Imperative

This is one of the most important mental shifts when moving from plain Docker to Kubernetes.

With Docker, you typically work **imperatively** — you type the exact command you want executed right now: `docker run`, `docker stop`, `docker rm`. You are telling the system *how* to do something, step by step.

Kubernetes works **declaratively**. You write a YAML file describing the *desired state* of your system — “I want 3 copies of this container running, using this image, exposing this port” — and you hand that file to Kubernetes with a single command. Kubernetes then continuously compares your desired state against the actual state of the cluster, and takes whatever actions are necessary to close the gap. This is called a **reconciliation loop**, and it never stops running. If a Pod crashes, the loop notices the actual state no longer matches the desired state, and creates a replacement — without you typing another command.

BEST PRACTICE

Always keep your YAML manifests in version control (such as Git). Since Kubernetes is declarative, your YAML files become the authoritative description of your infrastructure, not whatever commands someone happened to run.

This declarative approach is what makes self-healing, scaling, and rollbacks possible without constant human intervention: you state your intent once, and Kubernetes keeps enforcing it.

Talking to Kubernetes: kubectl

kubectl (pronounced “cube-control” or “cube-C-T-L”) is the command-line tool used to interact with a Kubernetes cluster. Every kubectl command is translated into a request sent to the API server.

Here are the core commands you will use constantly throughout the rest of this book.

Applying a Manifest

```
kubectl apply -f deployment.yaml
```

This tells Kubernetes: “read this YAML file and make the cluster’s actual state match it.” If the resources described do not exist yet, they are created. If they already exist, they are updated to match the file. You can run apply again and again safely — this is called being **idempotent**.

Viewing Resources

```
kubectl get pods
kubectl get deployments
kubectl get services
```

kubectl get lists resources of a given type currently known to the cluster, along with a short status summary. Add -o wide for more columns, or -n <namespace> to look in a specific namespace instead of the default one.

Inspecting a Resource in Detail

```
kubectl describe pod my-app-6f8d9c9b7-x2j4k
```

kubectl describe prints a detailed, human-readable report about a single resource, including its configuration, current status, and — critically — an **Events** section at the bottom showing recent scheduling decisions, image pulls, and errors. This is usually the first command to run when something is not working.

Viewing Logs

```
kubectl logs my-app-6f8d9c9b7-x2j4k
```

This prints the standard output of the container running inside that Pod, exactly like docker logs does for a single container. Add -f to stream logs continuously, or --previous to see logs from a container that already crashed and restarted.

Running a Command Inside a Container

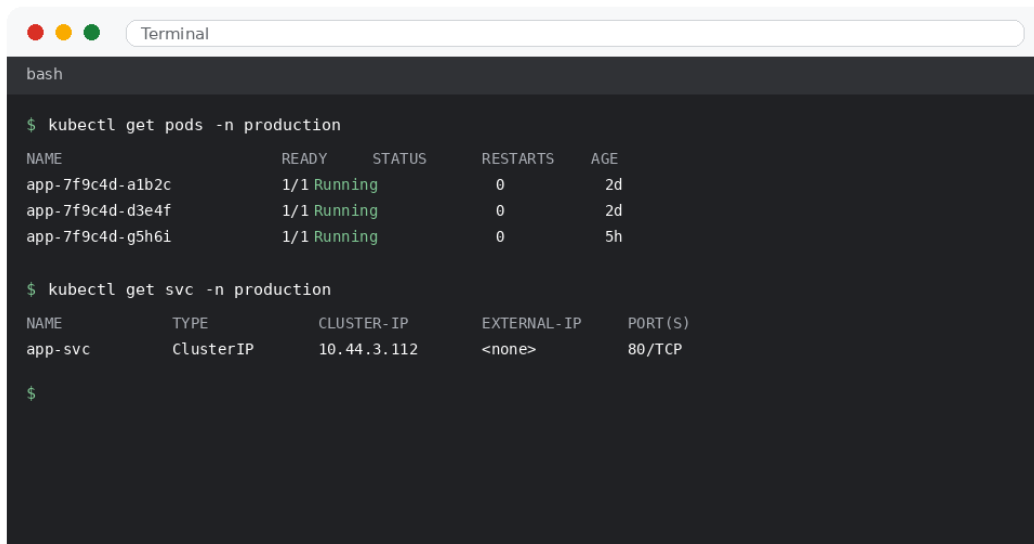
```
kubectl exec -it my-app-6f8d9c9b7-x2j4k -- /bin/sh
```

kubectl exec opens an interactive shell (or runs any one-off command) directly inside a running container, which is invaluable for debugging — checking environment variables, testing DNS resolution, or inspecting files exactly as the application sees them.

TIP

Almost every kubectl command accepts -n <namespace> to target a specific namespace, and --help to show all available flags — for example, kubectl get pods --help.

These five commands — apply, get, describe, logs, and exec — will cover the large majority of your day-to-day interaction with any Kubernetes cluster, including the GKE clusters you will build starting in Chapter 13.



```
bash

$ kubectl get pods -n production
NAME                                READY   STATUS    RESTARTS   AGE
app-7f9c4d-a1b2c                    1/1     Running   0           2d
app-7f9c4d-d3e4f                    1/1     Running   0           2d
app-7f9c4d-g5h6i                    1/1     Running   0           5h

$ kubectl get svc -n production
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
app-svc   ClusterIP   10.44.3.112  <none>        80/TCP

$
```

Figure 12: Illustrative `kubectl get pods` and `kubectl describe pod`.

Key Takeaways

- Kubernetes automates scheduling, self-healing, scaling, and rolling updates for containerized applications — tasks that become impossible to manage by hand once you have more than a handful of containers.
- The control plane (API server, etcd, scheduler, controller manager) decides what should run and where; worker nodes (kubelet, kube-proxy, container runtime) actually run it.
- Kubernetes is declarative: you describe desired state in YAML, and a continuous reconciliation loop drives the real cluster toward that state, rather than you issuing one-off imperative commands.
- On GKE, Google manages the control plane for you, removing the operational burden of running etcd, the API server, and the scheduler yourself.
- `kubectl apply -f`, `get`, `describe`, `logs`, and `exec` form the core toolkit for deploying and debugging workloads on any Kubernetes cluster.

Chapter 11: Pods, Deployments & ReplicaSets

Chapter 10 introduced the control plane and worker nodes that make Kubernetes work, and the declarative model that drives everything. Now it is time to actually deploy something. In this chapter you will learn about the three building blocks Kubernetes uses to run and manage your application containers: the Pod, the ReplicaSet, and the Deployment.

The Pod: Kubernetes' Smallest Deployable Unit

In Kubernetes, you never deploy a container directly. Instead, the smallest unit you can create and manage is a **Pod**. A Pod is a thin wrapper around one or more containers that are always scheduled together, on the same node, sharing the same network address (the same IP) and, optionally, the same storage volumes.

Most of the time, a Pod contains exactly **one** container, and in practice it is common to think of “a Pod” and “my application container” as roughly the same thing. But Kubernetes allows more than one container in a single Pod when those containers are tightly coupled and need to share resources — this is called the **sidecar** pattern. A classic example is a main application container paired with a small logging or metrics-shipping container that reads files the main container writes and forwards them elsewhere. Because both containers in a Pod share the same network namespace, the sidecar can reach the main container over localhost, and because they can share a volume, the sidecar can read files the main container writes to disk.

NOTE

You will rarely create a Pod directly in real projects. Instead, you create a higher-level object — almost always a Deployment — which creates and manages Pods on your behalf. This chapter shows you why.

Why You Need More Than Just a Pod

If you create a single Pod directly and its node crashes, that Pod is gone — nothing recreates it. If you want three copies of your app running for redundancy, you would have to create three separate Pods by hand and keep track of each one yourself. And if you want to roll out a new image version, you would have to manually delete and recreate every Pod, one at a time, hoping you do not cause an outage.

This is exactly the gap that the **Deployment** and **ReplicaSet** objects close.

The Deployment: Your Primary Tool for Running Applications

A **Deployment** is the object you will use, day to day, to run a stateless application on Kubernetes. You describe, declaratively, what you want — which container image, how many copies, what ports to expose, how much CPU and memory each copy needs — and the Deployment controller takes care of creating, monitoring, and updating the underlying Pods to match.

Here is a complete Deployment manifest for an application image stored in Artifact Registry:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
```

```

  app: my-app
template:
  metadata:
    labels:
      app: my-app
  spec:
    containers:
      - name: my-app
        image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
        ports:
          - containerPort: 8080
        resources:
          requests:
            cpu: "250m"
            memory: "256Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"

```

Let's walk through this field by field:

- **apiVersion: apps/v1** — Deployments belong to the apps/v1 API group, one of several versioned APIs the Kubernetes API server exposes.
- **kind: Deployment** — tells Kubernetes what type of object to create.
- **metadata.name** — the name of this Deployment object within its namespace; used when you run commands like `kubectl get deployment my-app-deployment`.
- **spec.replicas: 3** — the desired number of identical Pod copies to keep running at all times. If a Pod dies, this number is what the controller compares against to decide whether to create a replacement.
- **spec.selector.matchLabels** — tells the Deployment which Pods belong to it, by matching on labels. This **must** match the labels defined under `template.metadata.labels`, or the Deployment will not be able to find and manage its own Pods.
- **spec.template** — a full Pod specification, used as the blueprint every time the Deployment needs to create a new Pod. Everything from `template` downward is exactly what you would put in a standalone Pod manifest.
- **containers[].name** — a name for this container within the Pod, used in commands like `kubectl logs my-pod -c my-app`.
- **containers[].image** — the exact container image to run, referenced by its full Artifact Registry path, including the region (`us-central1-docker.pkg.dev`), your project ID, the repository name (`my-repo`), and an explicit version tag (`v1`) rather than `latest`.
- **containers[].ports.containerPort** — documents which port the application listens on inside the container; this is informational for Kubernetes (it does not open a firewall) but important for other objects, like Services, that need to know where to send traffic.
- **resources.requests** — the minimum CPU and memory Kubernetes guarantees to reserve for this container, and what the scheduler uses to decide which node has room for the Pod.
- **resources.limits** — the maximum CPU and memory this container is allowed to use; exceeding the memory limit gets the container killed and restarted, while exceeding the CPU limit simply throttles it.

BEST PRACTICE

Always pin container images to a specific, immutable tag or digest (like `v1`) rather than `latest`. With `latest`, you lose the ability to know exactly what code is running, and rolling back becomes guesswork.

To create this Deployment in your cluster:

```
kubectl apply -f deployment.yaml
```

```
deployment.apps/my-app-deployment created
```

The ReplicaSet: The Layer You Rarely Touch Directly

When you create a Deployment, it does not manage Pods directly. Instead, it creates and manages a **ReplicaSet**, and the ReplicaSet is the object that actually creates and watches over the Pods.

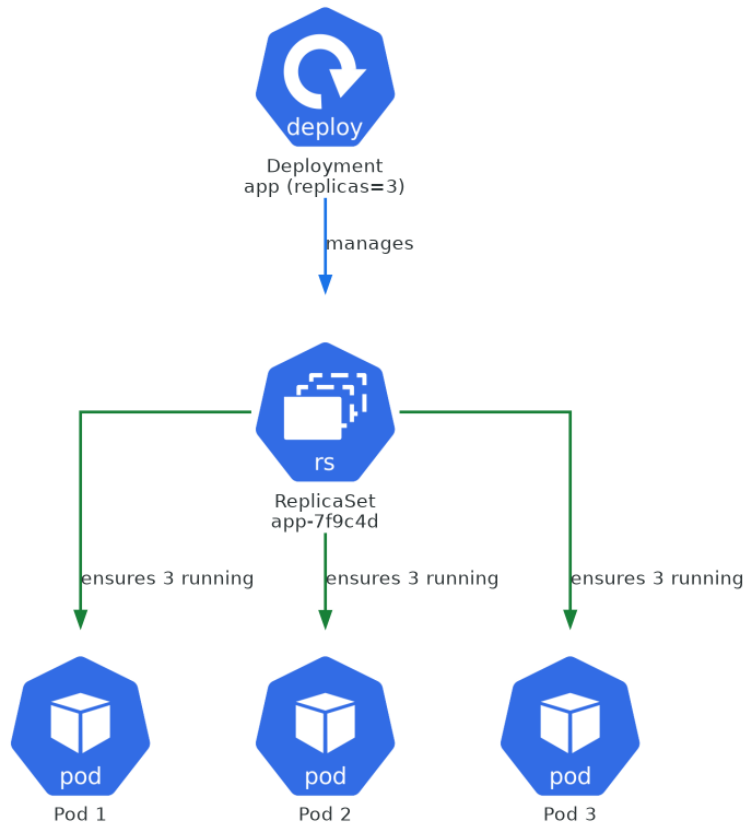


Figure 13: A Deployment manages a ReplicaSet, which in turn ensures the desired number of Pods are always running.

A ReplicaSet’s only job is simple: make sure exactly the requested number of Pods matching a given label selector are running at all times. If you were to manually delete a Pod that belongs to a ReplicaSet, the ReplicaSet notices the count has dropped below its target and immediately creates a new one to replace it.

So why does the Deployment exist as a separate layer on top of the ReplicaSet? Because ReplicaSets are good at maintaining a fixed set of identical Pods, but they have no built-in concept of *changing* what “identical” means over time. The Deployment adds that capability: when you update the container image in a Deployment’s Pod template, the Deployment creates a **new** ReplicaSet for the new version, and gradually shifts Pods from the old ReplicaSet to the new one. This is what enables rolling updates and rollbacks, and it is why, in almost all real-world usage, you create and edit Deployments and let Kubernetes manage the ReplicaSets underneath automatically.

You can see this relationship yourself:

```
kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
my-app-deployment-7d9f8c6b5	3	3	3	2m

Notice the ReplicaSet's name is automatically derived from the Deployment's name, plus a hash — that hash changes every time you roll out a new Pod template.

Rolling Updates and Rollbacks

Suppose you build and push `app:v2` to Artifact Registry, and want to update your running application. You simply edit the `image` field in your Deployment manifest and re-apply it:

```
kubectl set image deployment/my-app-deployment
↪ my-app=us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v2
```

This triggers a **rolling update**: the Deployment creates a new ReplicaSet for `v2`, brings up new Pods gradually, and only terminates old `v1` Pods as new ones become healthy — by default, never taking the whole application offline at once. You can watch this progress with:

```
kubectl rollout status deployment/my-app-deployment
```

```
Waiting for deployment "my-app-deployment" rollout to finish: 2 out of 3 new
↪ replicas have been updated...
deployment "my-app-deployment" successfully rolled out
```

If something goes wrong with the new version — say, it starts crashing or fails health checks — you can instantly revert to the previous working version:

```
kubectl rollout undo deployment/my-app-deployment
```

```
deployment.apps/my-app-deployment rolled back
```

Kubernetes keeps a history of previous ReplicaSets specifically to make this instant rollback possible, without you needing to remember what the old image tag was.

TIP

Run `kubectl rollout history deployment/my-app-deployment` to see a numbered list of past rollouts, and pass `--to-revision=<N>` to `rollout undo` to roll back to a specific one instead of just the immediately preceding version.

Self-Healing in Action

To see self-healing for yourself, list your running Pods, then delete one manually:

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
my-app-deployment-7d9f8c6b5-4k9pl	1/1	Running	0	5m
my-app-deployment-7d9f8c6b5-8zqwr	1/1	Running	0	5m
my-app-deployment-7d9f8c6b5-x2j4k	1/1	Running	0	5m

```
kubectl delete pod my-app-deployment-7d9f8c6b5-x2j4k
```

Within a few seconds, list the Pods again:

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
my-app-deployment-7d9f8c6b5-4k9pl	1/1	Running	0	6m
my-app-deployment-7d9f8c6b5-8zqwr	1/1	Running	0	6m
my-app-deployment-7d9f8c6b5-p7mvz	1/1	Running	0	8s

Notice a brand new Pod (p7mvz) appeared with an age of only a few seconds, replacing the one you deleted. Nobody ran a command to create it — the ReplicaSet controller noticed the actual count (2) had dropped below the desired count (3) and immediately created a replacement, exactly as the reconciliation loop from Chapter 10 promised it would.

WARNING

Self-healing only protects you against Pod failure, not data loss. If your application writes important data to the container's local filesystem instead of a persistent volume or external database, that data is gone forever when the Pod is replaced.

Key Takeaways

- A Pod is Kubernetes' smallest deployable unit, usually wrapping a single container, though multiple tightly-coupled containers (like sidecars) can share one Pod.
- A Deployment is the object you create and edit day to day; it manages a ReplicaSet, which in turn manages the actual Pods — you rarely create or edit ReplicaSets directly.
- Key Deployment fields include replicas (how many copies), selector/template.metadata.labels (which must match), the container image pointing at your Artifact Registry repository, ports, and resources.requests/limits.
- `kubectl rollout status` and `kubectl rollout undo` give you visibility into, and an instant safety net for, rolling out new image versions.
- Deleting a Pod managed by a Deployment demonstrates self-healing directly: a replacement Pod appears automatically because the ReplicaSet continuously reconciles actual state against desired replica count.

Chapter 12: Services & Ingress Networking

In Chapter 11 you deployed an application using a Deployment, and saw Kubernetes create, replace, and roll out Pods automatically. But there is a problem we have not addressed yet: every time a Pod is created or replaced, it gets a brand new internal IP address. If another part of your system — or a user on the internet — had to keep track of individual Pod IP addresses, everything would break the moment Kubernetes replaced a single Pod. Kubernetes solves this with two networking objects: the **Service**, which gives a stable address to a group of Pods, and the **Ingress**, which manages external HTTP(S) traffic into your cluster.

The Problem: Pods Are Disposable, But Traffic Needs a Stable Target

Pods come and go — they are recreated on failure, replaced during rollouts, and rescheduled onto different nodes. Their IP addresses change every time. What you need is a stable, unchanging address that automatically forwards traffic to whichever healthy Pods happen to exist right now. That is exactly what a Service provides.

The Three Core Service Types

Kubernetes offers three main Service types, each solving a different networking need.

ClusterIP (the default)

A ClusterIP Service gets a stable virtual IP address that is only reachable **from inside the cluster**. It is the right choice for internal communication — for example, a front-end Pod talking to a back-end Pod, or a back-end Pod talking to a database Pod. Most Services in a typical application are ClusterIP, because most communication between microservices should never be exposed to the public internet directly.

NodePort

A NodePort Service opens a specific port (in the range 30000–32767) on **every node** in the cluster, and forwards traffic that arrives on that port to the Service's Pods. This makes your application reachable from outside the cluster by hitting `<any-node-IP>:<nodePort>`, but it is a fairly blunt tool — you are relying on knowing node IPs directly, there's no built-in load balancing across multiple nodes from a single external entry point, and it is rarely used for production-facing traffic. It is mostly useful for quick testing or as a low-level building block that other objects (like LoadBalancer) are built on top of.

LoadBalancer

A LoadBalancer Service asks the underlying cloud provider — Google Cloud, in the case of GKE — to provision an actual external load balancer with its own public IP address, which then forwards traffic to the Service. This is the simplest way to expose a single service directly to the internet on GKE, but if you have many services, provisioning one cloud load balancer per Service can become expensive and harder to manage than a single shared entry point — which is where Ingress comes in.

NOTE

In GKE, creating a LoadBalancer Service automatically provisions a real Google Cloud Network Load Balancer behind the scenes — you will see a corresponding forwarding rule appear in the Google Cloud Console within a minute or two of applying the manifest.

Service Type	Reachable From	Typical Use Case
ClusterIP	Inside the cluster only	Internal service-to-service traffic

Service Type	Reachable From	Typical Use Case
NodePort	Outside, via <node-IP>:<port>	Quick testing, low-level building block
LoadBalancer	Public internet, via cloud LB IP	Exposing one service directly, simple setups

BEST PRACTICE

For any real-world HTTP(S) application, prefer ClusterIP Services fronted by a single Ingress over multiple LoadBalancer Services — you get one external IP and one set of routing rules to manage instead of one per service.

A Complete Service Manifest

Here is a ClusterIP Service that targets the my-app-deployment Pods from Chapter 11:

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

A quick field walkthrough:

- **spec.selector** — this is the critical link. Kubernetes looks for every Pod whose labels match `app: my-app` (exactly the label set in the Deployment's Pod template from Chapter 11) and automatically adds them as endpoints for this Service. As Pods are created or destroyed, this list of endpoints updates continuously — you never have to update it by hand.
- **spec.ports.port** — the port the Service itself listens on, which is what other Pods (or clients) connect to.
- **spec.ports.targetPort** — the port on the Pod/container that traffic should actually be forwarded to; this matches the `containerPort` defined in the Deployment.

Apply it the same way as any other manifest:

```
kubectl apply -f service.yaml
```

```
service/my-app-service created
```

DNS-Based Service Discovery

Every Service you create is automatically given a DNS name inside the cluster, thanks to Kubernetes' built-in DNS system (typically CoreDNS). The full form of that name is:

```
<service-name>.<namespace>.svc.cluster.local
```

For the Service above, running in the default namespace, that would be `my-app-service.default.svc.cluster.local`. Any Pod in the cluster can reach it using that full name, or often just the short name `my-app-service` if the calling Pod happens to be in the same namespace. This means application code never needs

to hard-code IP addresses — it just connects to a predictable, human-readable hostname, and Kubernetes handles resolving that name to whichever healthy Pods currently exist.

Routing External Traffic: The Ingress

A Service gives you a stable internal address, and a LoadBalancer Service can expose one thing to the internet — but real applications usually need smarter HTTP routing: sending `/api` to one backend and `/` to another, all under one domain and one public IP, often with TLS termination. That is the job of an **Ingress**.

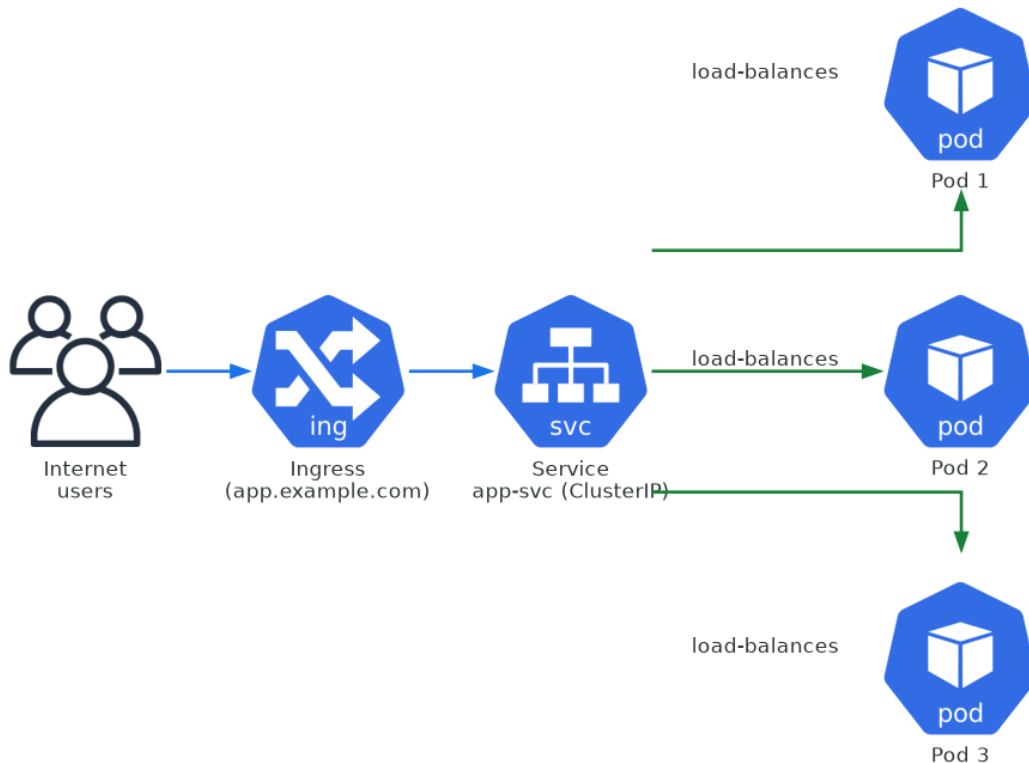


Figure 14: Ingress routes external traffic to a Service, which load-balances requests across a set of Pods.

An Ingress is a set of routing rules. On GKE, creating an Ingress resource causes Google Cloud to automatically provision and configure a Google Cloud Load Balancer that implements those rules, so incoming internet traffic flows: **Internet → Ingress-managed Load Balancer → Service → Pods**.

Here is a complete Ingress performing path-based routing to the Service defined above:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
    kubernetes.io/ingress.class: "gce"
spec:
  rules:
    - http:
        paths:
          - path: /api
```

```

    pathType: Prefix
  backend:
    service:
      name: my-app-service
      port:
        number: 80
- path: /
  pathType: Prefix
  backend:
    service:
      name: my-app-service
      port:
        number: 80

```

Key parts of this manifest:

- **metadata.annotations** — the `kubernetes.io/ingress.class: "gce"` annotation tells GKE to provision its native Google Cloud Load Balancer implementation for this Ingress.
- **spec.rules[].http.paths** — a list of path-matching rules evaluated in order; each one maps an incoming URL path to a backend Service and port.
- **path and pathType: Prefix** — `/api` matches any URL beginning with `/api`, and `/` matches everything else, letting you route different URL paths to different backend Services (in a multi-service application, `/api` might point to a different Service than `/`).
- **backend.service.name / backend.service.port.number** — identifies which Service (and which port on that Service) should receive traffic matching this rule.

Apply the Ingress the same way:

```
kubectl apply -f ingress.yaml
```

```
ingress.networking.k8s.io/my-app-ingress created
```

WARNING

On GKE, provisioning the actual Google Cloud Load Balancer behind a new Ingress can take several minutes. Do not be alarmed if `kubectl get ingress` shows no external IP address immediately after creation — wait and check again.

Once provisioned, check the assigned external IP with:

```
kubectl get ingress my-app-ingress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
my-app-ingress	<none>	*	34.120.10.55	80	4m

Checking Your Work

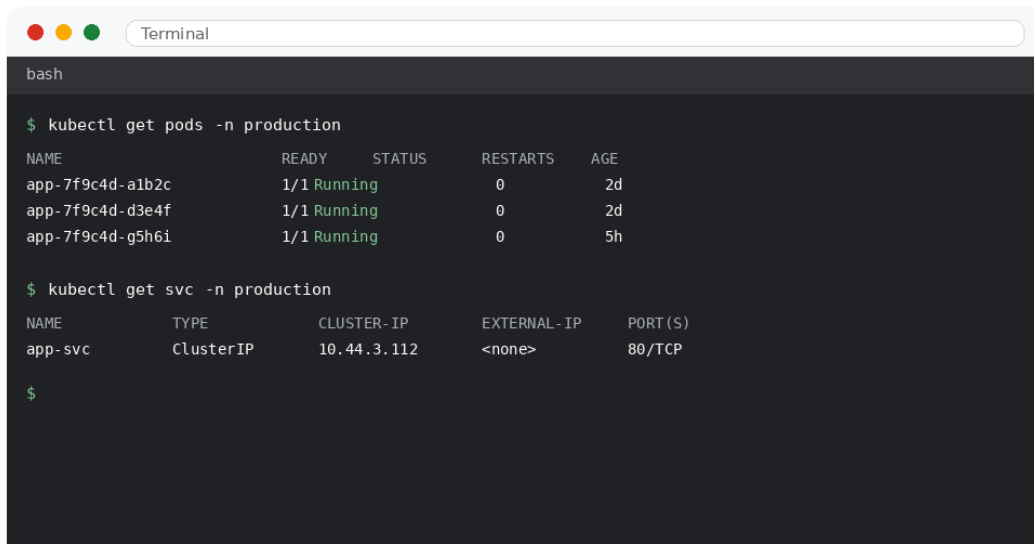
At this point it is worth confirming that both your Pods and your Service are healthy and correctly wired together:

```

kubectl get pods
kubectl get svc

```

Look for Running status and 1/1 (or 3/3) readiness on your Pods, and confirm your Service shows a ClusterIP along with the correct PORT(S) mapping — 80/TCP in the example above. If the Ingress's ADDRESS is reachable in a browser and correctly routes to your application, the full chain — Internet, Ingress, Service, Pods — is working end to end.



```
bash

$ kubectl get pods -n production

NAME                READY   STATUS    RESTARTS   AGE
app-7f9c4d-a1b2c    1/1     Running   0           2d
app-7f9c4d-d3e4f    1/1     Running   0           2d
app-7f9c4d-g5h6i    1/1     Running   0           5h

$ kubectl get svc -n production

NAME    TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
app-svc ClusterIP    10.44.3.112   <none>        80/TCP

$
```

Figure 15: Illustrative `kubectl get pods` and `kubectl get svc`.

Key Takeaways

- Services provide a stable address for a group of Pods whose individual IP addresses constantly change; choose ClusterIP for internal traffic, NodePort for quick low-level testing, and LoadBalancer for direct external exposure of a single service.
- A Service's selector field is what links it to Pods, matching on the same labels used in the Deployment's Pod template — Kubernetes keeps the endpoint list updated automatically as Pods come and go.
- Every Service gets a predictable internal DNS name (`<service-name>.<namespace>.svc.cluster.local`), so application code never needs to hard-code Pod or Service IP addresses.
- Ingress adds HTTP-aware, path-based routing on top of Services, letting one external IP and one Load Balancer serve many different backend paths.
- On GKE, Ingress and LoadBalancer Services both provision real Google Cloud Load Balancer infrastructure automatically, which can take a few minutes to become fully available.

Chapter 13: ConfigMaps, Secrets, Volumes & Storage

Real applications need more than an image and a running Pod. They need configuration values that change between environments, sensitive credentials that must be handled carefully, and — for anything that writes data — storage that survives a container restart. Kubernetes provides three building blocks for this: ConfigMaps, Secrets, and the Volume/PersistentVolume system. This chapter covers all three, and how they come together inside a single Pod.

ConfigMaps: Non-Sensitive Configuration

A ConfigMap holds configuration data as key-value pairs, kept separate from your container image. This means the same image can be deployed to development, staging, and production with different configuration, without ever rebuilding it.

```
kubectl create configmap app-config \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_FLAG_NEW_UI=true
```

This creates a ConfigMap named app-config with two keys. You can also define one declaratively:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  FEATURE_FLAG_NEW_UI: "true"
```

ConfigMaps are plain text. Anyone with read access to the namespace can view their contents, so they should never hold passwords, API keys, or tokens.

Secrets: Sensitive Data

Secrets look and behave almost identically to ConfigMaps, but are intended for sensitive values — database passwords, API tokens, TLS certificates.

```
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=appuser \
  --from-literal=DB_PASSWORD='S3cur3P@ss!'
```

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  DB_USER: YXBwdXNlcg==
  DB_PASSWORD: UzNjdXIzUEBzcyE=
```

WARNING

By default, the values inside a Kubernetes Secret are only base64-encoded, not encrypted. Base64 is a reversible encoding, not a cipher — anyone who can read the Secret object (via `kubectl get secret -o yaml`, an etcd backup, or a leaked manifest) can decode it in one command. Treat Secrets as sensitive by convention and access control, not because Kubernetes protects the bytes for you.

To get real protection you need two additional layers, both of which are worth enabling on any pro-

duction GKE cluster:

- **Encryption at rest / envelope encryption:** GKE can encrypt Secret data in etcd using a key you control in Cloud KMS, so the underlying storage is not plaintext-adjacent base64.
- **RBAC restrictions:** limit which ServiceAccounts, users, and groups can get, list, or watch Secret objects at all. Even with encryption at rest, anyone permitted to call the Kubernetes API and read a Secret still gets the decoded value back — RBAC is what actually gates that.

BEST PRACTICE

Never commit raw Secret YAML (with real values) to source control. Generate Secrets at deploy time from a secret manager (such as Google Secret Manager) or a sealed-secrets style tool, and restrict get/list on Secrets to the smallest possible set of RBAC roles.

Consuming ConfigMaps and Secrets: Environment Variables vs. Mounted Files

Both ConfigMaps and Secrets can be injected into a Pod in two ways.

As Environment Variables

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
      envFrom:
        - configMapRef:
            name: app-config
        - secretRef:
            name: db-credentials
```

envFrom pulls every key from the referenced ConfigMap and Secret into the container's environment automatically, so LOG_LEVEL and DB_PASSWORD are simply available as environment variables inside the container. This is simple and works well for values read once at startup, but two caveats are worth knowing: environment variables are visible to anything that can inspect the process (for example via /proc), and updating the ConfigMap or Secret does not update an already-running container's environment — it requires a Pod restart.

As Mounted Volume Files

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
        - name: secret-volume
          mountPath: /etc/secrets
```

```

      readOnly: true
volumes:
- name: config-volume
  configMap:
    name: app-config
- name: secret-volume
  secret:
    secretName: db-credentials

```

Here, each key in the ConfigMap and Secret becomes a separate file inside the mounted directory (for example `/etc/secrets/DB_PASSWORD` containing the decoded password). This suits applications that expect configuration as files, such as TLS certificates or a `.properties` file, and — unlike environment variables — mounted ConfigMap and Secret volumes are updated automatically by the kubelet when the source object changes, without restarting the Pod (though the application itself still needs to notice the file changed).

TIP

Prefer mounted volumes for genuinely sensitive files like TLS keys, and environment variables for simple flags and non-critical settings — it keeps sensitive material out of environment dumps and process listings.

Storage: PersistentVolumes, PersistentVolumeClaims, and StorageClasses

Containers are ephemeral by design — anything written to a container’s own filesystem disappears when the container restarts. For workloads that need durable storage (databases, file uploads, caches that must survive a restart), Kubernetes provides a three-part abstraction:

- **PersistentVolume (PV)**: a piece of actual storage in the cluster — for example, a GCE Persistent Disk — represented as a Kubernetes object. PVs can be provisioned ahead of time by an administrator, or created automatically on demand.
- **PersistentVolumeClaim (PVC)**: a request for storage made by a Pod’s author, specifying size and access mode, without needing to know exactly which underlying disk will satisfy it.
- **StorageClass**: a template that defines *how* a PV should be dynamically provisioned when a PVC asks for one — which disk type, which zone, which reclaim policy.

The PVC is the piece application authors actually write. Kubernetes matches it to a suitable PV, creating one automatically if a StorageClass is specified (or defaulted).

NOTE

GKE ships with a default StorageClass that provisions a standard GCE Persistent Disk automatically, so in most cases you can create a PVC with no `storageClassName` at all and get working, durable block storage without any manual disk setup.

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi

```


This claims 10 GiB of storage with ReadWriteOnce access (mountable read-write by a single node at a time — the normal mode for a GCE Persistent Disk). A Pod references the claim by name, not the underlying disk:

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod-with-storage
spec:
  containers:
    - name: app
      image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
      volumeMounts:
        - name: data-volume
          mountPath: /var/lib/app/data
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: app-data-pvc
```

Whatever the container writes to `/var/lib/app/data` is persisted on the underlying Persistent Disk. If the Pod is deleted and recreated (even on a different node), the same PVC can be reattached and the data is still there.

Bringing It Together

A typical production Pod combines all three mechanisms: a ConfigMap for non-sensitive settings, a Secret for credentials, and a PVC for durable data — each mounted or injected exactly where the application expects it, with the container image itself staying identical across every environment.

Key Takeaways

- ConfigMaps hold plain-text, non-sensitive configuration; Secrets hold sensitive values but are only base64-encoded by default, not encrypted — enable encryption at rest and restrict Secret access with RBAC for real protection.
- Both can be consumed as environment variables (simple, but static until restart and visible to process inspection) or as mounted files (auto-updates in place, better suited to sensitive files like TLS keys).
- PersistentVolumeClaims let application authors request storage without knowing the underlying disk; StorageClasses control how that storage gets provisioned.
- GKE's default StorageClass automatically provisions a GCE Persistent Disk when a PVC doesn't specify one, so durable block storage works out of the box.
- Data written to a volume backed by a PVC survives Pod deletion and rescheduling, unlike a container's own writable filesystem layer.

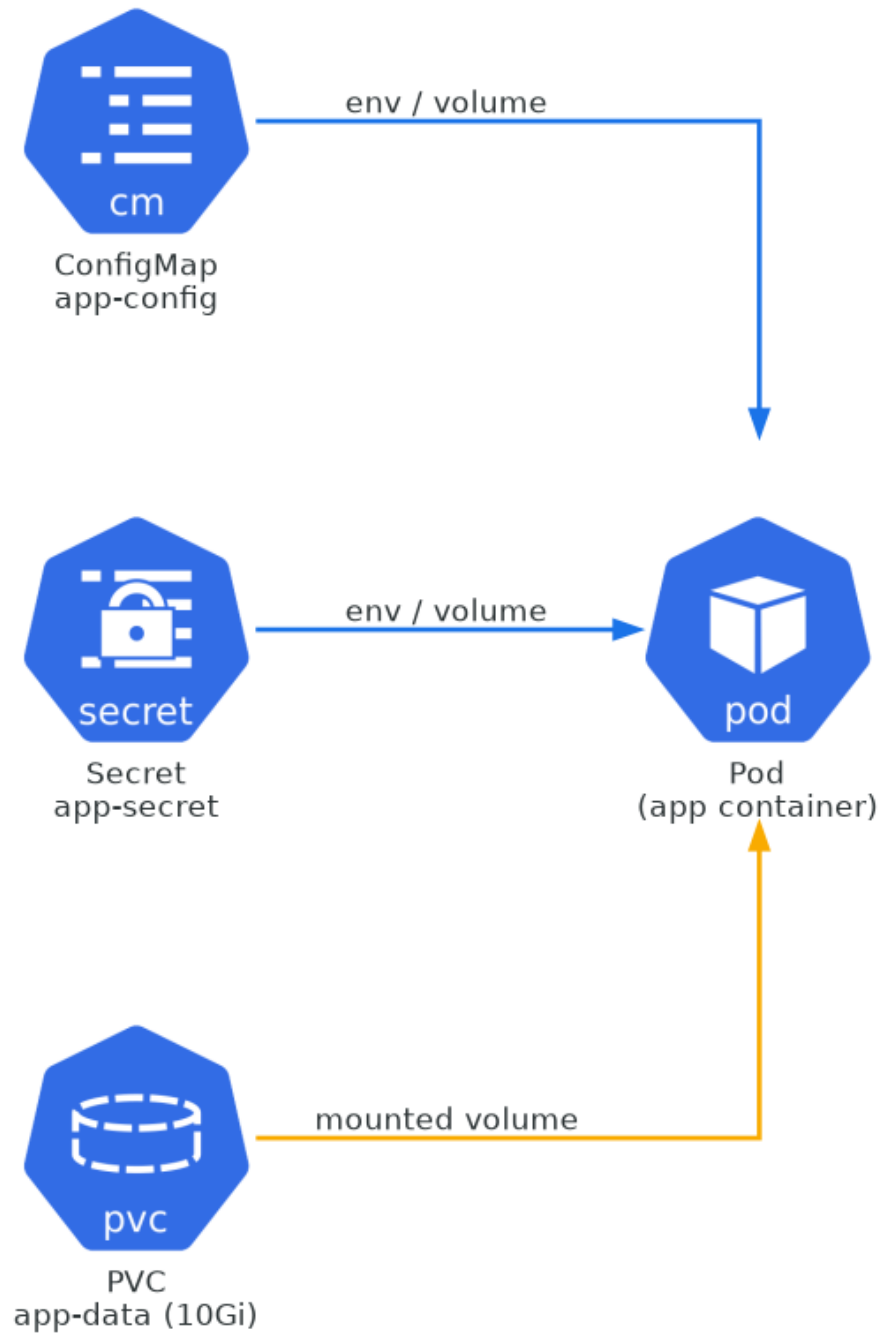


Figure 16: ConfigMaps and Secrets inject configuration into a Pod, while a PersistentVolumeClaim provides durable storage.

Chapter 14: Introduction to GKE

Everything covered so far in the Kubernetes chapters applies to any Kubernetes cluster, anywhere. Google Kubernetes Engine (GKE) is Google Cloud's managed Kubernetes service — the same open-source Kubernetes API and object model, but with Google operating the hardest parts for you. This chapter explains exactly what GKE adds on top of vanilla Kubernetes, the two operating modes you can choose from, and how to create your first cluster.

What GKE Adds on Top of Open-Source Kubernetes

Running Kubernetes yourself means standing up and operating the control plane — the API server, etcd, the scheduler, the controller manager — in addition to the worker nodes that actually run your Pods. That control plane needs to be highly available, patched for security vulnerabilities, upgraded as new Kubernetes versions ship, and monitored around the clock. Getting this wrong is a common source of outages and security incidents in self-managed clusters.

GKE removes that burden entirely for the control plane, and optionally for the nodes as well:

- **Fully managed, Google-operated control plane:** Google runs the API server, etcd, and schedulers for you, across multiple zones for high availability, with a financially backed SLA. You never SSH into a control plane machine or patch etcd yourself.
- **Automatic upgrades and patching:** GKE can automatically apply Kubernetes version upgrades and security patches to both the control plane and (if you choose) the nodes, on a schedule you control, so clusters don't quietly drift onto unsupported or vulnerable versions.
- **Deep integration with the rest of GCP:** IAM roles govern who can create, modify, or access clusters and their resources, down to the Kubernetes RBAC level via Workload Identity; clusters live inside your VPC network with the same firewall rules, subnets, and private connectivity options as any other GCP resource; Cloud Logging and Cloud Monitoring are wired in automatically, so Pod logs, cluster events, and infrastructure metrics show up in the same GCP console you already use for everything else, with no extra logging agent to install and manage.

In short: you still write the same Deployment, Service, ConfigMap, and PVC manifests from earlier chapters — GKE changes who is responsible for keeping the cluster itself alive, patched, and observable.

Autopilot vs. Standard Mode

GKE offers two distinct modes for how much of the cluster you manage yourself. Choosing between them is one of the first decisions when creating a cluster, and it shapes both operational effort and how you think about cost.

Autopilot Mode

In Autopilot mode, Google manages the nodes as well as the control plane. You never see or choose an underlying VM, never size a node pool, and never patch a node's operating system — GKE provisions exactly the compute capacity your Pods request, automatically, and scales it up or down as workloads change.

- Billing is per-Pod, based on the CPU, memory, and storage each Pod actually requests (not per underlying VM), so idle node capacity essentially disappears from your bill.
- Security and configuration best practices (such as not running privileged containers by default) are enforced automatically, which reduces operational and security burden.
- In exchange, Autopilot places some restrictions on what a Pod spec can do — for example, certain privileged or host-level operations are disallowed, since you don't control the underlying node.

Standard Mode

In Standard mode, Google still fully manages the control plane, but you manage node pools yourself: choosing machine types, disk sizes, autoscaling ranges, and how many pools to run.

- You get full control over node shape and placement — useful for workloads with specific hardware needs (GPUs, particular CPU platforms, local SSDs) or very particular cost-optimization strategies (committed-use discounts, preemptible/Spot nodes).
- You take on more operational responsibility: sizing node pools correctly, watching for underutilized nodes, and choosing when to apply node upgrades if you don't fully automate that.

	Autopilot	Standard
Node management	Fully managed by Google	Managed by you (node pools)
Billing basis	Per Pod resource request	Per node (VM)
Operational burden	Lowest	Higher, more control
Flexibility	Some Pod-spec restrictions	Full node and Pod flexibility

BEST PRACTICE

Default to Autopilot for new projects and general-purpose workloads — it minimizes operational overhead and tends to right-size cost automatically. Reach for Standard mode when you have specific node-level requirements (GPUs, custom machine types, specialized autoscaling behavior) that Autopilot cannot express.

Creating a Cluster

Autopilot

```
gcloud container clusters create-auto my-gke-cluster \
  --project=my-gcp-project-123456 \
  --region=us-central1
```

This single command creates a fully working Autopilot cluster named `my-gke-cluster` in the `us-central1` region. There is no node pool to specify — Google provisions node capacity automatically the moment you deploy your first workload.

Standard, with an Explicit Node Pool

```
gcloud container clusters create my-gke-cluster \
  --project=my-gcp-project-123456 \
  --zone=us-central1-a \
  --num-nodes=3 \
  --machine-type=e2-medium
```

This creates a Standard-mode cluster in the `us-central1-a` zone with one node pool of three `e2-medium` nodes. Unlike Autopilot, you are explicitly choosing the machine type and node count here — and you are billed for those three VMs whether or not Pods are currently using their full capacity.

NOTE

`create-auto` provisions a regional cluster by default with a highly available control plane spread across zones, while the plain `create` command in the example above targets a single zone; Standard mode also supports `--region` for a multi-zone, more resilient control plane and node footprint at a higher baseline cost.

Node Pools in Standard Mode

A node pool is a group of nodes within a cluster that all share the same configuration — the same machine type, disk type and size, and autoscaling settings. Every Standard-mode cluster has at

least one node pool (created automatically by the create command above), but you are not limited to one.

Multiple node pools let you run different workload shapes side by side in the same cluster:

```
gcloud container node-pools create high-memory-pool \
  --cluster=my-gke-cluster \
  --zone=us-central1-a \
  --machine-type=e2-highmem-4 \
  --num-nodes=2
```

This adds a second pool, `high-memory-pool`, built from a memory-optimized machine type, alongside the original default pool. A memory-hungry workload can then be scheduled specifically onto that pool (using node selectors or taints and tolerations), while lighter workloads stay on the default pool — all within a single cluster and a single control plane.

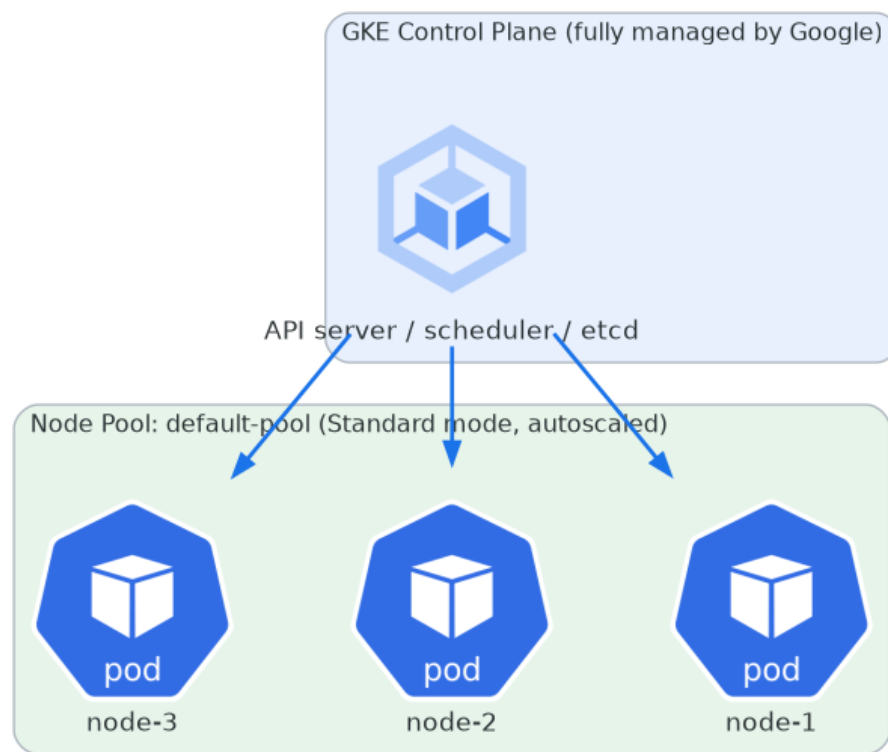


Figure 17: GKE fully manages the control plane; you manage (or let Autopilot manage) the node pools that run your workloads.

The relationship is straightforward: one managed control plane sits above one or more node pools, each pool being a set of identically configured nodes; the scheduler places Pods onto whichever pool best matches their resource requests and constraints.

Key Takeaways

- GKE keeps the same open-source Kubernetes API you already know, but Google fully manages and automatically upgrades the control plane, and integrates it deeply with IAM, VPC networking, and Cloud Logging/Monitoring.
- Autopilot mode manages nodes for you and bills per Pod resource request, minimizing opera-

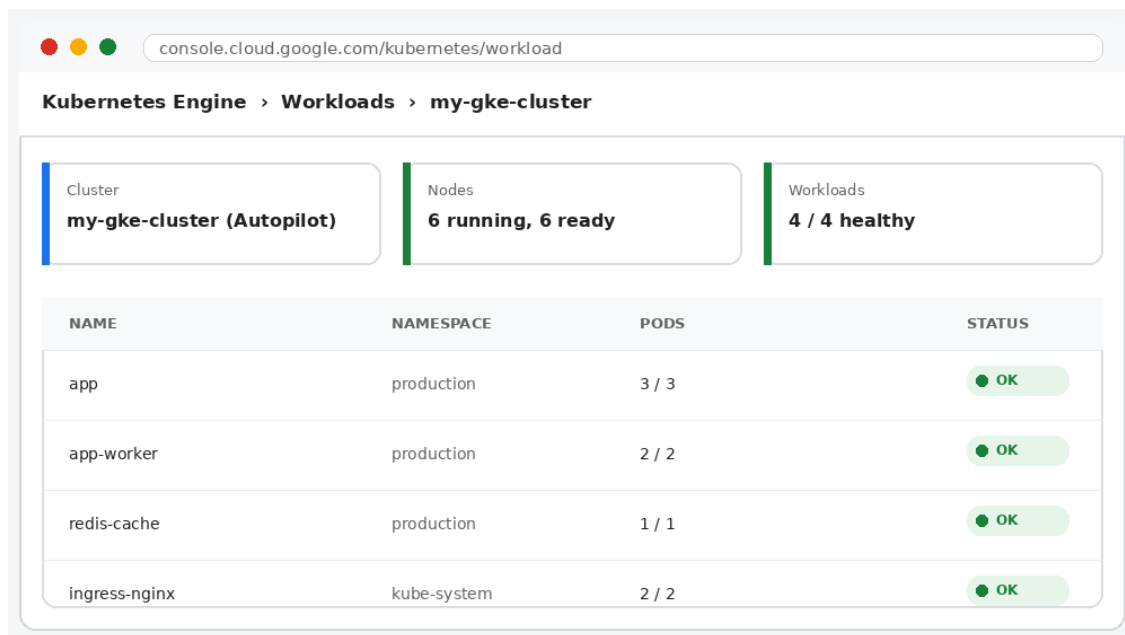


Figure 18: Illustrative the GKE Workloads console, showing healthy Deployments.

- tional burden at the cost of some Pod-spec flexibility.
- Standard mode gives you full control over node pools — machine types, sizing, autoscaling — at the cost of more operational responsibility.
 - Create an Autopilot cluster with `gcloud container clusters create-auto`; create a Standard cluster with an explicit node pool using `gcloud container clusters create --num-nodes ... --machine-type ....`
 - A node pool is a group of identically configured nodes; Standard-mode clusters can run multiple node pools to serve different workload shapes side by side.

Chapter 15: Deploying to GKE from Artifact Registry

This chapter ties together everything from the previous chapters into a single, complete workflow: build a container image, push it to Artifact Registry, and deploy it to a running GKE cluster using a Deployment and a Service. If you have followed along since Chapter 7, every command here should feel familiar — this is the payoff.

Step 1: Build and Push the Image

If you have a local Dockerfile, build and tag it directly against the Artifact Registry image path:

```
docker build -t us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .
docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
```

Alternatively, using Cloud Build from Chapter 8, the same result is produced without a local Docker daemon at all:

```
gcloud builds submit --tag
  us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1 .
```

Either path ends the same way: a single image, `us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1`, sitting in Artifact Registry, ready to be pulled by anything with the right permissions.

TIP

Always deploy an explicit, immutable tag (like `v1`) rather than `latest`. GKE pulls images by exact reference, and `latest` makes it ambiguous which build is actually running in the cluster at any given moment.

Step 2: Write the Deployment and Service Manifest

A minimal but complete production-style manifest needs two objects: a Deployment to run and manage Pods, and a Service to expose them on a stable network address. Both can live in the same YAML file, separated by `---`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
          ports:
            - containerPort: 8080
          resources:
```

```

      requests:
        cpu: "250m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
    ---
apiVersion: v1
kind: Service
metadata:
  name: app-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080

```

A few points worth calling out:

- The `image` field references the exact Artifact Registry path built in Step 1 — GKE nodes will pull this precise image.
- `replicas: 3` tells the Deployment to keep three identical Pods running at all times, rescheduling automatically if one fails or its node goes down.
- The Service's selector (`app: my-app`) must match the Pod template's labels; this is how the Service knows which Pods to route traffic to.
- `type: LoadBalancer` provisions an external GCP network load balancer with a public IP address automatically — no manual load balancer setup required.

Step 3: Apply the Manifest

```
kubectl apply -f deployment.yaml
```

```
deployment.apps/app-deployment created
service/app-service created
```

`kubectl apply` sends the manifest to the cluster's API server, which reconciles the current state of the cluster toward the desired state described in the YAML — creating the Deployment, its ReplicaSet, and the three Pods, plus the Service and its backing load balancer.

Step 4: Verify the Deployment

```
kubectl get pods
```

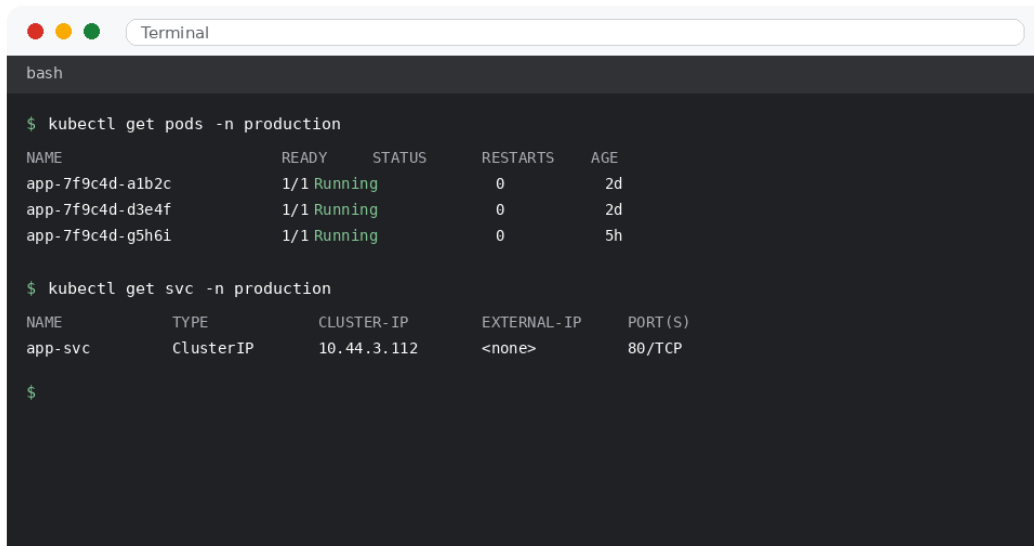
NAME	READY	STATUS	RESTARTS	AGE
app-deployment-7d9f8c6b5d-2xk9p	1/1	Running	0	45s
app-deployment-7d9f8c6b5d-8mv2q	1/1	Running	0	45s
app-deployment-7d9f8c6b5d-lq7wt	1/1	Running	0	45s

All three Pods show Running with 1/1 containers ready — exactly what `replicas: 3` requested.

```
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
app-service	LoadBalancer	10.44.3.201	34.123.45.67	80:31893/TCP	60s

The EXTERNAL-IP column is the public address of the provisioned load balancer. Once it moves from <pending> to an actual IP (this can take a minute or two on first creation), the application is reachable at <http://34.123.45.67>.

A terminal window titled "Terminal" showing the output of two kubectl commands. The first command, 'kubectl get pods -n production', lists three pods: 'app-7f9c4d-a1b2c', 'app-7f9c4d-d3e4f', and 'app-7f9c4d-g5h6i', all in a 'Running' state. The second command, 'kubectl get svc -n production', lists a single service 'app-svc' of type 'ClusterIP' with a cluster IP of '10.44.3.112' and an external IP of '<none>'.

```
bash
$ kubectl get pods -n production
NAME                                READY   STATUS    RESTARTS   AGE
app-7f9c4d-a1b2c                    1/1     Running   0           2d
app-7f9c4d-d3e4f                    1/1     Running   0           2d
app-7f9c4d-g5h6i                    1/1     Running   0           5h

$ kubectl get svc -n production
NAME    TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
app-svc ClusterIP    10.44.3.112   <none>        80/TCP

$
```

Figure 19: Illustrative kubectl output showing Running pods and an external Service IP.

How GKE Nodes Authenticate to Artifact Registry

A natural question at this point: nobody ran `docker login` or `gcloud auth configure-docker` on the GKE nodes themselves, so how did they pull a private image from Artifact Registry?

The answer is that GKE nodes authenticate as their attached Google Cloud service account, not as a human developer. Every GKE node runs as some service account — either the Compute Engine default service account, or a dedicated node service account you specify at cluster/node-pool creation time — and the kubelet on each node uses that service account's identity, obtained automatically from the node's metadata server, to pull images. As long as that service account holds `roles/artifactregistry.reader` (or broader) on the repository, the pull succeeds silently, with no interactive login step anywhere.

```
gcloud artifacts repositories add-iam-policy-binding my-repo \
  --project=my-gcp-project-123456 \
  --location=us-central1 \
  --member="serviceAccount:gke-node-sa@my-gcp-project-123456.iam.gserviceaccount.com" \
  --role="roles/artifactregistry.reader"
```

In many default GKE setups, the Compute Engine default service account is already granted broad project-level access (including effective Artifact Registry read access) the first time a repository is created in the project, so pulls simply work with no extra configuration. That default is convenient for getting started, but it is also broader than necessary for a production environment.

BEST PRACTICE

For production clusters, create a dedicated, minimally scoped node service account and grant it `roles/artifactregistry.reader` explicitly on only the repositories your workloads actually need, rather than relying on the Compute Engine default service account's broad project-level permissions.

WARNING

If Pods get stuck in `ImagePullBackOff`, the most common cause on GKE is not a typo in the image path — it's the node's service account lacking `artifactregistry.reader` on that specific repository. Check `kubectl describe pod <pod-name>` for the exact pull error before assuming the image itself is missing.

This is the key contrast with Chapter 7: a developer's laptop authenticates interactively (via ADC or a service account key), while GKE nodes authenticate silently and automatically through their attached service account identity — there is no docker login step to configure on the cluster side at all.

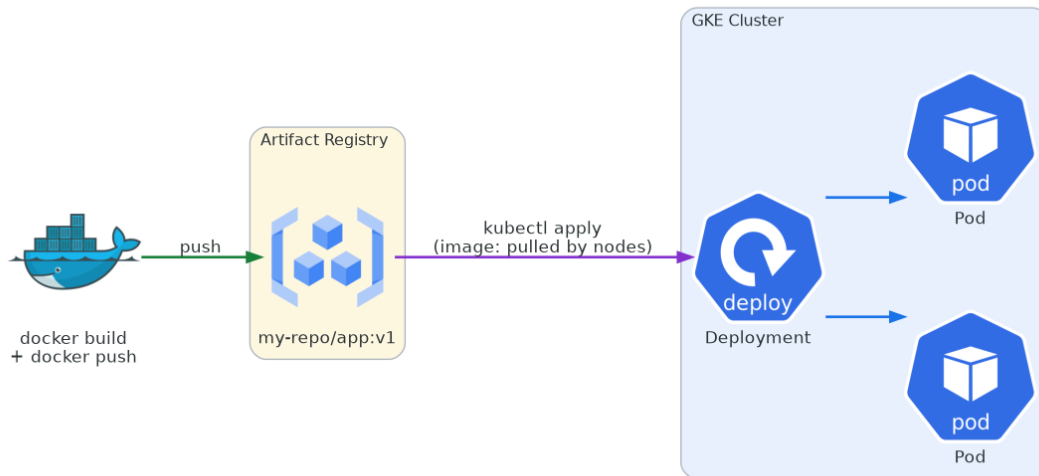


Figure 20: An image built and pushed to Artifact Registry is pulled by GKE nodes the moment a Deployment referencing it is applied.

Key Takeaways

- The end-to-end flow is: build the image, push it to Artifact Registry with an explicit tag, reference that exact path in a Deployment manifest, then `kubectl apply -f` and verify with `kubectl get pods` and `kubectl get svc`.
- Always deploy immutable, versioned tags rather than `latest`, so it is always clear which build is running in the cluster.
- A type: `LoadBalancer` Service automatically provisions a GCP network load balancer with a public IP, with no manual setup required.
- GKE nodes authenticate to Artifact Registry using their attached Google Cloud service account (the Compute Engine default, or a dedicated node service account) — no `docker login` is ever run on the nodes.
- Grant the node's service account `roles/artifactregistry.reader` explicitly in production rather than relying on default project-level permissions, and check `kubectl describe pod` first whenever a Pod shows `ImagePullBackOff`.

Chapter 16: GKE Networking

Every Kubernetes cluster needs a networking model: how do Pods get IP addresses, how do Services get discovered, and how does traffic from the internet reach a Pod running somewhere inside the cluster? On GKE, Google Cloud's networking stack is deeply integrated with Kubernetes' own networking primitives, and understanding that integration is essential once you move past `kubectl port-forward` and start exposing real applications. This chapter covers how GKE clusters get their IP addressing (VPC-native networking), how external traffic reaches your Services (Ingress and the newer Gateway API), and how to lock down traffic between Pods (Network Policies).

VPC-Native Clusters and Alias IP Ranges

Every GKE cluster needs to hand out two pools of IP addresses: one for Pods, and one for Services (the stable virtual IPs behind a `ClusterIP`). Historically, GKE offered two ways to do this.

The legacy approach, **routes-based networking**, created custom static routes in the VPC's routing table for every node, so traffic to a Pod IP would be routed to the correct node. This worked, but it did not scale well — VPC routing tables have hard limits on the number of custom routes, and the Pod and Service IP ranges were not natively understood by the rest of Google Cloud's networking (firewall rules, VPC peering, Shared VPC, and Cloud VPN all had to work around it).

The modern default — and the approach used for every new cluster today — is **VPC-native networking**, which uses **alias IP ranges**. Instead of custom routes, each GKE node is assigned a secondary IP range from the VPC subnet, and Pods draw their addresses directly from that range. Because these are genuine VPC-level alias IP ranges rather than routing table hacks, Google Cloud's networking layer treats Pod IPs as first-class citizens: they are visible to VPC firewall rules, they work cleanly across VPC Peering and Shared VPC, and they scale far beyond the old custom-route limits.

```
gcloud container clusters create my-gke-cluster \
  --zone us-central1-a \
  --enable-ip-alias \
  --cluster-ipv4-cidr=/17 \
  --services-ipv4-cidr=/22
```

The `--enable-ip-alias` flag is what makes a cluster VPC-native. `--cluster-ipv4-cidr` sizes the secondary range used for Pod IPs, and `--services-ipv4-cidr` sizes the secondary range used for Service (`ClusterIP`) addresses. If you omit these sizing flags, GKE picks sensible defaults automatically.

BEST PRACTICE

Always create new clusters as VPC-native. Routes-based clusters are considered legacy, cannot be migrated in place, and are blocked outright in some organizational policies. If you inherit an old routes-based cluster, plan a rebuild rather than trying to convert it.

Sizing the Pod CIDR deserves a moment of thought before you create a cluster, because it is difficult to change afterward. Each node reserves a fixed-size block of Pod IPs (by default enough for 110 Pods per node), so the Pod CIDR needs to be large enough to cover $(\text{number of nodes}) \times (\text{Pods per node block})$ for the maximum size your cluster might ever grow to, including future node pools. Under-sizing this range is one of the most common GKE scaling mistakes, since it silently caps how many nodes the cluster can ever have.

Exposing Services: GKE Ingress and the Gateway API

A `ClusterIP` Service is only reachable from inside the cluster. To expose an application to the internet with a single stable IP address, a TLS certificate, and Layer-7 (HTTP-aware) routing across multiple backend Services, GKE offers two mechanisms: the long-standing **GKE Ingress controller**, and the newer, more general **Gateway API**.

GKE Ingress is built on the standard Kubernetes Ingress resource. When you create an Ingress object on GKE, the built-in GKE Ingress controller watches for it and automatically provisions a Google Cloud external HTTP(S) Load Balancer on your behalf — no manual load balancer configuration required. This has been the default, well-understood way to expose HTTP(S) traffic on GKE for years, and it is what most existing GKE documentation, tutorials, and production clusters use today.

The **Gateway API** is the Kubernetes project's successor to Ingress. It is a more expressive, role-oriented API (splitting cluster-wide GatewayClass/Gateway configuration from per-team HTTPRoute objects) designed to support advanced scenarios that the original Ingress spec was never built for: fine-grained traffic splitting, multi-cluster routing, and clean multi-tenant setups where a platform team owns the load balancer and application teams own their own routes. Google is actively investing in the Gateway API as the direction for GKE networking going forward, and it is the recommended choice for new multi-cluster or multi-tenant designs. For this chapter, however, we focus on GKE Ingress as the primary worked example, since it remains the simpler and more widely deployed option for a single cluster exposing a handful of Services.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: default
  annotations:
    kubernetes.io/ingress.class: "gce"
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app-service
                port:
                  number: 80
```

The `kubernetes.io/ingress.class: "gce"` annotation tells GKE that this Ingress should be handled by its built-in controller (this is also the default class on GKE, so it can often be omitted). Once this manifest is applied with `kubectl apply -f app-ingress.yaml`, GKE provisions a Google Cloud external HTTP(S) Load Balancer, allocates a static external IP, configures a URL map that routes the `/` path to `app-service`, and creates the associated health checks and backend services automatically. Provisioning typically takes a few minutes; you can check progress with `kubectl get ingress app-ingress` and watch for an `ADDRESS` to appear.

NOTE

The GKE Ingress controller reads the referenced Service's associated Pods directly through a NEG (Network Endpoint Group), so traffic goes straight from the Google Cloud load balancer to Pod IPs rather than bouncing through a node's kube-proxy rules — this gives lower latency and more accurate load balancing than a purely Service-based path.

Restricting Pod-to-Pod Traffic with Network Policies

By default, Kubernetes networking is flat: any Pod can send traffic to any other Pod in the cluster, regardless of namespace. That is convenient for getting started, but it is a poor security posture for production — a compromised front-end Pod should not necessarily be able to talk directly to a database Pod. **Network Policies** let you define, in a declarative and namespace-scoped way, exactly which traffic is allowed.

GKE enforces Network Policies using either Calico (the traditional option, enabled with `--enable-network-policy` at cluster creation) or, on newer clusters, GKE's Dataplane V2 (built on Cilium), which supports Network Policies natively without any extra add-on. Either way, the NetworkPolicy resource itself is standard Kubernetes API and portable across providers.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-except-frontend
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: frontend
```

This policy selects Pods labeled `app: backend` and, once selected, changes them from “accept all ingress” to “accept only what is explicitly listed.” The single ingress rule allows traffic only from Pods labeled `app: frontend` in the same namespace; every other source is denied. Note the effect of `podSelector`: a NetworkPolicy only restricts traffic to Pods it selects — Pods not matched by any policy remain fully open, so a common pattern is to also apply a default-deny-all policy across a namespace and then layer specific allow rules like the one above on top of it.

WARNING

Network Policies only take effect if the cluster's networking implementation actually enforces them. Creating a NetworkPolicy object on a cluster without policy enforcement enabled (an older routes-based cluster without Calico or Dataplane V2, for example) will silently have no effect — always verify enforcement is enabled before relying on policies for security.

Key Takeaways

- Always create GKE clusters as VPC-native (`--enable-ip-alias`), which uses genuine VPC alias IP ranges for Pods and Services instead of the legacy, less scalable routes-based model.
- Size the Pod CIDR range generously at cluster creation time — it is very difficult to change later and silently caps maximum cluster size if under-provisioned.
- GKE Ingress automatically provisions a Google Cloud external HTTP(S) Load Balancer from a standard Kubernetes Ingress resource; the Gateway API is the more advanced, actively developed successor recommended for multi-cluster and multi-tenant routing.
- Network Policies restrict Pod-to-Pod traffic using label selectors, but require an enforcement layer (Calico or Dataplane V2) to actually take effect on GKE.
- A NetworkPolicy only restricts Pods it selects — pair a default-deny policy with specific allow rules for real isolation.

Chapter 17: GKE Identity & Security

Getting workloads running on GKE is one milestone; running them securely is the next, and arguably more important one. This chapter covers how identity and access work in GKE at multiple layers: how individual Pods authenticate to Google Cloud APIs without ever holding a static credential, how GCP IAM and Kubernetes RBAC work together to control who can do what, how Binary Authorization can block risky images from ever running, and the basic Pod-level hardening settings every production workload should have.

Workload Identity: Pods Authenticating to GCP APIs

Before Workload Identity existed, a common pattern was to grant a GCP service account key or broad node-level permissions to every Pod on a node, so any application code running there could call GCP APIs — including pulling images from Artifact Registry, writing to Cloud Storage, or reading secrets. The problem is that this is a shared, overly broad credential: every Pod on that node inherits whatever permissions the node's service account has, whether it needs them or not.

Workload Identity solves this by letting a Kubernetes ServiceAccount (KSA) impersonate a specific GCP IAM service account. When a Pod uses that KSA, GKE's metadata server transparently issues it short-lived, scoped credentials for the mapped GCP service account — no key file, no broad node-wide access, and credentials that automatically rotate and expire.

Setting this up has three parts: creating the GCP service account with the permissions the workload actually needs, creating a matching Kubernetes ServiceAccount, and binding the two together with an IAM policy binding.

```
gcloud iam service-accounts create app-workload-sa \
  --project=my-gcp-project-123456 \
  --display-name="App workload service account"

gcloud iam service-accounts add-iam-policy-binding \
  app-workload-sa@my-gcp-project-123456.iam.gserviceaccount.com \
  --project=my-gcp-project-123456 \
  --role="roles/iam.workloadIdentityUser" \
  --member="serviceAccount:my-gcp-project-123456.svc.id.goog[default/app-ksa]"
```

The `add-iam-policy-binding` command is the core of the setup: it grants the Kubernetes ServiceAccount named `app-ksa` in the default namespace permission to act as `app-workload-sa`. The special member string format, `PROJECT_ID.svc.id.goog[NAMESPACE/KSA_NAME]`, is how GCP IAM expresses “this specific Kubernetes ServiceAccount, in this specific cluster's workload identity pool.”

Next, create the Kubernetes ServiceAccount and annotate it to point back at the GCP service account:

```
kubectl create serviceaccount app-ksa --namespace default

kubectl annotate serviceaccount app-ksa \
  --namespace default \
  iam.gke.io/gcp-service-account=app-workload-sa@my-gcp-project-
↪ 123456.iam.gserviceaccount.com
```

Finally, the Pod spec references the KSA using `serviceAccountName`, exactly as it would with any other Kubernetes ServiceAccount:

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
  namespace: default
```

```
spec:
  serviceAccountName: app-ksa
  containers:
  - name: app
    image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
```

With this in place, any GCP client library or `gcloud/gsutil` call made from inside this Pod automatically picks up short-lived credentials scoped to `app-workload-sa` — including, for example, permission to pull further images from Artifact Registry or read a Cloud Storage bucket, provided that IAM role is granted to `app-workload-sa` separately.

BEST PRACTICE

Grant each workload's GCP service account only the specific IAM roles it needs (least privilege), and use a distinct GCP service account per application rather than one shared account for the whole cluster — this keeps a compromised Pod's blast radius small.

IAM and RBAC Working Together

GKE security operates at two layers that are easy to confuse. **GCP IAM** controls access to the cluster and to Google Cloud resources at large — for example, who can call `gcloud container clusters` commands, who can view cluster configuration in the console, or (as above) what a Pod's identity can do against GCP APIs. **Kubernetes RBAC** (Role-Based Access Control) controls access *inside* the Kubernetes API itself — who can `kubectl get pods`, who can create Deployments in a given namespace, and so on.

A common setup grants broad GCP IAM roles (like `roles/container.viewer` or `roles/container.developer`) to let a person authenticate to the cluster at all, and then layers Kubernetes Role and RoleBinding objects on top to restrict what they can actually do once inside, often scoped per namespace so that a team can manage its own workloads without seeing or touching another team's.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-team-edit
  namespace: dev-team-namespace
subjects:
- kind: User
  name: dev-user@example.com
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: edit
  apiGroup: rbac.authorization.k8s.io
```

This binding grants the built-in `edit` ClusterRole, but scoped only to the `dev-team-namespace` namespace, because it is created as a `RoleBinding` rather than a `ClusterRoleBinding`. The user can create, update, and delete most resources in that namespace but has no visibility into other namespaces.

NOTE

GCP IAM and Kubernetes RBAC are independent systems that both must grant access — a user needs a GCP IAM role sufficient to reach the cluster's API server, and then Kubernetes RBAC permissions to actually act once there. Removing one without the other does not fully revoke access.

Binary Authorization: Gating Deployment on Trust

Chapter 9 covered scanning container images for known vulnerabilities before they are pushed or deployed. **Binary Authorization** is what turns that scanning into an enforced policy at deploy time, rather than a report someone might ignore. It is a GKE (and Cloud Run) feature that requires images to satisfy one or more attestations before the cluster will run them at all.

A typical policy requires that an image either passed a vulnerability scan below a defined severity threshold, or was explicitly signed off by a trusted attester (for example, a CI pipeline attesting “this image passed all tests and security gates”), or both. If an image does not carry the required attestation, the GKE admission controller for Binary Authorization rejects the Pod outright — it never starts running, regardless of what the Deployment manifest says.

```
gcloud container clusters update my-gke-cluster \
  --zone us-central1-a \
  --binauthz-evaluation-mode=PROJECT_SINGLETON_POLICY_ENFORCE
```

This command enables Binary Authorization enforcement on an existing cluster using the project’s single Binary Authorization policy. From this point on, every Pod creation is checked against that policy before the kubelet is allowed to pull and run the image.

WARNING

Enabling Binary Authorization enforcement on a cluster that already has workloads running unattested images will not stop those already-running Pods, but it will block any new rollout, scale-up, or Pod restart until the images satisfy the policy — plan a migration window and make sure your CI pipeline is producing valid attestations before enforcing this on a production cluster.

Pod Security Basics

Even with identity and deploy-time gating handled, individual Pods should be configured defensively, since a vulnerability inside application code is still possible. A `securityContext` on the Pod or container lets you enforce several hardening settings directly in the manifest.

```
apiVersion: v1
kind: Pod
metadata:
  name: hardened-app-pod
spec:
  containers:
    - name: app
      image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        readOnlyRootFilesystem: true
        allowPrivilegeEscalation: false
        capabilities:
          drop:
            - ALL
```

Each field closes off a specific escape route: `runAsNonRoot` and `runAsUser` prevent the container process from running as root inside the container, `readOnlyRootFilesystem` stops the container from writing to its own filesystem (forcing any needed writable storage into an explicit volume), `allowPrivilegeEscalation: false` blocks a process from gaining more privileges than its parent, and dropping ALL Linux capabilities removes powerful kernel-level permissions the container almost certainly does not need (any specific capability it does need, such as binding to a low port, can be added back individually).

These per-Pod settings are formalized cluster-wide by the **Pod Security Standards**, a Kubernetes-native replacement for the older, now-removed PodSecurityPolicy. They define three levels — privileged (unrestricted), baseline (blocks known privilege escalations but stays broadly compatible), and restricted (enforces the hardened settings shown above and more) — which can be applied per namespace via labels, giving platform teams a simple way to enforce a minimum security bar across every workload in a namespace.

Key Takeaways

- Workload Identity binds a Kubernetes ServiceAccount to a GCP IAM service account, giving Pods short-lived, scoped GCP credentials instead of broad node-level access or static keys.
- GCP IAM controls access to the cluster and GCP resources; Kubernetes RBAC controls access inside the Kubernetes API — both layers must grant access, and both should follow least privilege.
- Binary Authorization enforces, at deploy time, that images meet vulnerability-scanning and/or signing requirements before GKE will run them, directly building on the scanning practices from Chapter 9.
- Harden Pods with `runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, and dropped Linux capabilities.
- Pod Security Standards (privileged, baseline, restricted) let platform teams enforce these hardening rules consistently across namespaces.

Chapter 18: Autoscaling in GKE

One of Kubernetes' biggest advantages over manually managed infrastructure is its ability to react to changing demand automatically. On GKE this happens at two distinct levels: scaling the number of Pod replicas that serve a workload, and scaling the number of nodes that host those Pods. This chapter covers both, starting with the Horizontal Pod Autoscaler that adjusts replica counts, briefly touching the Vertical Pod Autoscaler that adjusts per-Pod resource requests, and finishing with how GKE Standard and Autopilot modes handle node-level scaling very differently.

Horizontal Pod Autoscaler (HPA)

The Deployment introduced in Chapter 11 declares a fixed replicas count. That works fine for predictable, steady-state traffic, but real applications see traffic that rises and falls throughout the day. The **Horizontal Pod Autoscaler (HPA)** watches a metric — most commonly CPU utilization — across the Pods of a Deployment, and adjusts the replicas field up or down automatically to keep that metric near a target value.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

scaleTargetRef points the HPA at the app-deployment Deployment from Chapter 11 — the HPA does not create or own Pods itself; it only edits that Deployment's replicas field. minReplicas and maxReplicas set the floor and ceiling the HPA is allowed to scale within, and the single metric here tells it to aim for an average CPU utilization of 60% across all Pods, calculated relative to each container's CPU requests value. If average utilization climbs above 60%, the HPA increases replicas (up to the maximum of 10); if it drops well below 60%, it scales back down (never below the minimum of 2).

Apply it exactly like any other manifest:

```
kubectl apply -f app-hpa.yaml
kubectl get hpa app-hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
app-hpa	Deployment/app-deployment	45%/60%	2	10	3	10m

The TARGETS column shows current utilization against the target, and REPLICAS shows how many Pods the HPA has settled on right now. Because the HPA works directly against the Deployment, it composes cleanly with a rolling update (Chapter 11): a new rollout replaces Pods gradually while the HPA keeps enforcing its replica bounds and target metric throughout.

NOTE

For CPU-based scaling to work at all, every container in the target Deployment must have a CPU request set — without it, the HPA has no baseline to calculate utilization percentages against, and it will report an unknown target.

The HPA is not limited to CPU. It can also scale on memory utilization, or on custom and external metrics (requests per second, queue depth, and so on) via the Kubernetes metrics APIs, but CPU-based scaling remains the simplest and most common starting point.

Vertical Pod Autoscaler (VPA)

Where HPA changes *how many* Pods run, the **Vertical Pod Autoscaler (VPA)** changes *how much* CPU and memory each individual Pod requests. VPA observes actual resource usage over time and recommends — or, in a more automated mode, directly applies — updated requests (and optionally limits) values for the containers in a workload.

VPA is most useful for workloads that cannot easily scale horizontally, such as a single-replica stateful service, or for right-sizing requests on any workload where the initial guesses in the Deployment manifest turn out to be too high (wasting money) or too low (causing throttling or evictions). It is generally not recommended to run HPA on CPU/memory and VPA on the same resource for the same workload simultaneously, since the two can fight each other — VPA changes the request VPA uses to size Pods while HPA is trying to react to utilization measured against that same request. A common pattern is VPA in “recommendation-only” mode to inform sizing decisions, combined with HPA doing the actual replica scaling.

TIP

Start with VPA in recommendation mode on a new workload to discover realistic CPU and memory requests, then set those values explicitly in the Deployment and let HPA handle demand-driven replica scaling from there.

Cluster Autoscaler: Standard Mode

Scaling Pod replicas only helps if there is enough node capacity to actually schedule the new Pods. In **GKE Standard mode**, where you manage node pools directly, the **Cluster Autoscaler** watches for Pods that cannot be scheduled due to insufficient resources and automatically adds nodes to the relevant node pool to fit them — and just as importantly, removes nodes when they sit underutilized for a sustained period, as long as doing so would not evict Pods that have nowhere else to go.

```
gcloud container node-pools update default-pool \
  --cluster=my-gke-cluster \
  --zone=us-central1-a \
  --enable-autoscaling \
  --min-nodes=1 \
  --max-nodes=5
```

This enables autoscaling on `default-pool`, letting it range between 1 and 5 nodes. The Cluster Autoscaler makes its decisions purely based on pending Pod demand and node utilization — it is a separate mechanism from the HPA, and the two typically work together: the HPA creates more Pods in response to load, those Pods go Pending if no node has room, and the Cluster Autoscaler then provisions nodes to fit them, usually within a couple of minutes.

WARNING

Scale-down is intentionally conservative — the Cluster Autoscaler will not remove a node if doing so would violate a Pod’s `PodDisruptionBudget`, if the node runs Pods without a controller (bare Pods), or if it hosts Pods that request local storage that cannot be evicted safely. Review these

constraints if nodes are not scaling down as expected.

Autopilot's Fully Automatic Node Management

GKE Autopilot takes a fundamentally different approach: there is no Cluster Autoscaler to configure at all, because there are no node pools for you to manage in the first place. Google Cloud provisions and scales the underlying compute automatically based on the aggregate resource requests of the Pods you schedule — you specify what your Pods need in CPU, memory, and (optionally) GPU or ephemeral storage, and Autopilot handles provisioning right-sized nodes behind the scenes, scaling them up and down as workloads change, with no `--enable-autoscaling`, `--min-nodes`, or `--max-nodes` flags anywhere in the picture.

This trade-off removes an entire category of operational tuning (node pool sizing, machine type selection, bin-packing efficiency) at the cost of some flexibility — Autopilot enforces certain Pod configuration constraints and, at the time of writing, prices workloads based on requested Pod resources rather than whole-node consumption. For teams that want Kubernetes' scheduling and deployment model without owning node-level infrastructure decisions, Autopilot is usually the right default; for teams that need fine control over machine types, specific node-level configuration, or DaemonSets with elevated node access, Standard mode with the Cluster Autoscaler remains the better fit.

Node Auto-Provisioning (Standard Mode)

Even with the Cluster Autoscaler enabled, it can only add nodes to node pools that already exist — if a Pod requests a resource shape no existing node pool can satisfy (for example, a GPU type, a much larger machine size, or an ARM-based node), the Cluster Autoscaler alone cannot help. **Node auto-provisioning (NAP)** extends Standard mode's autoscaling one step further: it can create and delete entire node pools automatically, within limits and machine families you configure, to fit Pods that current node pools cannot accommodate.

```
gcloud container clusters update my-gke-cluster \
  --zone=us-central1-a \
  --enable-autoprovisioning \
  --min-cpu=1 --max-cpu=32 \
  --min-memory=1 --max-memory=128
```

This enables NAP on the cluster with overall CPU and memory bounds across all auto-provisioned node pools combined, so a Pod requesting a node shape outside the existing pools can still be scheduled without an operator having to pre-create a matching pool by hand. NAP is a Standard-mode feature; Autopilot achieves the same outcome by default, since it always provisions whatever shape a Pod needs.

Key Takeaways

- The Horizontal Pod Autoscaler adjusts a Deployment's replica count based on a target metric (commonly CPU utilization) and requires CPU requests to be set on every container it scales.
- The Vertical Pod Autoscaler adjusts per-Pod resource requests rather than replica count, and is best used in recommendation mode alongside HPA rather than fighting it on the same resource.
- In GKE Standard mode, the Cluster Autoscaler adds and removes nodes within a node pool's configured min/max bounds based on pending Pod demand.
- GKE Autopilot removes cluster autoscaling configuration entirely — Google Cloud provisions and scales nodes automatically based on Pod resource requests.
- Node auto-provisioning extends Standard mode to create entirely new node pools automatically when workloads need a node shape the existing pools cannot provide.

Chapter 19: CI/CD: Docker to Artifact Registry to GKE

Every chapter so far has shown you individual pieces: building a Docker image, pushing it to Artifact Registry, scanning it for vulnerabilities, enforcing Binary Authorization, and deploying it to GKE. In the real world, nobody runs these steps by hand every time a developer changes a line of code. Instead, they are wired together into a single automated pipeline that runs on every code change. This chapter connects every one of those pieces into one continuous, end-to-end flow, and gives you a complete, working GitHub Actions example you can adapt directly.

The Big Picture: One Push, One Pipeline

The pattern is the same regardless of which CI/CD tool you use — GitHub Actions, GitLab CI, Cloud Build, or Jenkins. Only the syntax changes; the stages do not.

1. **Trigger** — a developer pushes code to a branch (often `main`, or a tag like `v1.2.0`). This push event triggers the CI/CD pipeline automatically; no human clicks “deploy.”
2. **Build and test** — the pipeline checks out the code, runs unit tests, and builds a Docker image using the `Dockerfile` from Chapters 3 through 8 (ideally a multi-stage build for a small, production-ready image).
3. **Push to Artifact Registry** — the pipeline authenticates to Google Cloud and pushes the freshly built image to Artifact Registry, tagged with something traceable, such as the Git commit SHA or a semantic version.
4. **Vulnerability scan gate** — Artifact Registry’s automatic vulnerability scanning (Chapter 9) inspects the new image. If Binary Authorization (Chapter 17) is enforced, an attestor must sign off before the image is allowed to run on GKE at all. This is the pipeline’s go/no-go checkpoint: an image with critical, unpatched vulnerabilities, or one lacking a required attestation, is blocked from deployment.
5. **Deploy to GKE** — once the image passes its gate, the CD half of the pipeline updates the GKE Deployment to reference the new image tag, either with `kubectl apply` against an updated manifest, `kubectl set image` for a quick update, or a `helm upgrade` if you are managing releases with Helm.
6. **Rolling update** — Kubernetes takes it from there. The Deployment’s rolling update strategy (Chapter 13) replaces old Pods with new ones gradually, checking readiness probes along the way, so users experience no downtime and a bad rollout can be caught and rolled back before it reaches everyone.

NOTE

CI (Continuous Integration) and CD (Continuous Delivery/Deployment) are conceptually separate, but in practice most teams run them as one connected pipeline: CI produces a trustworthy artifact, and CD ships that exact artifact to production.

Why the Vulnerability Scan Is a Gate, Not a Report

It is tempting to treat vulnerability scanning as something you glance at occasionally in the console. In a mature pipeline, it is instead an automated gate: a step that can fail the pipeline outright. This is the direct link back to Chapter 9 (vulnerability scanning) and Chapter 17 (Binary Authorization). Binary Authorization enforces this at the cluster level — even if someone bypasses the pipeline and tries to deploy an unscanned or unsigned image directly with `kubectl`, the GKE admission controller rejects it. The pipeline gate and the cluster-level policy work together: the pipeline stops bad images early, and Binary Authorization guarantees that only images which passed that gate can ever run, no matter how the deployment was attempted.

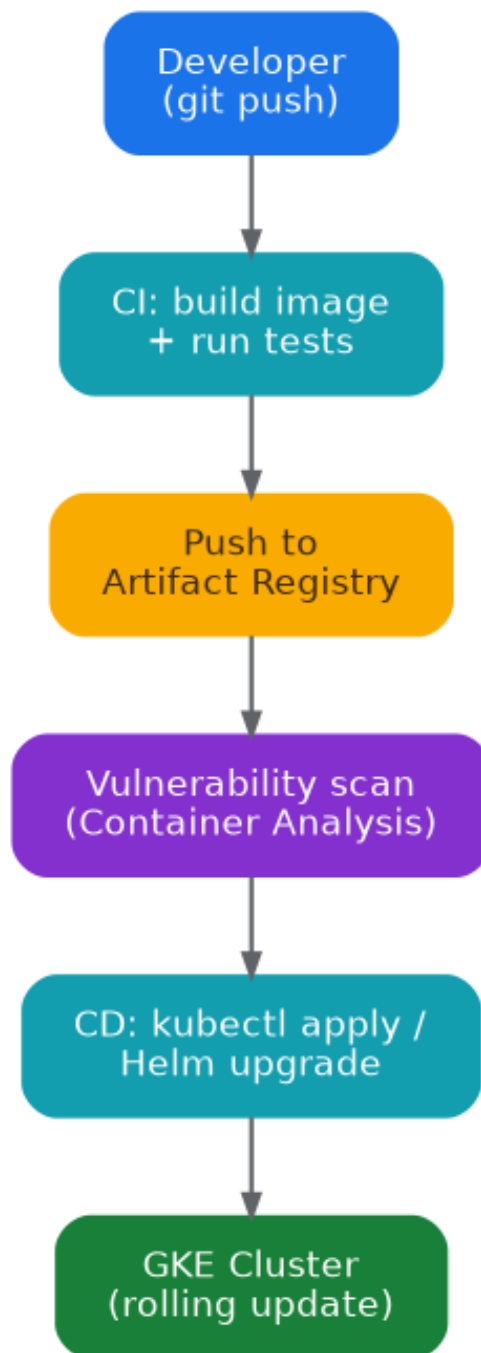


Figure 21: An end-to-end CI/CD pipeline: build and test, push to Artifact Registry, scan for vulnerabilities, then deploy to GKE.

BEST PRACTICE

Fail the pipeline on critical and high-severity vulnerabilities by default, and require an explicit, documented exception (not a code comment) to override it. Silent overrides defeat the entire purpose of the gate.

Authenticating CI/CD to Google Cloud: Workload Identity Federation

Older pipelines authenticated to Google Cloud using a downloaded JSON service account key stored as a CI secret. This works, but it is a standing security risk: a leaked key is valid indefinitely until someone manually revokes it. The modern, recommended approach is **Workload Identity Federation (WIF)**, which lets GitHub Actions (or another external CI system) exchange a short-lived, cryptographically signed GitHub token for a short-lived Google Cloud access token — with no long-lived key ever stored anywhere.

TIP

Workload Identity Federation for GitHub Actions requires a one-time setup in Google Cloud: creating a Workload Identity Pool, an OIDC provider trusting to-ken.actions.githubusercontent.com, and a service account that the pool is allowed to impersonate, scoped to your repository.

A Complete GitHub Actions Workflow

The following workflow builds a Docker image, pushes it to Artifact Registry, and deploys it to GKE, using Workload Identity Federation for authentication.

name: Build, Scan, and Deploy to GKE

on:

push:

branches: [main]

env:

PROJECT_ID: my-gcp-project-123456

REGION: us-central1

REPOSITORY: my-repo

IMAGE_NAME: app

GKE_CLUSTER: my-gke-cluster

GKE_ZONE: us-central1-a

permissions:

contents: read

id-token: write *# required for Workload Identity Federation*

jobs:

build-and-deploy:

runs-on: ubuntu-latest

steps:

- **name:** Checkout code

uses: actions/checkout@v4

- **name:** Authenticate to Google Cloud

uses: google-github-actions/auth@v2

with:

workload_identity_provider:

↪ projects/123456789/locations/global/workloadIdentityPools/github-

↪ pool/providers/github-provider

```

    service_account: ci-deployer@my-gcp-project-123456.iam.gserviceaccount.com

- name: Set up Cloud SDK
  uses: google-github-actions/setup-gcloud@v2

- name: Configure Docker for Artifact Registry
  run: gcloud auth configure-docker us-central1-docker.pkg.dev --quiet

- name: Build Docker image
  run: |
    docker build \
      -t us-central1-docker.pkg.dev/${{ env.PROJECT_ID }}/${{ env.REPOSITORY
↪ }}/${{ env.IMAGE_NAME }}:${{ github.sha }} .

- name: Push image to Artifact Registry
  run: |
    docker push us-central1-docker.pkg.dev/${{ env.PROJECT_ID }}/${{
↪ env.REPOSITORY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}

- name: Get GKE credentials
  uses: google-github-actions/get-gke-credentials@v2
  with:
    cluster_name: ${ env.GKE_CLUSTER }
    location: ${ env.GKE_ZONE }

- name: Deploy new image to GKE
  run: |
    kubectl set image deployment/app-deployment \
      app=us-central1-docker.pkg.dev/${{ env.PROJECT_ID }}/${{ env.REPOSITORY
↪ }}/${{ env.IMAGE_NAME }}:${{ github.sha }} \
      --namespace=default

- name: Verify rollout
  run: |
    kubectl rollout status deployment/app-deployment --namespace=default
↪ --timeout=180s

```

Walking through what this workflow does: the permissions: id-token: write line and the google-github-actions/auth step are what make Workload Identity Federation work — GitHub issues a short-lived OIDC token for this specific run, and Google Cloud exchanges it for temporary credentials scoped to the ci-deployer service account, with nothing long-lived stored as a secret. The build and push steps are ordinary Docker commands, tagging the image with the Git commit SHA (github.sha) so every deployed image is traceable back to the exact commit that produced it. get-gke-credentials fetches a kubeconfig for my-gke-cluster so that kubectl commands in later steps target the right cluster. kubectl set image is a quick, imperative way to update just the container image referenced by an existing Deployment, triggering Kubernetes' normal rolling update process, and kubectl rollout status blocks the pipeline until the rollout finishes or fails, so a broken deployment shows up as a failed CI run rather than a silent problem in production.

BEST PRACTICE

For anything beyond a simple demo, prefer applying a full, version-controlled YAML manifest with kubectl apply -f deployment.yaml (with the image tag templated in) over kubectl set image, since kubectl apply keeps your Git repository as the single source of truth for the entire Deployment spec, not just the image tag.

If your team uses Helm to package Kubernetes manifests (common for applications with many related resources), the deploy step becomes a `helm upgrade --install app ./chart --set image.tag=${{ github.sha }}` command instead — same idea, different tool.

Key Takeaways

- A production CI/CD pipeline connects five stages automatically: build and test, push to Artifact Registry, a vulnerability scan gate, deploy to GKE, and a rolling update — all triggered by a single `git push`.
- The vulnerability scan and Binary Authorization act as a go/no-go gate, not just a report; a failing image should stop the pipeline, and the cluster should reject it even if someone bypasses the pipeline entirely.
- Workload Identity Federation lets CI/CD systems like GitHub Actions authenticate to Google Cloud using short-lived, exchanged tokens, removing the need to store long-lived service account keys as secrets.
- `kubectl set image` and `kubectl apply` both trigger Kubernetes' normal rolling update mechanism; `kubectl apply` against a version-controlled manifest is the more maintainable long-term choice.
- Tagging images with the Git commit SHA keeps every deployed artifact traceable back to the exact source code that produced it.

Chapter 20: Monitoring, Logging & Observability

A deployment pipeline that ships code is only half the job. Once your application is running on GKE, you need to know whether it is actually healthy: is it serving traffic correctly, is it using too much memory, is it crashing and restarting silently in the background? This chapter covers the tools GKE gives you out of the box to answer those questions, and the single most important mechanism for automatic self-healing: probes.

Cloud Monitoring and Cloud Logging Are On by Default

One of the practical advantages of using a managed platform like GKE, rather than running Kubernetes yourself on raw virtual machines, is that observability is largely switched on for you already. When you create a GKE cluster with the default settings, two things happen automatically, with no extra configuration required:

- **Cloud Logging** collects standard output and standard error from every container running in the cluster, along with system-level logs from the nodes and the Kubernetes control plane itself. Every `print()`, every `console.log`, every stack trace your application writes to `stdout` ends up searchable in Cloud Logging within seconds.
- **Cloud Monitoring** collects standard infrastructure and Kubernetes metrics automatically: CPU and memory usage per container, per Pod, and per node; network traffic; disk usage; Pod restart counts; and cluster-level capacity metrics.

NOTE

This automatic collection is often called “GKE system logging and monitoring.” It is enabled by default when you create a cluster through the Cloud Console or with default `gcloud container clusters create` flags, and it is what powers the GKE Monitoring dashboard described later in this chapter.

This matters because it removes an entire category of setup work that engineers running self-managed Kubernetes normally have to do by hand — deploying and maintaining their own logging agents and metrics pipelines. On GKE, you get a working baseline immediately, and you can layer custom application metrics and log-based alerts on top of it as your needs grow.

You can view logs for a specific workload directly from the command line as well:

```
kubectl logs deployment/app-deployment --namespace=default --tail=100
```

This explanation matters even for a one-line command: `--tail=100` limits output to the most recent 100 lines so you are not flooded with the entire log history, and targeting `deployment/app-deployment` rather than a specific Pod name means `kubectl` automatically picks one of the Deployment’s current Pods for you.

Probes: Teaching Kubernetes What “Healthy” Means

Cloud Monitoring tells you what happened. Probes are what let Kubernetes act on health information automatically, in real time, without a human watching a dashboard. A **probe** is a periodic health check that the kubelet runs against a container, and Kubernetes ships with two kinds that matter for almost every application: liveness probes and readiness probes.

- A **liveness probe** answers the question “is this container still working, or is it stuck and needs to be restarted?” If a liveness probe fails repeatedly, Kubernetes kills the container and starts a fresh one in its place — this is the mechanism behind Kubernetes’ self-healing behavior.
- A **readiness probe** answers a different question: “is this container ready to receive traffic right now?” If a readiness probe fails, Kubernetes does not restart the container — it simply stops sending it traffic through the Service, until the probe starts succeeding again. This is essential during startup (an app that takes 30 seconds to warm up should not receive requests during those 30 seconds) and during rolling updates (a new Pod should not receive traffic until it has

proven it is actually ready).

TIP

Think of it this way: a failed readiness probe means “wait, don’t send this Pod traffic yet.” A failed liveness probe means “this Pod is broken, kill it and start over.”

A Complete Probe Example

Here is a container spec with both probes configured, as part of a Deployment’s Pod template:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /healthz/ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /healthz/live
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
            failureThreshold: 3
```

Breaking this down: `readinessProbe` and `livenessProbe` each send an HTTP GET request to a specific path and port inside the container, and expect a 2xx or 3xx response to count as success. `initialDelaySeconds` gives the container a grace period after startup before the first check runs, which prevents a slow-starting application from being killed before it has even finished booting. `periodSeconds` controls how often the check repeats, and `failureThreshold` sets how many consecutive failures are tolerated before Kubernetes takes action — restarting the container for a failed liveness probe, or removing the Pod from Service endpoints for a failed readiness probe.

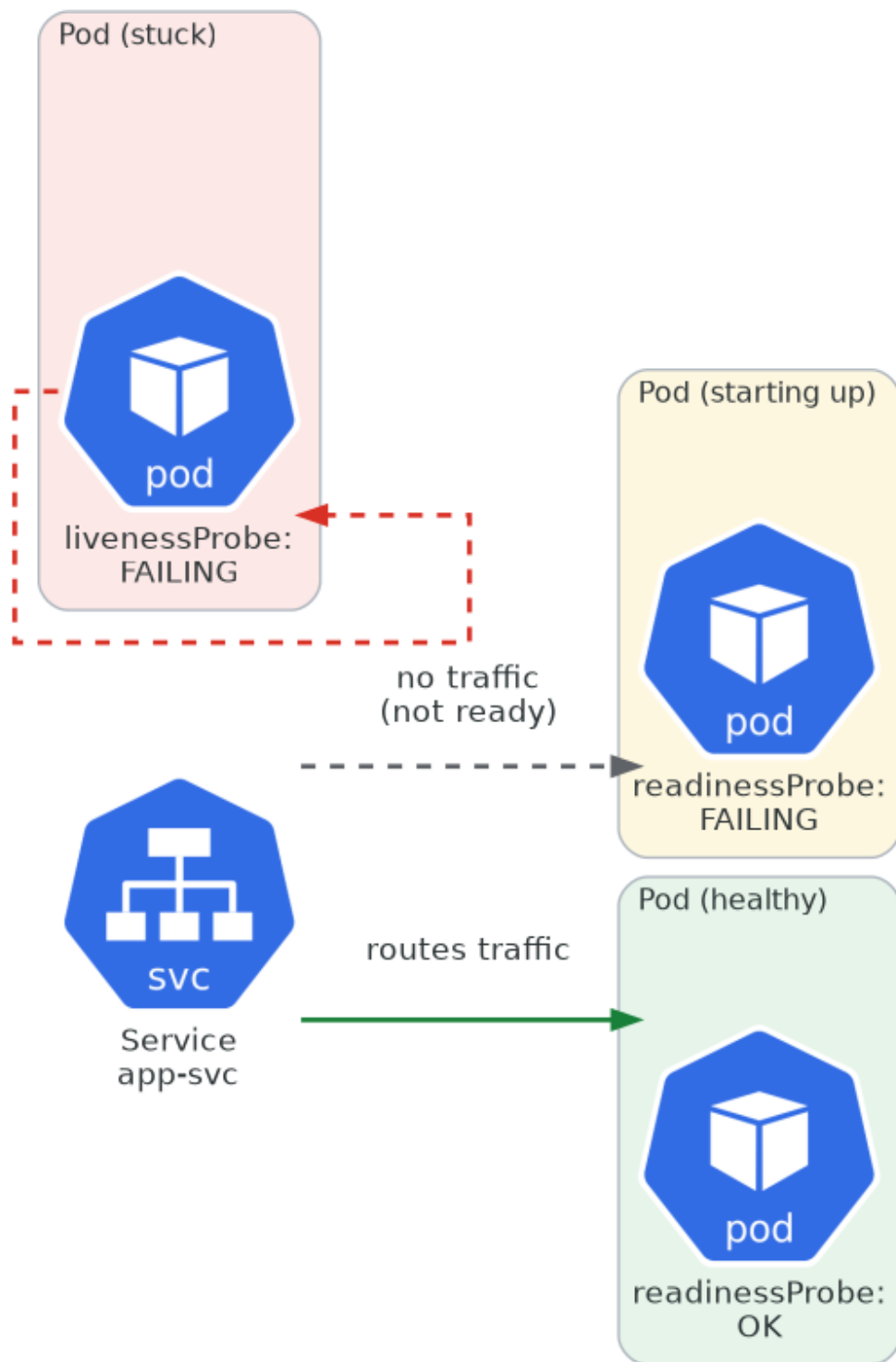


Figure 22: Readiness probes control whether a Pod receives traffic; liveness probes control whether a stuck container gets restarted.

WARNING

Never point a liveness probe at an endpoint that depends on external services like a database. If the database has a brief outage, every Pod's liveness probe would fail at once, causing Kubernetes to restart every Pod simultaneously — turning a downstream outage into a self-inflicted total outage of your own application.

These two probes are also what make rolling updates in Chapter 13 actually safe. During a rolling update, Kubernetes only considers a new Pod “up” and moves on to replacing the next old Pod once the new Pod's readiness probe succeeds. Without a readiness probe, Kubernetes would consider a Pod ready the instant its container process starts, even if the application inside is still initializing — which can cause real users to hit connection errors during a routine deployment.

Checking Resource Usage Day to Day

For quick, ad hoc checks of how much CPU and memory your workloads are actually using right now, `kubectl top` is the fastest tool available:

```
kubectl top pods --namespace=default
kubectl top nodes
```

NAME	CPU(cores)	MEMORY(bytes)
app-deployment-7d8f9c6b5d-2xk9p	45m	128Mi
app-deployment-7d8f9c6b5d-9mqwl	52m	131Mi
app-deployment-7d8f9c6b5d-fj3rt	48m	126Mi

`kubectl top pods` shows live CPU (in millicores, where 1000m equals one full CPU core) and memory usage per Pod, which is invaluable for spotting a Pod that is silently consuming far more memory than expected — often a sign of a memory leak, and useful evidence when deciding whether to adjust the resource requests and limits covered in Chapter 14. `kubectl top nodes` shows the same information aggregated per node, which helps answer “do I have room to schedule more workloads, or is this node close to full?”

BEST PRACTICE

Use `kubectl top` for quick, in-the-moment checks during active troubleshooting. For historical trends, alerting, and dashboards you can share with a team, use the Cloud Monitoring console instead — `kubectl top` does not retain any history.

The Built-In GKE Monitoring Dashboard

Beyond the command line, the Google Cloud Console provides a dedicated GKE Monitoring dashboard (under Kubernetes Engine, then Observability, in the Cloud Console) that visualizes cluster health without any setup. It shows cluster-wide and per-workload views of CPU and memory usage over time, Pod restart counts, error rates, and infrastructure health, and it lets you drill down from the cluster level, to a specific workload, down to an individual Pod's logs, all in one place. For most day-to-day operational questions — “is anything unhealthy right now, and has it been unhealthy for a while?” — this dashboard is usually the fastest place to look before reaching for `kubectl` at all.

Key Takeaways

- Cloud Logging and Cloud Monitoring are enabled by default on GKE clusters, automatically collecting container logs and standard infrastructure and Kubernetes metrics with no extra setup.
- Readiness probes control whether a Pod receives traffic; liveness probes control whether Kubernetes restarts a Pod — confusing the two, or pointing either at an external dependency, is a common and costly mistake.
- Probes are what make rolling updates safe: Kubernetes waits for a new Pod's readiness probe to succeed before considering the update complete for that Pod.

- `kubectl top pods` and `kubectl top nodes` give fast, live resource usage snapshots for troubleshooting, but they hold no history — use the GKE Monitoring dashboard for trends and historical context.
- The built-in GKE Monitoring dashboard in the Cloud Console is usually the fastest starting point for checking overall cluster and workload health.

Chapter 21: Troubleshooting Guide

Every engineer who works with GKE runs into the same handful of problems, usually in the first few weeks. The good news is that Kubernetes almost always tells you exactly what is wrong — you just need to know where to look. This chapter is a reference guide to the five most common failure modes, each with the exact error text, the one-sentence root cause, and a concrete fix, followed by a short toolkit of the commands you will reach for constantly.

ImagePullBackOff / ErrImagePull

Failed to pull image

```
↪ "us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1":  
rpc error: code = NotFound desc = failed to pull and unpack image
```

Root cause: the node (or the Workload Identity service account, if Workload Identity is enabled) does not have permission to read from Artifact Registry, or the image path or tag in your manifest simply does not exist.

Fix: first confirm the image path and tag are exactly correct — a typo in the project ID, repository name, or tag is the single most common cause. Then confirm the identity pulling the image has the `roles/artifactregistry.reader` role:

```
gcloud projects add-iam-policy-binding my-gcp-project-123456 \  
--member="serviceAccount:gke-node-sa@my-gcp-project-  
↪ 123456.iam.gserviceaccount.com"  
↪ \  
--role="roles/artifactregistry.reader"
```

NOTE

If your cluster uses Workload Identity, the relevant service account is the one bound to the Kubernetes service account your Pod uses, not necessarily the node's default service account — check your Pod spec's `serviceAccountName` field first.

CrashLoopBackOff

Back-off restarting failed container app in pod app-deployment-7d8f9c6b5d-2xk9p

Root cause: the container's process is starting and then exiting almost immediately, over and over — this is an application problem, not a scheduling problem. Common causes include a missing environment variable, a bad `CMD/ENTRYPOINT`, or the app crashing on startup because a dependency (like a database) is not reachable yet.

Fix: look at the logs from the previous, crashed attempt, since the current container is usually already gone or restarting again:

```
kubectl logs app-deployment-7d8f9c6b5d-2xk9p --previous
```

Also double-check the Dockerfile's `CMD` and `ENTRYPOINT` actually start a long-running process rather than a one-shot command that naturally exits, and confirm any required environment variables or mounted secrets are present in the Pod spec.

TIP

`--previous` is the single most useful flag you will use for this error — without it, you are often looking at the logs of a container that only just started and has not crashed yet.

Pending Pods That Never Get Scheduled

0/3 nodes are available: 3 Insufficient cpu.

Root cause: no node in the cluster currently has enough free CPU or memory to satisfy the Pod's resource requests, and — if cluster autoscaling is not enabled or has hit its maximum — no new node can be added to make room.

Fix: check exactly why scheduling failed with the Pod's events, and compare against actual node capacity:

```
kubectl describe pod app-deployment-7d8f9c6b5d-2xk9p --namespace=default
kubectl get nodes -o=custom-
  ↪ columns=NAME:.metadata.name,CPU:.status.allocatable.cpu,MEM:.status.allocatable.memory
```

If requests are simply too large for any node type in the cluster, lower them or use a larger machine type. If the cluster uses node auto-provisioning or a Cluster Autoscaler, confirm the node pool's maximum node count has not already been reached, since an autoscaler that is maxed out cannot add capacity no matter how long the Pod waits.

BEST PRACTICE

Set resource requests based on measured usage (see `kubectl top pods` in Chapter 20), not guesswork — requests set far too high are one of the most common causes of Pods stuck in Pending on an otherwise healthy cluster.

A Service That Is Unreachable

curl: (7) Failed to connect to my-service port 80: Connection refused

Root cause: almost always a mismatch between the Service's selector and the actual labels on the target Pods — usually a small typo, like `app: my-app` on the Deployment's Pods but `app: myapp` (missing the hyphen or space) on the Service.

Fix: compare the two directly:

```
kubectl get service my-service -o=jsonpath='{.spec.selector}'
kubectl get pods --show-labels
```

If the printed selector does not exactly match the labels shown on the running Pods, correct whichever one is wrong — usually the Service's selector field — and reapply it:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app    # must exactly match the Pod template's labels
  ports:
    - port: 80
      targetPort: 8080
```

A quick way to confirm the Service has found any endpoints at all is `kubectl get endpoints my-service`; an empty list confirms the selector is not matching anything.

An Ingress That Never Provisions an External IP

```
Warning Sync Ingress Error syncing to GCLB: could not find health check port for
↪ backend
```

Root cause: the Ingress's underlying health check cannot reach the backend, usually because the health check path does not match a route your application actually serves (returning a 404 instead of a 200), or because the Service or NodePort it points to is not correctly wired up to running, ready Pods.

Fix: check the Ingress's events for the specific complaint, then confirm the backend Service has healthy endpoints:

```
kubectl describe ingress my-ingress --namespace=default
kubectl get endpoints my-service --namespace=default
```

If the health check path is the issue, add a BackendConfig that points the health check at a path your app actually responds to with a 200:

```
apiVersion: cloud.google.com/v1
kind: BackendConfig
metadata:
  name: my-backendconfig
spec:
  healthCheck:
    requestPath: /healthz/ready
    port: 8080
```

Then reference it from the Service with the `cloud.google.com/backend-config` annotation. Provisioning an external IP and a working health check together can take several minutes even after the fix, so allow time before assuming it is still broken.

WARNING

An Ingress with no ready Pods behind its Service will never show a healthy backend, no matter how the health check path is configured — always confirm the Pods themselves are Running and Ready first before troubleshooting the Ingress itself.

Debugging Toolkit

Three commands cover the majority of real-world troubleshooting sessions, and are worth knowing well enough to type from memory:

- **kubectl describe <resource> <name>** — shows full details and, critically, a chronological list of events at the bottom of the output. For a Pod stuck Pending, a Service with no endpoints, or an Ingress that will not provision, the events section almost always names the exact problem in plain English.
- **kubectl logs --previous** — retrieves logs from a container's last run before it crashed and restarted, which is essential for diagnosing CrashLoopBackOff, since the currently running container may not have failed yet or may have no logs at all.
- **kubectl get events --sort-by=.lastTimestamp** — lists every recent event across an entire namespace, sorted chronologically, which is useful when you are not even sure which resource is the problem yet and need a timeline of everything that has gone wrong recently.

```
kubectl get events --namespace=default --sort-by=.lastTimestamp
```

This command explains itself well in practice: it is often the very first thing to run when something is broken and you do not yet know where to look, since it surfaces scheduling failures, failed probes, and image pull errors together in one time-ordered list.

Key Takeaways

- `ImagePullBackOff` is almost always an IAM permissions problem (`roles/artifactregistry.reader`) or a typo in the image path or tag.
- `CrashLoopBackOff` means the application itself is failing on startup — `kubectl logs --previous` and a careful look at the Dockerfile's `CMD/ENTRYPOINT` are the fastest paths to a root cause.
- Pods stuck `Pending` point to a resource capacity problem — compare requests against `kubectl get nodes capacity` and confirm the autoscaler has room to grow.
- An unreachable Service is most often a label selector mismatch between the Service and the Pods it is supposed to route to.
- `kubectl describe`, `kubectl logs --previous`, and `kubectl get events --sort-by=.lastTimestamp` together resolve the large majority of everyday GKE issues.

Chapter 22: Production Best Practices & Enterprise Architecture

Every previous chapter in this book has taken one concept at a time: a Dockerfile, a Deployment, a Service, an Artifact Registry repository, an HPA. Production systems do not work that way. A real, enterprise-grade deployment is all of these pieces operating together, continuously, under load, with real users depending on it and real engineers on call for it. This capstone chapter pulls every thread from the book into a single coherent architecture, and then walks through the decisions a senior cloud architect actually has to make before signing off on a production rollout: how to isolate workloads, how to size resources correctly, how to manage multiple environments without duplicating effort, and how to think about cost, availability, security, and disaster recovery as a single connected set of trade-offs rather than four separate checklists.

Nothing here is a new API or a new `kubectl` verb. It is judgment — the layer that turns a working cluster into a production-grade one.

Architecture Overview

Picture a single, unified request path, end to end, for a real application running on GKE.

A user opens a browser and requests `https://app.example.com`. That request first reaches a **Global HTTP(S) Load Balancer**, a GCP-managed, anycast load balancer that terminates TLS at the edge, closest to the user, and is backed by Google’s global network rather than a single region. The load balancer’s backend is not a VM or a single Service — it is configured, via a **GKE Ingress** resource, to route traffic into the `my-gke-cluster` GKE cluster. GKE’s Ingress controller programs the load balancer automatically whenever the Ingress object changes, translating Kubernetes-native configuration (hostnames, paths, TLS certificates) into GCP load-balancing infrastructure.

Inside the cluster, the Ingress forwards matching traffic to a Kubernetes **Service**, a stable virtual IP and DNS name that decouples callers from the churn of individual Pods coming and going. The Service in turn load-balances across a set of **Pods**, all running in a production namespace, all running the same container image. Every Pod’s containers are pulled from a **scanned, vulnerability-gated Artifact Registry repository** (`my-repo`) — meaning no image reaches a running Pod in this cluster unless it has already passed a vulnerability scan and, ideally, a Binary Authorization attestation confirming it came from a trusted build pipeline. As load increases, a **Horizontal Pod Autoscaler (HPA)** attached to the Deployment behind that Service watches average CPU utilization across the Pods and adds or removes replicas automatically, so the same manifests that serve ten users at 2 a.m. also serve ten thousand users during a traffic spike, without manual intervention.

Wrapped around all of it, **Cloud Monitoring** and **Cloud Logging** observe the entire stack — load balancer request latency and error rates, node and Pod resource usage, container logs, HPA scaling events, and Kubernetes audit logs — giving operators one place to ask “what is happening right now” and “what happened during the incident three hours ago.”

Every piece of this diagram corresponds to a chapter you have already read: Dockerfiles and image builds, Artifact Registry and scanning, Deployments and Services, Ingress and load balancing, HPA and Cluster Autoscaler, RBAC and namespaces, Monitoring and Logging. What makes it a *production architecture* rather than a collection of demos is that every piece is deliberately sized, isolated, secured, and observed — which is exactly what the rest of this chapter addresses.

Namespaces for Isolation

A single GKE cluster can safely host many workloads at once, provided those workloads are properly isolated from one another. Kubernetes **namespaces** are the primary mechanism for that isolation within one cluster: a namespace scopes RBAC role bindings, resource quotas, network policies, and even naming (Service and Deployment names only need to be unique within a namespace, not cluster-wide).

A typical structure splits workloads by environment or by team:

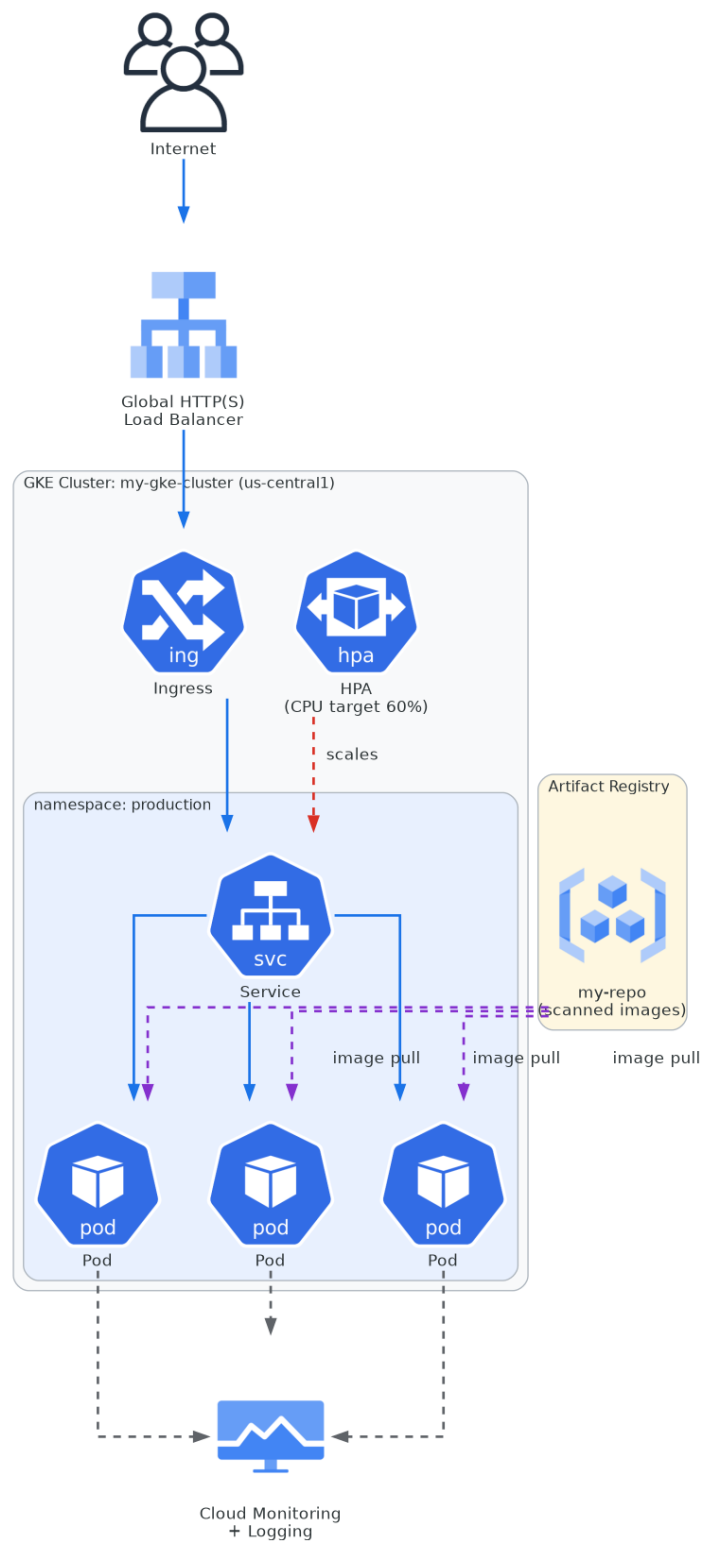


Figure 23: A production-grade GKE architecture: a global load balancer, autoscaling workloads, image provenance from Artifact Registry, and full observability.

```
kubectl create namespace production
kubectl create namespace staging
kubectl create namespace dev
```

Or, in an organization with multiple product teams sharing one cluster, by team instead of (or in addition to) environment:

```
kubectl create namespace team-payments
kubectl create namespace team-checkout
```

Namespaces let you attach a **ResourceQuota** to cap total CPU, memory, and object counts a namespace can consume, and RBAC **RoleBindings** to grant a team's engineers access to their own namespace without touching anyone else's:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: production-quota
  namespace: production
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 40Gi
    limits.cpu: "40"
    limits.memory: 80Gi
    pods: "50"
```

BEST PRACTICE

Pair every namespace intended for a specific team or environment with both a ResourceQuota and a RoleBindings — a namespace with no quota offers organizational clarity but no actual protection against one workload starving another for cluster resources.

The alternative — a fully separate GKE cluster per environment or per team — buys stronger isolation at the cost of duplicated infrastructure: separate node pools, separate Ingress and load-balancer configuration, separate monitoring dashboards, and separate upgrade cycles to manage. Namespaces within one cluster make sense when workloads share a similar risk profile, trust each other reasonably well, and benefit from shared node capacity (better bin-packing, lower idle cost). Separate clusters make sense when isolation requirements are strict — regulatory separation, a noisy or untrusted tenant, or a production environment that must never share a control plane, node pool, or blast radius with anything less trusted. Most enterprises land on a hybrid of the two, covered in the Multi-Environment Strategy section below.

Resource Requests, Limits & Quality of Service

Every container in a production Pod should declare both requests and limits:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: app
```

```

template:
  metadata:
    labels:
      app: app
  spec:
    containers:
      - name: app
        image: us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:v1
        resources:
          requests:
            cpu: "250m"
            memory: "256Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"

```

requests is what the scheduler guarantees a container will get and uses to decide which node has room for a Pod; limits is the ceiling the container is not allowed to exceed. A container that exceeds its memory limit is OOM-killed; a container that exceeds its CPU limit is throttled, not killed.

These two fields also determine a Pod's **Quality of Service (QoS) class**, which governs eviction order when a node runs low on resources:

- **Guaranteed** — every container's requests equal its limits, for both CPU and memory. Evicted last.
- **Burstable** — at least one container has requests set, but they differ from limits. Evicted before Guaranteed, after BestEffort.
- **BestEffort** — no requests or limits set at all. Evicted first under resource pressure.

WARNING

Setting requests far below what a container actually uses does not save resources — it only postpones the problem. The scheduler will over-pack the node based on the understated request, and under real load the node runs out of headroom, triggering throttling, evictions, or an unplanned Cluster Autoscaler scale-up under pressure rather than in a calm, predictable way.

Accurate requests matter for two reasons beyond QoS: the scheduler uses requests (not actual usage) to decide whether a node has room for a new Pod, and the **Cluster Autoscaler** uses the same requested values to decide whether existing nodes are full enough to justify adding a new node, or empty enough to justify removing one. Chronically wrong requests — too high, and you pay for idle capacity and trigger unnecessary scale-ups; too low, and you risk noisy-neighbor resource contention and mistimed autoscaling.

Multi-Environment Strategy

Most organizations run at least three environments — development, staging, and production — and need a strategy for how those environments relate to GKE clusters and namespaces. Three patterns cover the vast majority of real deployments:

- **Separate cluster per environment.** dev, staging, and production each get their own GKE cluster, with their own node pools, IP ranges, and upgrade schedule. Maximum isolation, but maximum operational overhead — three (or more) sets of infrastructure to patch, monitor, and pay for.
- **Namespaces within one cluster.** A single cluster hosts dev, staging, and production namespaces side by side, each with its own quota and RBAC. Minimal infrastructure duplication, but a bug in cluster-wide configuration, a runaway workload, or a control-plane incident can affect every environment simultaneously.
- **Hybrid.** In practice, the most common enterprise pattern: production gets its own dedicated

cluster (or clusters, for regional HA), isolated from everything else, while development and staging share a second, smaller cluster split by namespace. This captures most of the isolation benefit where it matters most — protecting production from non-production activity — without paying full duplication cost for lower-stakes environments.

Whichever topology you choose, the image that runs in each environment should be **the same image, promoted forward, never rebuilt**. Build once in CI, push a single immutable artifact to Artifact Registry — tagged with a CI-generated identifier such as a commit SHA, and ideally referenced everywhere by its immutable digest — and deploy that exact artifact to staging, verify it, then deploy that exact same artifact to production:

```
# Build once, tag with the commit SHA
docker build -t
  ↪ us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:a1b2c3d .
docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:a1b2c3d

# Deploy the exact same tag to staging
kubectl set image deployment/app-deployment \
  app=us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:a1b2c3d \
  -n staging

# After verification, deploy the identical tag to production
kubectl set image deployment/app-deployment \
  app=us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:a1b2c3d \
  -n production
```

TIP

Reference images by digest (app@sha256:...) rather than by mutable tag wherever possible in production manifests. A tag can be overwritten to point at a different image; a digest is a content hash and cannot lie about what is actually running.

Rebuilding an image separately for each environment — even from identical source — reintroduces exactly the risk this pipeline is designed to eliminate: the possibility that “staging” and “production” are subtly different artifacts, built with different base image layers pulled on different days, undermining the entire premise of having tested staging in the first place.

Production Considerations

A production rollout is never just “does it run.” Four concerns deserve explicit, documented answers before a workload goes live.

NOTE

Cost. Right-size requests using real historical usage data from Cloud Monitoring rather than guesswork, and revisit them periodically as traffic patterns change. GKE Autopilot bills per Pod resource request and removes node management entirely, which suits variable or unpredictable workloads and smaller teams; GKE Standard gives full control over node pools and machine types and is typically cheaper at steady, high, predictable utilization. For steady-state baseline capacity in Standard clusters, committed-use discounts on the underlying Compute Engine node pools can meaningfully reduce cost compared to on-demand pricing.

BEST PRACTICE

High availability. Use a **regional** GKE cluster (control plane and nodes replicated across multiple zones in us-central1) rather than a zonal one, so a single zone outage does not take the cluster down. Spread node pools across at least three zones, and use Pod anti-affinity or topology spread

constraints so replicas of the same Deployment are not all scheduled onto the same node or the same zone — otherwise a single node or zone failure can take out every replica of a “highly available” Service at once.

WARNING

Security. Enable Workload Identity so Pods authenticate to GCP APIs as scoped, dedicated identities instead of a broadly-privileged node service account. Enforce Binary Authorization so only images signed by a trusted build pipeline can be deployed, closing the loop with the vulnerability-gated Artifact Registry repository described earlier in this chapter. Run production node pools as private nodes inside a private cluster, so nodes have no public IP addresses and are unreachable directly from the internet.

NOTE

Backup and disaster recovery. Use GKE Backup for GKE (or Velero, for portability across clusters and clouds) to take scheduled backups of both Kubernetes object configuration and any attached persistent volume data. A backup that has never been restored is not a verified backup — schedule regular restore-drill exercises into a scratch cluster or namespace, and keep the restore runbook itself as carefully documented and version-controlled as the application manifests it protects.

Cost, availability, security, and disaster recovery are not independent boxes to check — they trade against each other constantly. A third availability zone raises baseline cost; a private cluster complicates certain CI/CD network paths; a stricter Binary Authorization policy slows down an emergency hotfix. Treating them as one connected design conversation, revisited whenever the architecture changes, is what separates a system that merely runs from one that is genuinely production-ready.

Key Takeaways

- A complete production architecture connects a Global HTTP(S) Load Balancer, GKE Ingress, Service, and HPA-scaled Pods in a production namespace, pulling from a scanned, gated Artifact Registry repository, all observed by Cloud Monitoring and Cloud Logging.
- Namespaces isolate workloads, RBAC, and quotas within a single cluster at low overhead; fully separate clusters trade that efficiency for stronger isolation when regulatory or trust boundaries demand it.
- Accurate requests and limits drive both a Pod’s QoS class (Guaranteed, Burstable, BestEffort) and the decisions the scheduler and Cluster Autoscaler make — understating requests causes contention and unpredictable scaling, not savings.
- A hybrid multi-environment strategy — an isolated production cluster alongside a shared, namespace-split non-production cluster — balances isolation against operational overhead for most organizations.
- The same image, promoted by immutable digest or CI-assigned tag from staging to production, preserves the guarantee that what was tested is exactly what is deployed; rebuilding per environment breaks that guarantee.
- Cost, high availability, security, and disaster recovery are interdependent design decisions, not independent checklists, and should be revisited together as the architecture evolves.

Chapter 23: Interview Questions

Docker Interview Questions (1-16)

Q1. What is the difference between a Docker image and a Docker container?

An image is a read-only, layered template that packages an application, its dependencies, and the metadata needed to run it (entrypoint, environment variables, exposed ports). A container is a running (or stopped) instance of an image, created by adding a thin writable layer on top of the image's read-only layers. You can create many containers from a single image, and each container has its own isolated filesystem changes, network namespace, and process tree while still sharing the underlying image layers. Deleting a container does not delete the image it was created from.

Q2. What are the most important instructions in a Dockerfile and what do they do?

FROM sets the base image; RUN executes a command at build time and commits the result as a new layer; COPY/ADD copy files from the build context into the image (ADD also supports remote URLs and auto-extraction of archives, which is why COPY is generally preferred for clarity); WORKDIR sets the working directory for subsequent instructions; EXPOSE documents the port the container listens on; ENV sets environment variables available at build and run time; and CMD/ENTRYPOINT define what runs when the container starts. ARG defines build-time-only variables passed via --build-arg, unlike ENV which persists into the running container.

Q3. What are Docker image layers and how does the build cache work?

Each instruction in a Dockerfile (except a few like LABEL) produces a new filesystem layer, and images are built as a stack of these read-only layers plus content-addressable metadata. When you rebuild an image, Docker checks each instruction against its cache: if the instruction and its build context (e.g., the files being copied) are unchanged, Docker reuses the cached layer instead of re-executing it. As soon as one layer's cache is invalidated, every subsequent layer must be rebuilt, which is why Dockerfiles order rarely-changing instructions (installing OS packages) before frequently-changing ones (copying application source code).

Q4. What is a multi-stage build and why is it used?

A multi-stage build uses multiple FROM statements in one Dockerfile, where each stage can have its own base image, and later stages selectively copy artifacts from earlier stages using COPY --from=<stage>. This lets you compile or build an application in a stage with full build tooling (compilers, SDKs, dev dependencies) and then copy only the compiled binary or built assets into a minimal final runtime image, such as distroless or alpine. The result is a much smaller, more secure production image because build tools, source code, and intermediate artifacts never end up in the final layer set.

Q5. What is the difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the fixed executable that always runs when the container starts, while CMD provides default arguments to that executable (or the default command if no ENTRYPOINT is set) that can be overridden by arguments passed to docker run. When both are specified in exec form, CMD's values are appended as arguments to ENTRYPOINT, giving you a container that behaves like a configurable CLI tool with sensible defaults. If only CMD is set, docker run <image> <args> fully replaces the command; if ENTRYPOINT is set, the same invocation only replaces the arguments.

Q6. What is the difference between a Docker volume and a bind mount?

A volume is storage managed entirely by Docker, typically stored under `/var/lib/docker/volumes/` on the host, created and referenced by name, and portable across container recreation without depending on the host's directory layout. A bind mount maps a specific path on the host filesystem directly into the container, giving direct visibility and control to the host but tying the container to that host's directory structure, which makes it less portable. Volumes are generally preferred for production data persistence and for sharing data between containers, while bind mounts are common in local development to sync source code for live reload.

Q7. What are Docker's built-in networking modes?

bridge is the default mode, creating an isolated private network per Docker host where containers get their own IP and communicate through NAT and port mapping. host mode removes network isolation, so the container shares the host's network namespace directly, giving lower latency but no port isolation. none disables networking entirely for the container. overlay networks span multiple Docker hosts and are used in Swarm mode for multi-host container communication, and macvlan assigns containers a MAC address so they appear as physical devices directly on the network.

Q8. What is the difference between an image tag and an image digest?

A tag (e.g., `us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/app:1.2.0`) is a mutable, human-readable label that can be reassigned to point to a different underlying image at any time, including the latest tag which is just a convention, not a guarantee of the newest version. A digest (e.g., `app@sha256:abcd...`) is an immutable, content-addressable SHA256 hash of the image manifest, so pulling by digest always retrieves the exact same bytes. Production deployments and CI/CD pipelines should pin to digests, not floating tags, to guarantee reproducibility and prevent silent drift when a tag is overwritten.

Q9. What is the purpose of a .dockerignore file?

`.dockerignore` excludes specified files and directories from the build context sent to the Docker daemon, using the same glob pattern syntax as `.gitignore`. This speeds up builds by reducing the amount of data transferred to the daemon, prevents accidentally copying secrets, `.git` directories, local `node_modules`, or build artifacts into the image via broad `COPY . .` instructions, and reduces cache invalidation caused by irrelevant file changes. It is one of the simplest and highest-impact optimizations for both build speed and image security.

Q10. Why should containers run as a non-root user, and how do you configure it?

Running as root inside a container means that a container breakout or kernel-level vulnerability grants the attacker root privileges, and any bind-mounted host files created by the process will be owned by root on the host too. You mitigate this by creating a dedicated user in the Dockerfile (`RUN groupadd -r app && useradd -r -g app app`) and switching to it with `USER app` before the final `CMD/ENTRYPOINT`, or by using base images that already define a non-root user like many official `-slim` or `distroless` images. Orchestrators like Kubernetes can also enforce this at the platform level via `securityContext.runAsNonRoot: true` and `runAsUser`, rejecting pods that try to run as UID 0.

Q11. What are the main techniques for reducing Docker image size?

Use a minimal base image such as `alpine`, `slim` variants, or `distroless` instead of a full OS image; combine related `RUN` commands with `&&` and clean up package manager caches (`apt-get clean`, `rm -rf /var/lib/apt/lists/*`) within the same layer so the cleanup doesn't just add another layer on top of the bloat. Use multi-stage builds to discard build-time dependencies from the final image, order instructions to maximize cache reuse, and use `.dockerignore` to avoid copying unnecessary

files. Tools like `docker image ls`, `dive`, or `docker history` help identify which layers contribute the most size so they can be targeted for optimization.

Q12. What is Docker Compose and what problem does it solve?

Docker Compose is a tool for defining and running multi-container applications using a single declarative YAML file (`docker-compose.yml`), where each service, its image or build context, environment variables, networks, volumes, and dependencies are specified together. Instead of manually running multiple `docker run` commands with matching network and volume flags, `docker compose up` builds and starts the entire stack, wires services onto a shared network where they can reach each other by service name, and `docker compose down` tears it all down cleanly. It is widely used for local development environments and simple single-host deployments, though it is not intended to replace an orchestrator like Kubernetes for production-scale, multi-host workloads.

Q13. How do Docker health checks work?

A `HEALTHCHECK` instruction in the Dockerfile (or the equivalent `healthcheck` block in Compose) defines a command that Docker periodically runs inside the container, such as `curl -f http://localhost/health || exit 1`, along with `--interval`, `--timeout`, `--start-period`, and `--retries` parameters. Docker uses the command's exit code to mark the container's status as starting, healthy, or unhealthy, which is visible in `docker ps` and can be queried programmatically via `docker inspect`. This status doesn't restart the container by itself in plain Docker, but orchestrators and monitoring tools use it to decide whether to route traffic to, restart, or replace the container.

Q14. What happens during a container's lifecycle, and how are signals handled at shutdown?

A container transitions through created, running, paused, and stopped/exited states, with `docker run` triggering the entrypoint process as PID 1 inside its own PID namespace. On `docker stop`, Docker sends `SIGTERM` to PID 1 to request graceful shutdown, waits for a configurable grace period (default 10 seconds), and then sends `SIGKILL` if the process hasn't exited. A common pitfall is that shell-form `CMD/ENTRYPOINT` (e.g., `CMD myapp`) wraps the process in `/bin/sh -c`, which does not forward signals to the child process, so applications should use exec form (`CMD ["myapp"]`) or a lightweight init like `tini` to ensure `SIGTERM` reaches the actual application and enables clean shutdown.

Q15. What restart policies does Docker support?

`no` is the default, meaning the container never restarts automatically regardless of exit status. `on-failure[:max-retries]` restarts the container only if it exits with a non-zero status, optionally capped at a maximum retry count. `always` restarts the container whenever it stops, including after a manual stop if the Docker daemon itself restarts, while `unless-stopped` behaves the same but will not restart the container if it was explicitly stopped by the user before the daemon restarted. These are set via the `--restart` flag on `docker run` or the `restart` key in Compose, and are typically used for standalone Docker deployments rather than orchestrated environments where the scheduler handles restarts instead.

Q16. How does Docker registry authentication work, and how do you authenticate to a private registry like Artifact Registry?

Registry authentication is handled via `docker login <registry-host>`, which stores an encoded credential (ideally backed by a credential helper rather than plaintext) in `~/.docker/config.json`, and this token is presented on every subsequent push or pull against that registry host. For Google

Artifact Registry, the standard approach is to run `gcloud auth configure-docker us-central1-docker.pkg.dev`, which registers `gcloud` itself as a Docker credential helper so that `docker push` and `docker pull` transparently use the caller's Google Cloud identity and IAM permissions instead of a static password. In CI/CD pipelines and on GKE nodes, this is typically backed by a service account key or, preferably, Workload Identity, avoiding long-lived credentials stored on disk.

Kubernetes Interview Questions (1-16)

Q1. What is a Pod, and why does Kubernetes not schedule containers directly?

A Pod is the smallest deployable unit in Kubernetes, wrapping one or more containers that share the same network namespace (a single IP address and port space), the same set of storage volumes, and are always scheduled together onto the same node. Kubernetes schedules Pods rather than individual containers because tightly coupled processes, such as a main application container and a logging or proxy sidecar, often need to share localhost networking and a filesystem, and treating them as one schedulable unit guarantees they are co-located and lifecycle-managed together. Multi-container Pods commonly use the sidecar, ambassador, or adapter patterns to add cross-cutting functionality without modifying the main application container.

Q2. What is the difference between a Deployment, a ReplicaSet, and a StatefulSet?

A ReplicaSet's sole job is to ensure a specified number of identical Pod replicas are running at any time, replacing Pods that fail or are deleted, but it is rarely created directly. A Deployment manages ReplicaSets on top of that, adding declarative rolling updates, rollback history, and revision tracking, so updating a Deployment's Pod template creates a new ReplicaSet and gradually shifts traffic to it. A StatefulSet is used for workloads that need stable, unique network identities and persistent per-Pod storage, such as databases: Pods get predictable names like `web-0`, `web-1` with stable DNS entries, and each Pod is bound to its own PersistentVolumeClaim that survives rescheduling, unlike the interchangeable, identity-less Pods of a Deployment.

Q3. What is a DaemonSet used for?

A DaemonSet ensures that a copy of a specific Pod runs on every node in the cluster (or on a filtered subset matching a node selector), and automatically adds or removes that Pod as nodes join or leave the cluster. It's the standard mechanism for deploying node-level infrastructure such as log collectors (Fluentd/Fluent Bit), metrics agents (node-exporter), network plugins (CNI daemons), or storage plugins that must run exactly once per node rather than being scaled independently like a normal application workload.

Q4. What are the main Kubernetes Service types and when do you use each?

ClusterIP (the default) exposes the Service only on an internal cluster IP, reachable from within the cluster, suitable for internal service-to-service communication. NodePort opens a static port (30000-32767) on every node's IP, forwarding to the Service, useful for simple external access or debugging. LoadBalancer provisions a cloud provider's external load balancer (e.g., a GCP Network Load Balancer) that routes external traffic to the Service, commonly used for internet-facing production endpoints. ExternalName maps a Service to an external DNS name via a CNAME record with no proxying, used to give in-cluster workloads a stable name for an external dependency.

Q5. What is the difference between an Ingress and a Service?

A Service provides stable networking and load balancing at Layer 4 for a set of Pods identified by a label selector, but it has no concept of HTTP routing rules, hostnames, or paths. An Ingress is a

Layer 7 (HTTP/HTTPS) routing resource that sits in front of one or more Services, defined declaratively with rules mapping hostnames and URL paths to backend Services, and typically also handles TLS termination; it requires an Ingress controller (like GKE's native controller or NGINX) to actually implement those rules by provisioning and configuring a real load balancer. In effect, Ingress lets a single external IP and load balancer serve many Services via host- or path-based routing, instead of provisioning a separate LoadBalancer Service for each one.

Q6. What is the difference between a ConfigMap and a Secret?

A ConfigMap stores non-sensitive configuration data as key-value pairs, such as feature flags, URLs, or configuration files, and its contents are stored in etcd as plain text and visible unencoded via `kubectl get configmap -o yaml`. A Secret is intended for sensitive data such as passwords, API keys, or TLS certificates; its values are base64-encoded (not encrypted by default) in the API but are usually stored encrypted at rest by the cluster when encryption-at-rest is enabled, and Kubernetes takes extra precautions such as avoiding writing Secrets to disk on nodes when using tmpfs-backed volume mounts. Both can be mounted as volumes or exposed as environment variables to Pods, but Secrets should additionally be protected via RBAC so only authorized workloads and users can read them, and ideally integrated with an external secret manager for stronger security.

Q7. Explain PersistentVolumes, PersistentVolumeClaims, and StorageClasses.

A PersistentVolume (PV) is a cluster-level storage resource, either provisioned in advance by an administrator (static provisioning) or created on demand, that represents an actual piece of storage like a GCE Persistent Disk. A PersistentVolumeClaim (PVC) is a namespaced request for storage made by a user or Pod, specifying size and access mode, which Kubernetes binds to a matching available PV. A StorageClass defines a class of storage with a specific provisioner (e.g., `pd.csi.storage.gke.io`) and parameters (disk type, replication), enabling dynamic provisioning so that a PVC referencing a StorageClass automatically triggers creation of a new PV that satisfies the request, rather than relying on pre-created PVs.

Q8. What are namespaces used for in Kubernetes?

Namespaces provide a way to logically partition a single cluster into multiple virtual clusters, scoping most resource names (Pods, Services, ConfigMaps, etc.) so the same name can be reused across namespaces without collision. They are the primary boundary for applying RBAC policies, ResourceQuotas, and LimitRanges to isolate teams or environments (e.g., `dev`, `staging`, `prod`) on the same physical cluster, and they scope DNS as well, since Services get a fully qualified name like `my-service.my-namespace.svc.cluster.local`. Cluster-scoped resources like Nodes, PersistentVolumes, and ClusterRoles are not namespaced, since they represent cluster-wide infrastructure.

Q9. How do labels and selectors work in Kubernetes?

Labels are arbitrary key-value pairs attached to objects like Pods, Nodes, and Services (e.g., `app: frontend`, `tier: web`, `env: prod`) used purely for identification and organization, not to convey uniqueness. Selectors are queries against those labels: Services and Deployments use equality-based selectors (`app=frontend`) to determine which Pods they manage or route traffic to, while more expressive set-based selectors (`environment in (prod, staging)`) support `kubectl` filtering and NetworkPolicy rules. This decoupled label-selector model is fundamental to how Kubernetes dynamically groups and targets resources instead of relying on fixed references.

Q10. What is the difference between liveness, readiness, and startup probes?

A liveness probe determines whether a container is still functioning correctly; if it fails repeatedly, the kubelet kills and restarts the container, which is useful for recovering from deadlocks that a

running process can't self-heal from. A readiness probe determines whether a container is ready to accept traffic; if it fails, the Pod is removed from the Endpoints of any Service targeting it (but the container is not restarted), which is essential during startup or when a Pod is temporarily overloaded. A startup probe is used for slow-starting applications, disabling liveness and readiness checks until the startup probe itself succeeds, preventing the kubelet from prematurely killing a container that just needs more time to initialize than the liveness probe's threshold would normally allow.

Q11. How do rolling updates and rollbacks work for a Deployment?

When a Deployment's Pod template changes, Kubernetes creates a new ReplicaSet and incrementally scales it up while scaling the old ReplicaSet down, governed by the `strategy.rollingUpdate.maxSurge` (how many extra Pods can be created above the desired count) and `maxUnavailable` (how many Pods can be unavailable during the rollout) settings, combined with readiness probes to ensure only healthy new Pods receive traffic before old ones are terminated. Kubernetes retains a configurable number of previous ReplicaSets (via `revisionHistoryLimit`) as rollout history, so `kubectl rollout undo deployment/my-app` reverts the Deployment to the previous (or a specific `--to-revision`) ReplicaSet, triggering another rolling update in reverse. `kubectl rollout status` and `kubectl rollout history` are used to monitor progress and inspect past revisions respectively.

Q12. How does the Horizontal Pod Autoscaler (HPA) work?

The HPA controller periodically (by default every 15 seconds) queries the Metrics Server (or a custom/external metrics adapter) for current resource utilization, such as average CPU or memory usage across a Deployment's Pods, or custom metrics like requests-per-second. It compares the observed value to the target specified in the HPA object and computes the desired replica count using the formula $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} * \text{currentMetricValue} / \text{desiredMetricValue})$, then updates the target Deployment's or StatefulSet's replica count accordingly, subject to configured min/max bounds and stabilization windows that prevent rapid flapping. Because HPA relies on resource requests being set to compute utilization percentages, Pods without requests defined cannot be scaled based on CPU or memory.

Q13. What are the core building blocks of Kubernetes RBAC?

A Role (namespace-scoped) or ClusterRole (cluster-scoped) defines a set of permissions as rules combining API groups, resources, and verbs (get, list, watch, create, update, delete), for example allowing get and list on pods in the apps API group. A RoleBinding grants the permissions in a Role (or ClusterRole) to a subject — a User, Group, or ServiceAccount — within a specific namespace, while a ClusterRoleBinding grants a ClusterRole's permissions cluster-wide across all namespaces. This model follows least-privilege principles: workloads should run under dedicated ServiceAccounts bound only to the narrow set of permissions they actually require, rather than using the default ServiceAccount or cluster-admin bindings.

Q14. What are the roles of the API server, etcd, scheduler, and controller manager?

The kube-apiserver is the single entry point to the cluster's control plane, exposing the Kubernetes REST API, handling authentication/authorization/admission control, and validating and persisting all object changes. etcd is the distributed, strongly consistent key-value store that serves as the cluster's single source of truth, holding the entire state of every object; the API server is the only component that talks to etcd directly. The kube-scheduler watches for newly created Pods with no assigned node and selects the best-fit node for each based on resource requirements, affinity/anti-affinity rules, taints/tolerations, and topology constraints. The kube-controller-manager runs the many reconciliation control loops (Deployment controller, Node controller, ReplicaSet controller, etc.) that

continuously compare actual cluster state to desired state and issue changes through the API server to converge them.

Q15. What are some essential kubectl commands and what do they do?

`kubectl get <resource> [-n namespace] [-o wide|yaml|json]` lists resources and their status; `kubectl describe <resource> <name>` shows detailed configuration plus recent events, invaluable for debugging scheduling or startup failures. `kubectl logs <pod> [-c container] [-f]` streams container logs, and `kubectl exec -it <pod> -- /bin/sh` opens an interactive shell inside a running container. `kubectl apply -f <file.yaml>` declaratively creates or updates resources from a manifest (the standard way to manage resources, as opposed to imperative `kubectl create/kubectl run`), and `kubectl rollout status/undo/history deployment/<name>` manages Deployment roll-outs. `kubectl get events --sort-by=.lastTimestamp` and `kubectl top pods/nodes` (requiring Metrics Server) round out the common troubleshooting toolkit.

Q16. Why are resource requests and limits important, and what happens if you omit them?

requests tell the scheduler the minimum CPU/memory a container needs to be scheduled onto a node with sufficient capacity, and they also form the baseline the kubelet guarantees via cgroups; limits cap the maximum a container can consume, with CPU being throttled and memory over-limit resulting in the container being OOMKilled. Together they determine a Pod's QoS class: Guaranteed (requests equal limits for all containers), Burstable (requests set but below limits, or only some containers have both), or BestEffort (no requests or limits set at all), which directly affects eviction priority under node resource pressure — BestEffort Pods are evicted first. Omitting requests/limits means the Pod is BestEffort, can be scheduled onto any node regardless of actual capacity, may starve other workloads or get evicted first during contention, and breaks HPA's ability to compute CPU/memory utilization percentages.

GKE & Artifact Registry Interview Questions (1-16)

Q1. What is the difference between GKE Autopilot and GKE Standard?

In GKE Standard, you provision and manage node pools yourself, choosing machine types, node counts, and configuring autoscaling, and you are billed for the underlying VM infrastructure regardless of Pod-level utilization, giving maximum flexibility including DaemonSets, privileged Pods, and custom node configurations. GKE Autopilot is a fully managed mode where Google provisions and manages nodes automatically based on the resource requests declared in your workloads, bills per Pod resource consumption rather than per node, and enforces a hardened, opinionated security baseline (no privileged containers, restricted hostPath usage, mandatory resource requests). Autopilot trades some flexibility and typically higher per-unit compute cost for significantly reduced operational overhead, making it attractive for teams that don't want to manage node infrastructure.

Q2. What is a node pool in GKE?

A node pool is a group of nodes within a GKE cluster that all share the same configuration, including machine type, disk type/size, OS image, labels, taints, and autoscaling settings. A single cluster can have multiple node pools, for example a general-purpose pool for standard workloads, a GPU-enabled pool for ML inference, and a spot-VM pool for cost-sensitive batch jobs, with workloads targeted to specific pools using node selectors, taints/tolerations, or affinity rules. Node pools can be independently resized, upgraded, or deleted, which allows rolling out new node configurations or Kubernetes versions to one pool at a time without affecting the whole cluster.

Q3. What does it mean for a GKE cluster to be VPC-native, and why is it preferred?

A VPC-native cluster (using alias IP ranges) allocates Pod and Service IP addresses from secondary IP ranges of the VPC subnet itself, so Pod IPs are natively routable within the VPC and to peered networks without requiring custom routes or NAT for east-west traffic. This is contrasted with the legacy routes-based networking model, which required Google Cloud to manage a custom route per node and had scaling limitations (a hard cap on routes per VPC). VPC-native clusters are required for many modern GKE features, including private clusters, Google Cloud Armor integration with GKE Ingress, and direct integration with VPC Firewall rules and VPC Service Controls, and have been the default for new clusters for years due to their scalability and security advantages.

Q4. What is Workload Identity and why is it preferred over service account key files?

Workload Identity lets a Kubernetes ServiceAccount in a GKE cluster impersonate a Google Cloud IAM service account, so Pods can call Google Cloud APIs (Cloud Storage, Artifact Registry, BigQuery) with the permissions of that IAM service account without ever storing a downloaded JSON key file inside the cluster. It works by federating the cluster's identity through the `<project>.svc.id.goog` workload identity pool and binding the Kubernetes ServiceAccount to the IAM service account via `roles/iam.workloadIdentityUser`, with credentials issued as short-lived tokens fetched transparently by the GKE metadata server. This eliminates the risk of leaked, long-lived static keys, removes manual key rotation, and lets you scope permissions precisely per-workload rather than per-node, which was the major weakness of the older node-level service account approach.

Q5. What is Binary Authorization in GKE?

Binary Authorization is a deploy-time security control that enforces a policy requiring container images to be signed and attested by trusted authorities before GKE will admit them to run in the cluster. Typically this is wired into a CI/CD pipeline so that an image only receives an attestation after passing required checks such as vulnerability scanning, unit tests, or a QA sign-off, and the cluster's admission webhook rejects any Pod referencing an unattested image or an image from an untrusted registry. It's a key control for preventing arbitrary or compromised images from being deployed, especially in regulated environments, and pairs naturally with Artifact Registry's vulnerability scanning results as one of the attestation gates.

Q6. What is the difference between GKE's native Ingress controller and the Gateway API?

GKE's built-in Ingress controller consumes the standard Kubernetes Ingress resource and provisions Google Cloud load balancers (external or internal HTTP(S) load balancers) based on annotations for features like SSL policies, Cloud Armor, or IAP; however, the Ingress API itself is fairly limited and vendor-specific behavior has to be expressed through non-standard annotations. The Gateway API is a newer, more expressive, portable Kubernetes API (Gateway, HTTPRoute, GatewayClass) that models traffic routing with role-oriented resources, natively supports advanced use cases like traffic splitting, header-based routing, and cross-namespace routing without vendor-specific annotations, and GKE offers a managed Gateway controller implementing it. Google has positioned Gateway API as the long-term direction for ingress traffic management on GKE, offering more granular control and multi-cluster support compared to the older Ingress resource.

Q7. What is the difference between the Cluster Autoscaler and node auto-provisioning (NAP) in GKE?

The Cluster Autoscaler automatically resizes existing node pools up or down based on Pod scheduling pressure and underutilization, adding nodes when Pods can't be scheduled due to insufficient resources and removing nodes when they're underutilized and their Pods can be safely rescheduled elsewhere, but it only operates within the node pools you've already defined. Node auto-provisioning

goes a step further by automatically creating and deleting entire node pools with appropriate machine types, sized to match the specific resource and scheduling requirements (taints, GPU needs) of pending Pods, without an administrator having to pre-create a matching pool. NAP is especially useful for clusters with highly variable or unpredictable workload shapes, whereas standard Cluster Autoscaler is sufficient when the set of required node configurations is known in advance.

Q8. What is a private GKE cluster and what network topology does it enable?

In a private cluster, nodes are provisioned with only internal IP addresses (no public IPs), which prevents any inbound internet access to the nodes and significantly reduces the cluster's attack surface, while the control plane can be configured as either public (accessible via authorized networks) or fully private, reachable only from within the VPC or via VPC peering/Private Service Connect. Outbound internet access from nodes, when needed for pulling public images or reaching external APIs, is typically routed through Cloud NAT rather than direct node-level public IPs. This topology is standard for production and regulated environments, and is commonly combined with VPC Service Controls, private Google access, and authorized networks on the control plane endpoint for defense in depth.

Q9. What is the difference between Artifact Registry and the deprecated Container Registry?

Container Registry (`gcr.io`) was Google's original single-purpose Docker image registry backed transparently by Cloud Storage buckets, offering limited IAM granularity (essentially bucket-level permissions) and support for only Docker images. Artifact Registry is the successor, offering per-repository IAM permissions, regional and multi-regional repository placement for lower latency and data residency control, native vulnerability scanning, and support for multiple package formats beyond Docker (Maven, npm, Python, APT, YUM, Go modules) all within one service. Google deprecated Container Registry (March 2025 shutdown of new usage, with existing images migrated) in favor of Artifact Registry, so all new projects should provision repositories in Artifact Registry, for example `us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo`.

Q10. What repository formats does Artifact Registry support, and how is access controlled?

Artifact Registry supports multiple repository formats within a single service: Docker (container images), Maven and Gradle (Java), npm (Node.js), Python (pip-installable packages), Apt and Yum (Linux OS packages), and Go modules, each created as a distinct repository resource scoped to a project, region, and format. Access is controlled through Cloud IAM at the project or individual-repository level using predefined roles such as `roles/artifactregistry.reader` (pull), `roles/artifactregistry.writer` (push and pull), and `roles/artifactregistry.admin` (full management including repository creation and deletion), which can be granted to users, groups, or service accounts. This per-repository granularity lets you, for example, give a CI/CD service account write access only to a specific repository like `my-repo` while limiting production deployment pipelines to read-only access.

Q11. How does vulnerability scanning work in Artifact Registry?

Artifact Registry integrates with the On-Demand Scanning and Container Analysis APIs to automatically scan container images for known vulnerabilities (CVEs) against public vulnerability databases whenever an image is pushed, and continuously rescans stored images as new vulnerability data is published. Scan results are attached as metadata (occurrences) to the image and are viewable in the console or queryable via `gcloud artifacts docker images list --show-occurrences`, typically categorized by severity (critical, high, medium, low). These results are commonly wired into CI/CD gates or Binary Authorization policies so that images with critical or high-severity vulnerabilities above a defined threshold are automatically blocked from being promoted to production.

Q12. What is a recommended image promotion strategy across dev, staging, and production environments?

A robust strategy builds the image exactly once, tags it immutably (ideally referencing it by digest, not a mutable tag, in downstream deployments), and pushes it to a single Artifact Registry repository, then promotes the same digest across environments rather than rebuilding per environment, which guarantees that what was tested in staging is bit-for-bit identical to what runs in production. This is often implemented with separate repositories or a promotion mechanism that copies/re-tags the image from a dev or ci repository into a staging and then prod repository (or uses tags like `:candidate` and `:release` within one repo) only after passing automated tests, vulnerability scan thresholds, and any required approvals or Binary Authorization attestations. Environment-specific configuration should be externalized (ConfigMaps, Secrets, environment variables) rather than baked into the image, so the same artifact truly is portable across environments.

Q13. How do GKE cluster upgrades and release channels work?

GKE release channels — Rapid, Regular, and Stable — let you opt into a curated, Google-tested cadence of Kubernetes version upgrades instead of pinning to a specific static version: Rapid gets new minor versions soonest with less soak time, Stable receives versions only after extended validation in production fleets, and Regular balances currency with stability, which is the default for most production clusters. Within a channel, GKE automatically upgrades the control plane and, if enabled, node pools during configurable maintenance windows/exclusions, and supports surge upgrades (temporarily creating extra nodes) to upgrade node pools with minimal workload disruption. For clusters requiring precise control, you can also manage versions statically outside a channel, but you then take on responsibility for scheduling and testing upgrades yourself, including tracking the Kubernetes deprecation policy for removed APIs.

Q14. What are the primary levers for cost optimization on GKE?

Right-sizing workloads by setting accurate resource requests (avoiding both over-provisioning waste and under-provisioning that forces early scale-out) is foundational, combined with the Cluster Autoscaler and, for variable workloads, GKE Autopilot's pay-per-Pod-resource model to avoid paying for idle node capacity. Using Spot VMs for fault-tolerant, interruptible workloads (batch jobs, stateless workers) can cut compute costs significantly compared to standard on-demand nodes, and committed use discounts or GKE's Autopilot cost-saving flexible resource options reduce costs for stable, predictable baseline capacity. Additional levers include enabling vertical Pod autoscaling recommendations to catch chronically over-requested workloads, scheduling non-critical workloads to scale down or pause during off-hours, and choosing regional storage/machine types appropriate to actual latency and durability requirements rather than defaulting to the most expensive tier.

Q15. What is the difference between a regional and a zonal GKE cluster?

A zonal cluster runs its control plane in a single zone (with worker nodes optionally spread across that one zone or multiple zones in a multi-zone zonal cluster), meaning a zonal outage affecting the control plane's zone causes a control plane outage, though running workloads typically continue operating. A regional cluster replicates the control plane across three zones within a region (e.g., `us-central1`) for high availability, so control plane operations like `kubectl` apply remain available even if one zone fails, and by default also spreads nodes across multiple zones for workload resilience. Regional clusters have a higher baseline cost due to the replicated control plane and node footprint, but are the recommended choice for production workloads that require high availability guarantees.

Q16. How would you set up secure image push/pull between CI/CD, Artifact Registry, and a GKE cluster?

In CI/CD, avoid long-lived JSON key files by using Workload Identity Federation (for external CI systems like GitHub Actions) or a GKE-hosted CI runner using Workload Identity, so the pipeline obtains short-lived Google credentials scoped to a service account with `roles/artifactregistry.writer` on the target repository, then authenticates with `gcloud auth configure-docker us-central1-docker.pkg.dev` before pushing to `us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo`. On the GKE side, nodes and Pods pull images using the node's default service account or, preferably, a per-workload Workload Identity binding with only `roles/artifactregistry.reader`, following least privilege so pull access is separated from push access. Layering Binary Authorization on top ensures that even with valid pull credentials, only images that carry the required build and scan attestations are actually admitted to run in the cluster.

Chapter 24: Commands Cheat Sheet

This chapter is a quick-reference companion for the rest of the book. It does not teach new concepts — it collects the commands you have already learned into three scannable tables so you can look things up during real work without paging back through earlier chapters. Keep this chapter bookmarked; it is the page you will return to most often.

Docker Commands

Command	Purpose	Example
<code>docker build</code>	Build an image from a Dockerfile in the current build context	<code>docker build -t my-repo/web-app:1.0 .</code>
<code>docker run</code>	Create and start a container from an image	<code>docker run -d -p 8080:80 --name web my-repo/web-app:1.0</code>
<code>docker ps</code>	List running containers (add <code>-a</code> for all, including stopped)	<code>docker ps -a</code>
<code>docker exec</code>	Run a command inside a running container	<code>docker exec -it web /bin/sh</code>
<code>docker logs</code>	View a container's stdout/stderr output	<code>docker logs -f web</code>
<code>docker stop</code>	Gracefully stop a running container	<code>docker stop web</code>
<code>docker rm</code>	Remove a stopped container	<code>docker rm web</code>
<code>docker images</code>	List locally stored images	<code>docker images</code>
<code>docker rmi</code>	Remove a local image	<code>docker rmi my-repo/web-app:1.0</code>
<code>docker tag</code>	Create an additional tag/reference for an image	<code>docker tag web-app:1.0 us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/web-app:1.0</code>
<code>docker push</code>	Upload an image to a remote registry	<code>docker push us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/web-app:1.0</code>
<code>docker pull</code>	Download an image from a remote registry	<code>docker pull us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/web-app:1.0</code>
<code>docker system prune</code>	Remove unused data (stopped containers, dangling images, unused networks)	<code>docker system prune</code>
<code>docker inspect</code>	Show detailed JSON metadata about an object (image, container, network, volume)	<code>docker inspect web</code>

WARNING

`docker system prune -a` removes *all* unused images, not just dangling ones, and can also delete build cache and unused volumes if you add `--volumes`. On a shared build machine this can wipe out images your teammates rely on locally. Run it without `-a` first, review what it reports, and only escalate to `-a` when you are certain nothing important depends on those images.

TIP

Add `--no-cache` to `docker build` only when you suspect stale layers are causing a bug — it disables Docker’s layer cache entirely and will noticeably slow down every rebuild. For day-to-day work, let the cache do its job.

kubectl Commands

Command	Purpose	Example
<code>kubectl apply</code>	Create or update resources declared in a manifest file	<code>kubectl apply -f deployment.yaml</code>
<code>kubectl get</code>	List one or more resources and their current state	<code>kubectl get pods -n production</code>
<code>kubectl describe</code>	Show detailed information and recent events for a resource	<code>kubectl describe pod web-app-7d8f9c-abcde</code>
<code>kubectl logs</code>	View logs from a pod or a specific container within it	<code>kubectl logs web-app-7d8f9c-abcde -c web</code>
<code>kubectl exec</code>	Run a command inside a container in a pod	<code>kubectl exec -it web-app-7d8f9c-abcde -- /bin/sh</code>
<code>kubectl delete</code>	Remove a resource by name or by manifest file	<code>kubectl delete -f deployment.yaml</code>
<code>kubectl rollout status</code>	Watch the progress of a rolling deployment update	<code>kubectl rollout status deployment/web-app</code>
<code>kubectl rollout undo</code>	Roll back a deployment to its previous revision	<code>kubectl rollout undo deployment/web-app</code>
<code>kubectl scale</code>	Change the number of replicas for a deployment	<code>kubectl scale deployment/web-app --replicas=5</code>
<code>kubectl port-forward</code>	Forward a local port to a port on a pod or service	<code>kubectl port-forward svc/web-app 8080:80</code>
<code>kubectl top</code>	Show live CPU/memory usage for nodes or pods (requires metrics-server)	<code>kubectl top pods -n production</code>
<code>kubectl config current-context</code>	Show which cluster/context kubectl is currently pointed at	<code>kubectl config current-context</code>
<code>kubectl explain</code>	Show field-level documentation for a resource type	<code>kubectl explain deployment.spec.template</code>

BEST PRACTICE

Before applying a change you are unsure about, run it with `--dry-run=client -o yaml` to see exactly what Kubernetes would create or update without actually touching the cluster:

```
kubectl apply -f deployment.yaml --dry-run=client -o yaml
```

This habit catches typos in field names, indentation mistakes, and unexpected default values before they reach a live cluster — especially valuable when editing manifests for my-gke-cluster in a shared production namespace.

TIP

Always confirm your context with `kubectl config current-context` before running delete or scale commands. Switching between a local cluster and my-gke-cluster without noticing is one of the most common causes of accidental production changes.

gcloud Commands (Artifact Registry & GKE)

Command	Purpose	Example
<code>gcloud artifacts repositories create</code>	Create a new Artifact Registry repository	<code>gcloud artifacts repositories create my-repo --repository-format=docker --location=us-central1</code>
<code>gcloud artifacts repositories list</code>	List Artifact Registry repositories in the project	<code>gcloud artifacts repositories list --location=us-central1</code>
<code>gcloud auth configure-docker</code>	Configure Docker to authenticate with an Artifact Registry host	<code>gcloud auth configure-docker us-central1-docker.pkg.dev</code>
<code>gcloud container clusters create</code>	Create a GKE Standard cluster with manually managed node pools	<code>gcloud container clusters create my-gke-cluster --zone=us-central1-a --num-nodes=3</code>
<code>gcloud container clusters create-auto</code>	Create a GKE Autopilot cluster (nodes fully managed by Google)	<code>gcloud container clusters create-auto my-gke-cluster --region=us-central1</code>
<code>gcloud container clusters get-credentials</code>	Fetch cluster credentials and configure kubectl context	<code>gcloud container clusters get-credentials my-gke-cluster --region=us-central1</code>
<code>gcloud container clusters resize</code>	Change the node count of an existing node pool	<code>gcloud container clusters resize my-gke-cluster --node-pool=default-pool --num-nodes=5</code>
<code>gcloud artifacts docker images list</code>	List container images stored in an Artifact Registry repository	<code>gcloud artifacts docker images list us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo</code>
<code>gcloud artifacts docker images describe</code>	Show metadata and digests for a specific image	<code>gcloud artifacts docker images describe us-central1-docker.pkg.dev/my-gcp-project-123456/my-repo/web-app:1.0</code>

NOTE

`gcloud container clusters create` provisions a Standard-mode cluster where you manage node pools, machine types, and autoscaling settings yourself. `create-auto` provisions an Autopilot cluster where Google manages node infrastructure and you are billed per pod resource request instead of per node. Choose based on how much infrastructure control you need versus how much operational overhead you want to avoid.

With these three tables at hand, you should rarely need to search for basic command syntax again — the remaining chapters build on exactly these commands in real deployment scenarios.

Chapter 25: Summary

You have now traveled the full path from a single `docker run` command to a production-grade application running on Google Kubernetes Engine, with images stored, scanned, and versioned in Google Artifact Registry. This closing chapter steps back from the mechanics to recap what matters, propose a learning roadmap for revisiting the material, point to certifications that validate these exact skills, and leave you with one practical, money-saving habit.

Key Takeaways

- Containers solve the “it works on my machine” problem by packaging an application together with its runtime, libraries, and configuration into a single portable unit.
- Docker images are built in layers; understanding layer caching and writing a lean, multi-stage Dockerfile directly reduces image size, build time, and attack surface.
- A production-ready Dockerfile is not just a working Dockerfile — it uses non-root users, pinned base image versions, `.dockerignore` files, and multi-stage builds to separate build-time and run-time dependencies.
- Container registries are the handoff point between building an image and running it somewhere else; understanding tags, digests, and immutability prevents “which version is actually deployed” confusion.
- Google Artifact Registry extends that registry concept with IAM-based access control, regional storage, vulnerability scanning, and native integration with the rest of Google Cloud.
- Kubernetes introduces a declarative model: you describe the desired state (deployments, services, ingress, config) and the control plane continuously reconciles reality to match it.
- Pods, Deployments, Services, and Namespaces are the vocabulary of Kubernetes; almost everything else in the ecosystem builds on these four primitives.
- GKE removes the operational burden of running the Kubernetes control plane yourself, and Autopilot removes node management too — at the cost of some flexibility and a different billing model.
- Production architecture on GKE is about more than “it deploys” — it means health checks, resource requests/limits, horizontal pod autoscaling, rolling updates, secrets management, and observability are all in place before real traffic arrives.
- Infrastructure that is running costs money whether or not anyone is using it — this is true of GKE nodes, load balancers, and persistent disks alike, and cost discipline is as much a professional skill as writing a correct Dockerfile.

A Suggested Learning Roadmap

If you want to revisit this material in a structured way — for review, for teaching others, or for onboarding a new team member — follow this progression:

1. **Docker fundamentals** — images, containers, the Docker CLI, volumes, and networking basics.
2. **Production Dockerfiles** — multi-stage builds, minimizing image size, non-root users, and build-time secrets handling.
3. **Docker registries** — tagging strategy, pushing and pulling, and understanding registry authentication.
4. **GCP Artifact Registry** — repository creation, IAM permissions, vulnerability scanning, and migrating away from the deprecated Container Registry.
5. **Kubernetes fundamentals** — pods, deployments, services, config maps, secrets, and namespaces, ideally practiced on a local cluster (Minikube or kind) before touching a cloud cluster.
6. **GKE** — cluster creation (Standard versus Autopilot), node pools, `kubectl` context management, and workload deployment against a real managed cluster.
7. **Production architecture** — autoscaling, ingress and load balancing, observability, rolling updates and rollbacks, and cost management.

Each stage builds directly on the one before it, so resist the urge to skip straight to GKE without a solid grip on plain Docker and plain Kubernetes concepts first — most real-world debugging happens

at those lower layers.

Recommended Certifications

If you want external validation of the skills this book covers, the following certifications map closely to what you have learned:

- **Google Associate Cloud Engineer** — validates the ability to deploy, monitor, and manage applications on Google Cloud, including core services like Artifact Registry, Compute Engine, and GKE at a foundational operational level.
- **Google Professional Cloud DevOps Engineer** — validates the ability to build and operate CI/CD pipelines, manage service reliability, and run production workloads on Google Cloud with an emphasis on automation and monitoring.
- **Certified Kubernetes Application Developer (CKAD)** — validates the ability to design, build, and deploy cloud-native applications for Kubernetes, focusing on the developer's perspective: pods, deployments, services, and configuration.
- **Certified Kubernetes Administrator (CKA)** — validates the ability to install, configure, and manage a Kubernetes cluster, including networking, storage, security, and troubleshooting from the administrator's perspective.

BEST PRACTICE

Pursue certifications in the order your role demands rather than in a fixed sequence. A developer-focused reader benefits most from CKAD first; someone managing infrastructure day-to-day gets more immediate value from CKA or the Google Cloud certifications.

A Closing Note on Cost

One last, practical reminder before you close this book: GKE Standard-mode clusters keep their node VMs running, and any load balancer created by a Kubernetes Service of type LoadBalancer keeps its forwarding rule and external IP address allocated, for as long as those resources exist — regardless of whether you are actively sending traffic to them. Autopilot reduces but does not eliminate this; a running cluster with active pods still incurs charges for the resources those pods request.

WARNING

A forgotten `my-gke-cluster` left running over a weekend, or a load balancer left exposed after a demo, will keep billing your project quietly in the background. These are some of the most common sources of surprise GCP charges for people learning Kubernetes.

Make it a habit: when you are done studying or experimenting for the day, tear down what you created.

```
gcloud container clusters delete my-gke-cluster --region=us-central1
```

If you only need to pause spending temporarily rather than delete everything, resizing a Standard cluster's node pool to zero nodes is a faster, less destructive alternative that keeps your cluster configuration intact for next time. Either way, closing the loop on cleanup is the final skill this book asks of you — and it is the one that will save you the most money as you keep practicing everything else you have learned.